Facoltà di Ingegneria

Corso di Laurea in Ingegneria Informatica e dell'Automazione

Progetto di Programmazione Mobile

Applicazione GoRide kotlin-flutter

Autore:

HACHEM BELHOUCHETTE & AYMANE LAFDILI

Contents

1 Descrizione In Linguaggio Naturale **5**

2 Glossario **6**

3 Kotlin App **7**

3.1 Analisi dei Requisiti 7

3.1.1 REQUISITI FUNZIONALI 7

3.1.2 REQUISITI NON FUNZIONALI 8

3.2 Casi d'Uso 9

3.2.1 UC: Gestione Account 9

3.2.2 UC: Gestione Profilo 11

3.2.3 UC: Gestione Mappa e Mezzi 13

3.2.4 UC: Gestione Corsa 14

3.2.5 UC: Gestione Storico 17

3.2.6 UC: Gestione Community 18

3.3 Mappa dell'Architettura 20

3.3.1 Architettura dell'Avvio e della Navigazione 22

3.3.2 Architettura di Autenticazione e Profilo 22

3.3.3 Architettura della Mappa e dei Mezzi 23

3.3.4 Architettura della Corsa Attiva 24

3.3.5 Architettura dello Storico Corse 25

3.3.6 Architettura della Community 26

3.4 Data Storage 27

3.4.1 Database 27

3.4.2 Firebase 28

3.4.3 Sorgente Dati Mock 29

3.5 MockUp UI 30

3.5.1 NavBar 30

3.5.2 Sezione Account 31

3.5.3 Sezione Mappa 34

3.5.4 Sezione Corsa 36

3.5.5 Sezione Storico 37

3.5.6 Sezione Community 39
3.5.7 Sezione Profilo 41
3.6 Rappresentazione Grafica dei Casi d'Uso 42
3.6.1 Acesso e creazione Account 42
3.6.2 Avviamento di una corsa 43
3.6.3 Estensioe e terminazione corsa 44
3.6.4 Consultazione storico 45
3.6.5 Pubblicazione recensione 46
3.6.6 Modificazione profilo e logout 47
3.7 Approccio e Problematiche di Sviluppo 47
3.7.1 Bug Risolti 47
3.7.2 Bug 48
3.7.3 Librerie di Terze Parti 48
3.8 Descrizione Componenti 49
3.8.1 MainActivity 49
3.8.2 Modelli 49
3.8.3 GoRideDatabase 50
3.8.4 RideDao 50
3.8.5 ActiveRideDao 51
3.8.6 RideEntity e ActiveRideEntity 51
3.8.7 VehicleRepository e MockVehicleDataSource 51
3.8.8 UserRepository 51
3.8.9 RideRepository 52
3.8.10 ReviewRepository 52
3.8.11 ACCOUNT: LoginFragment e LoginViewModel 53
3.8.12 ACCOUNT: RegisterFragment e RegisterViewModel 53
3.8.13 ACCOUNT: ForgotPasswordFragment e ForgotPasswordViewModel 54
3.8.14 PROFILO: ProfileFragment e ProfileViewModel 54
3.8.15 MAPPA: MapFragment 54
3.8.16 MAPPA: MapViewModel 55
3.8.17 MAPPA: VehicleDetailFragment 55
3.8.18 CORSA: RideFragment 55
3.8.19 CORSA: RideViewModel 56

3.8.20 STORICO: HistoryFragment e HistoryViewModel 56
3.8.21 STORICO: RideAdapter 57
3.8.22 STORICO: RideDetailFragment 57
3.8.23 COMMUNITY: CommunityFragment e CommunityViewModel . . 57
3.8.24 COMMUNITY: NewReviewFragment e NewReviewViewModel . . . 57
3.8.25 COMMUNITY: ReviewAdapter 58
3.8.26 Formatters 58
3.9 Testing 58
3.9.1 ExampleUnitTest 58

4 Flutter App **59**
4.1 Introduzione 59
4.2 Analisi dei Requisiti 60
4.2.1 REQUISITI FUNZIONALI 60
4.2.2 REQUISITI NON FUNZIONALI 61
4.3 Casi d'Uso 62
4.3.1 UC: Gestione Account 63
4.3.2 UC: Gestione Profilo 63
4.3.3 UC: Gestione Mappa e Mezzi 64
4.3.4 UC: Gestione Corsa 64
4.3.5 UC: Gestione Storico 66
4.3.6 UC: Gestione Community 66
4.4 Mappa dell'Architettura 66
4.4.1 Architettura dell'Avvio e della Navigazione 68
4.4.2 Architettura di Autenticazione e Profilo 69
4.4.3 Architettura della Mappa e del Noleggio 70
4.4.4 Architettura dello Storico Corse 71
4.4.5 Architettura della Community 72
4.5 Database 72
4.5.1 Firebase 73
4.6 Mockup UI 73
4.6.1 NavBar 73
4.6.2 Sezione Account 74

4.6.3 Sezione Mappa 74
4.6.4 Sezione Storico 75
4.6.5 Sezione Community 75
4.6.6 Sezione Profilo 76
4.7 Rappresentazione Grafica dei Casi d'Uso 77
4.7.1 Accesso e creazione account 77
4.7.2 Noleggio di un mezzo 78
4.7.3 Consultazione storico 78
4.7.4 Pubblicazione recensione 79
4.7.5 Modifica profilo e logout 80
4.8 Commenti sull'Approccio di Sviluppo 80
4.8.1 Componenti Importate 80
4.9 Descrizione delle Componenti 81
4.9.1 Main e GoRideApp 81
4.9.2 AppScope e Routes 81
4.9.3 HomeShell 82
4.9.4 Modelli 82
4.9.5 GoRideDatabase e RideDao 83
4.9.6 VehicleRepository e MockVehicleDataSource 83
4.9.7 UserRepository 83
4.9.8 RideRepository 84
4.9.9 ReviewRepository 84
4.9.10 ACCOUNT: LoginScreen 84
4.9.11 ACCOUNT: RegisterScreen 85
4.9.12 ACCOUNT: ForgotPasswordScreen 85
4.9.13 PROFILO: ProfileScreen 85
4.9.14 MAPPA: MapScreen 85
4.9.15 MAPPA: VehicleDetailSheet 85
4.9.16 STORICO: HistoryScreen 86
4.9.17 STORICO: RideCard e RideDetailScreen 86
4.9.18 COMMUNITY: CommunityScreen 86
4.9.19 COMMUNITY: NewReviewScreen 86
4.9.20 COMMUNITY: ReviewCard 86

Progetto Programmazione Mobile

NOME COGNOME

MESE ANNO

1 Descrizione In Linguaggio Naturale

GoRide è un'app di bike sharing che permette di noleggiare biciclette classiche ed e-bike disponibili nella città di Ancona. L'utente visualizza su una mappa i mezzi liberi nelle vicinanze, distinti per tipologia. Selezionando un mezzo consulta le informazioni utili prima del noleggio: per le e-bike la percentuale di batteria e l'autonomia in chilometri, per tutti le tariffe di sblocco e al minuto. Dopo aver sbloccato il mezzo sceglie quanto tempo acquistare in anticipo (15, 30 o 60 minuti) e parte la corsa, con un timer che mostra il tempo residuo e il costo che cresce in tempo reale.

Alla scadenza del tempo prepagato l'app mostra un avviso e l'utente deve scegliere se estendere la corsa, acquistando altri minuti, oppure terminarla. Può anche estenderla o chiuderla in qualsiasi momento prima della scadenza. Il costo finale è dato dalla tariffa di sblocco più i minuti effettivamente utilizzati, e ogni minuto iniziato viene conteggiato per intero. La corsa conclusa viene salvata e resta consultabile nello storico, anche senza connessione, con durata, mezzo utilizzato e costo.

L'accesso all'app avviene tramite credenziali, composte da un indirizzo email e da una password di almeno 6 caratteri. L'utente può registrarsi indicando nome, cognome e numero di telefono, recuperare la password via email e modificare i propri dati dal profilo. Infine, la sezione Community permette agli utenti registrati di pubblicare e leggere recensioni di tre tipi (guasto, opinione, consiglio), che si aggiornano in tempo reale quando un altro utente pubblica.

L'app quindi risulta modulare nel proprio utilizzo. Le macro categorie sono:

- Ricerca e sblocco dei mezzi: mappa dei mezzi disponibili, distinzione tra bici classiche ed e-bike, consultazione di batteria, autonomia e tariffe, sblocco del mezzo
- Gestione della corsa: timer del tempo prepagato, costo in tempo reale, alert alla scadenza, estensione o termine della corsa.
- Gestione dell'account: registrazione, accesso, recupero della password e modifica del profilo.
- Storico delle corse: elenco delle corse concluse e dettaglio di ciascuna (durata, mezzo, costo).
- Sezione Community: pubblicazione e lettura di recensioni di tipo guasto, opinione e consiglio.

2 Glossario

Termine	Definizione	Tipo	Sinonimi
Utente	Persona registrata che utilizza l'app per noleggiare i mezzi e partecipare alla community.	Attore	Cliente, Ciclista
Mezzo	Bicicletta disponibile al noleggio, identificata da un codice e da una posizione sulla mappa.	Entità	Veicolo, Bici
Bici classica	Mezzo a sola pedalata muscolare, senza batteria.	Entità	Bici muscolare
E-bike	Mezzo a pedalata assistita, dotato di batteria e di un'autonomia in chilometri.	Entità	Bici elettrica
Corsa	Periodo di utilizzo di un mezzo, dallo sblocco al termine, con durata e costo.	Entità	Noleggio
Sblocco	Operazione con cui l'utente attiva il mezzo e avvia la corsa.	Funzionalità	Attivazione
Tariffa di sblocco	Importo fisso addebitato a ogni corsa, indipendente dalla durata.	Regola di Business	Costo di sblocco
Tariffa al minuto	Importo addebitato per ogni minuto di utilizzo del mezzo.	Regola di Business	Costo al minuto
Tempo prepagato	Durata della corsa scelta dall'utente prima della partenza (15, 30 o 60 minuti).	Regola di Business	Durata acquistata
Estensione	Acquisto di minuti aggiuntivi per prolungare una corsa in corso.	Funzionalità	Proroga
Storico	Elenco delle corse concluse dall'utente, consultabile anche offline.	Entità	Cronologia corse
Recensione	Contributo pubblicato da un utente nella community, di tipo guasto, opinione o consiglio.	Entità	Commento
Community	Sezione dell'app in cui gli utenti registrati pubblicano e leggono le recensioni.	Funzionalità	Sezione recensioni

Figure 1: Glossario

3 Kotlin App

3.1 Analisi dei Requisiti

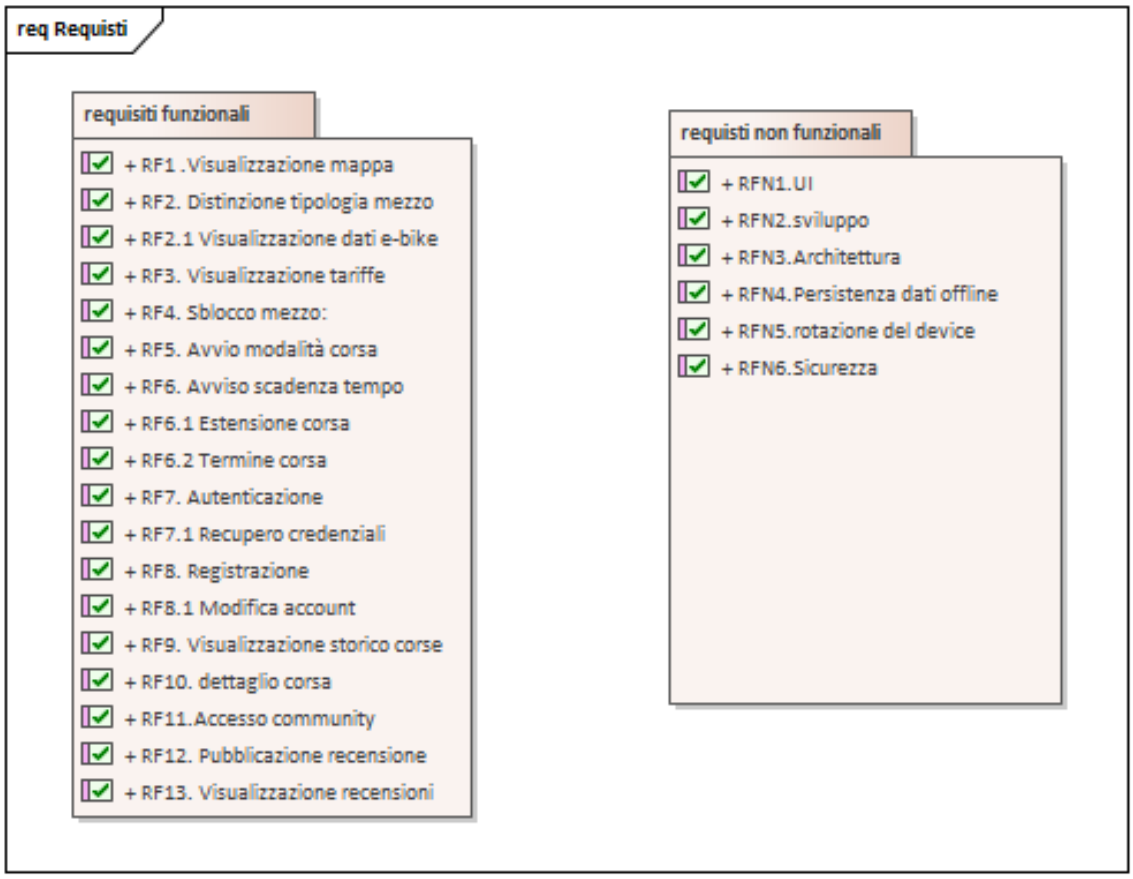


Figure 2: Requisiti di Sistema

3.1.1 REQUISITI FUNZIONALI

RF1. Visualizzazione mappa : L'utente deve poter visualizzare su una mappa i mezzi disponibili nelle vicinanze

RF2. Distinzione tipologia mezzo : L'utente deve poter distinguere sulla mappa le biciclette classiche dalle e-bike

RF2.1. Visualizzazione dati e-bike : Per ogni e-bike l'utente deve poter visualizzare la percentuale di batteria e l'autonomia residua in chilometri

RF3. Visualizzazione tariffe : L'utente deve poter visualizzare, prima del noleggio, la tariffa di sblocco e la tariffa al minuto del mezzo selezionato

RF4. Sblocco mezzo : L'utente deve poter sbloccare il mezzo selezionato dalla schermata di dettaglio

RF5. Avvio modalità corsa : L'utente deve poter avviare la corsa scegliendo il tempo da prepagare e visualizzare il tempo residuo e il costo aggiornati in tempo reale

RF6. Avviso scadenza tempo : Alla scadenza del tempo prepagato il sistema deve avvisare l'utente, che deve scegliere se estendere o terminare la corsa

RF6.1. Estensione corsa : L'utente deve poter estendere la corsa acquistando minuti aggiuntivi, sia alla scadenza sia in qualsiasi momento durante la corsa

RF6.2. Termine corsa : L'utente deve poter terminare la corsa, previa conferma, e il sistema deve registrarla nello storico con il costo finale

RF7. Autenticazione : L'utente deve poter autenticarsi inserendo email e password

RF7.1. Recupero credenziali : L'utente deve poter recuperare la password ricevendo una email con il link per impostarne una nuova

RF8. Registrazione : L'utente deve poter registrarsi inserendo nome, cognome, email, telefono e una password di almeno 6 caratteri, da confermare

RF8.1. Modifica account : L'utente deve poter visualizzare i propri dati dal profilo e modificarne nome, cognome e telefono

RF8.2. Uscita dall'account : L'utente deve poter uscire dal proprio account, previa conferma, tornando alla schermata di accesso

RF9. Visualizzazione storico corse : L'utente deve poter visualizzare l'elenco delle proprie corse concluse, dalla più recente, anche senza connessione

RF10. Dettaglio corsa : L'utente deve poter visualizzare per ogni corsa conclusa la durata, il mezzo utilizzato e il costo, con la composizione del costo

RF11. Accesso community : L'utente autenticato deve poter accedere alla sezione community

RF12. Pubblicazione recensione : L'utente deve poter pubblicare una recensione scegliendone il tipo (guasto, opinione o consiglio) e inserendo il testo

RF13. Visualizzazione recensioni : L'utente deve poter visualizzare le recensioni pubblicate dagli utenti, con autore e data, aggiornate in tempo reale

3.1.2 REQUISITI NON FUNZIONALI

RNF1. UI : L'interfaccia deve essere semplice ed intuitiva per migliorare la user experience

RNF2. Sviluppo : Il linguaggio di sviluppo è Kotlin

RNF3. Architettura : Il pattern architetturale utilizzato è MVVM, con Repository come unica porta verso i dati e un'unica Activity che ospita le schermate come Fragment

RNF4. Persistenza dati offline : Lo storico delle corse e la corsa in corso sono salvati in locale attraverso un db gestito con Room, in modo da essere disponibili anche senza connessione

RNF5. Rotazione del device : Alla rotazione dello schermo l'app deve mantenere il proprio stato, in particolare il timer e il costo della corsa in corso

RNF6. Sicurezza : La password è gestita solo da Firebase Authentication e non viene mai salvata in Firestore; la chiave di Google Maps è tenuta fuori dal codice versionato

3.2 Casi d'Uso

3.2.1 UC: Gestione Account

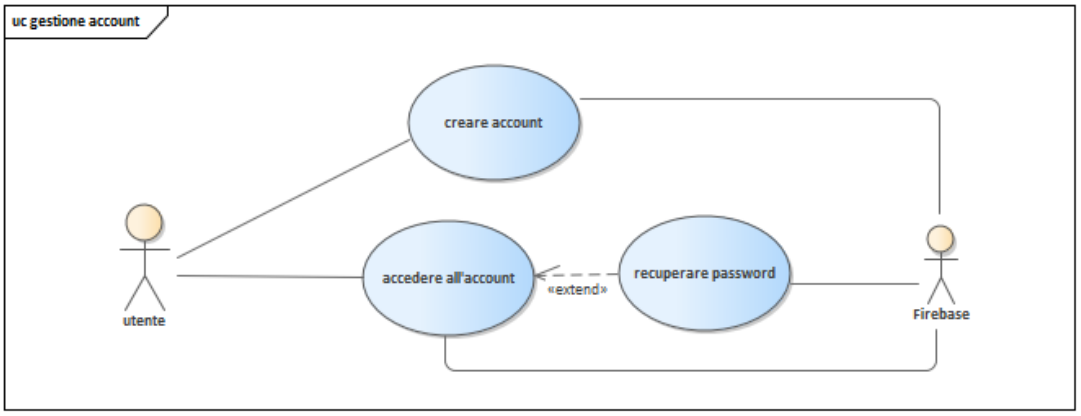


Figure 3: Gestione Account

Questi casi d'uso riguardano l'area di funzionalità per la gestione dell'account dell'utente. L'attore secondario Firebase interviene nella creazione dell'account, nell'autenticazione e nell'invio della mail di recupero.

Casi d'uso: Creare account

Questo caso d'uso si verifica quando un utente non ancora registrato vuole creare un account per usare l'applicazione.

Pre-condizioni: L'utente non è autenticato e si trova nella schermata di accesso.

Post-condizioni: L'account è creato in Firebase Authentication, il profilo è salvato in Firestore e l'utente è autenticato e visualizza la mappa.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente sceglie di registrarsi.

1. L'utente inserisce nome, cognome, email, telefono, password e conferma della password.
2. L'utente conferma la registrazione.
3. Il sistema valida i dati inseriti.
4. Il sistema chiede a Firebase di creare l'account e di salvare il profilo.
5. Firebase conferma l'operazione.
6. Il sistema porta l'utente alla mappa.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 3 se un dato non è valido (campo vuoto, email non valida, telefono troppo corto, password di meno di 6 caratteri, password diverse): il

sistema mostra un messaggio e l'utente resta nel modulo. Inizia al punto 5 se Firebase non completa l'operazione (email già registrata, rete assente): il sistema mostra un messaggio e l'utente resta nel modulo.

Casi d'uso: Accedere all'account

Questo caso d'uso si verifica quando un utente registrato vuole accedere all'applicazione.

Pre-condizioni: L'utente non è autenticato e si trova nella schermata di accesso.

Post-condizioni: L'utente è autenticato e visualizza la mappa.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente apre l'applicazione senza una sessione attiva.

1. L'utente inserisce email e password.
2. L'utente conferma l'accesso.
3. Il sistema verifica che i campi siano compilati e che l'email sia valida.
4. Il sistema invia le credenziali a Firebase Authentication.
5. Firebase conferma l'autenticazione.
6. Il sistema porta l'utente alla mappa.
7. extend(Recuperare password), se l'utente ha dimenticato la password.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 3 se i dati non sono validi, oppure al punto 5 se Firebase rifiuta l'accesso (credenziali errate, account inesistente, troppi tentativi, rete assente): il sistema mostra un messaggio e l'utente resta nella schermata di accesso.

Casi d'uso: Recuperare password

Questo caso d'uso si verifica quando l'utente non ricorda la password e vuole reimpostarla.

Pre-condizioni: L'utente non è autenticato e ha scelto il recupero password dalla schermata di accesso.

Post-condizioni: Se l'email è registrata, Firebase invia il link di recupero; l'utente torna alla schermata di accesso.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente apre la schermata di recupero password.

1. L'utente inserisce la propria email e richiede l'invio.
2. Il sistema verifica che l'email sia valida.
3. Il sistema chiede a Firebase di inviare il link di recupero.
4. Il sistema mostra un messaggio generico, che non rivela se l'email esiste.
5. Dopo pochi secondi il sistema riporta l'utente alla schermata di accesso.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 2 se l'email non è valida, oppure al punto 3 se Firebase non completa l'invio (ad esempio per rete assente): il sistema mostra un messaggio e l'utente resta nella schermata.

3.2.2 UC: Gestione Profilo

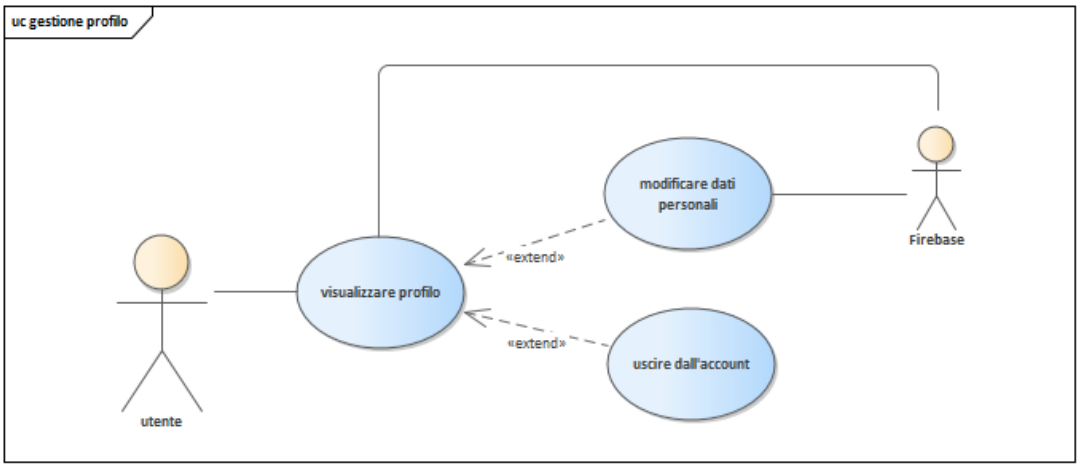


Figure 4: Gestione Profilo

Questi casi d'uso riguardano l'area di funzionalità per la consultazione e la modifica del profilo dell'utente autenticato e per l'uscita dall'account. L'attore secondario Firebase interviene nel caricamento e nel salvataggio dei dati del profilo.

Casi d'uso: Visualizzare profilo

Questo caso d'uso si verifica quando l'utente vuole consultare i propri dati.

Pre-condizioni: L'utente è autenticato.

Post-condizioni: L'utente visualizza nome, cognome, telefono, email e data di iscrizione.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente apre la sezione Profilo.

1. Il sistema richiede a Firebase il profilo dell'utente.
2. Firebase restituisce i dati.
3. Il sistema mostra i dati nella schermata.
4. extend(Modificare dati personali).
5. extend(Uscire dall'account).

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 2 se i dati non sono disponibili: il sistema mostra il messaggio "Impossibile caricare il profilo".

Casi d'uso: Modificare dati personali

Questo caso d'uso si verifica quando l'utente vuole aggiornare nome, cognome o telefono. Email e password non sono modificabili.

Pre-condizioni: L'utente è autenticato e visualizza il proprio profilo.

Post-condizioni: I dati modificati sono salvati in Firestore e il profilo è ricaricato.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente modifica uno o più campi del profilo.

1. L'utente modifica nome, cognome e/o telefono e conferma il salvataggio.
2. Il sistema verifica i dati (nome e cognome non vuoti, telefono di almeno 8 caratteri).
3. Il sistema invia l'aggiornamento a Firebase.
4. Firebase conferma l'operazione.
5. Il sistema mostra "Profilo aggiornato" e ricarica i dati.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 2 se i dati non sono validi, oppure al punto 4 se Firebase non completa il salvataggio: il sistema mostra un messaggio e il profilo resta invariato.

Casi d'uso: Uscire dall'account

Questo caso d'uso si verifica quando l'utente vuole terminare la sessione.

Pre-condizioni: L'utente è autenticato e visualizza il proprio profilo.

Post-condizioni: La sessione è terminata e l'utente visualizza la schermata di accesso, senza poter tornare indietro alle schermate precedenti.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente seleziona il pulsante di uscita.

1. Il sistema chiede conferma con una finestra di dialogo.
2. L'utente conferma.
3. Il sistema termina la sessione.
4. Il sistema porta l'utente alla schermata di accesso.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 2 se l'utente annulla: la finestra si chiude e l'utente resta nel profilo.

3.2.3 UC: Gestione Mappa e Mezzi

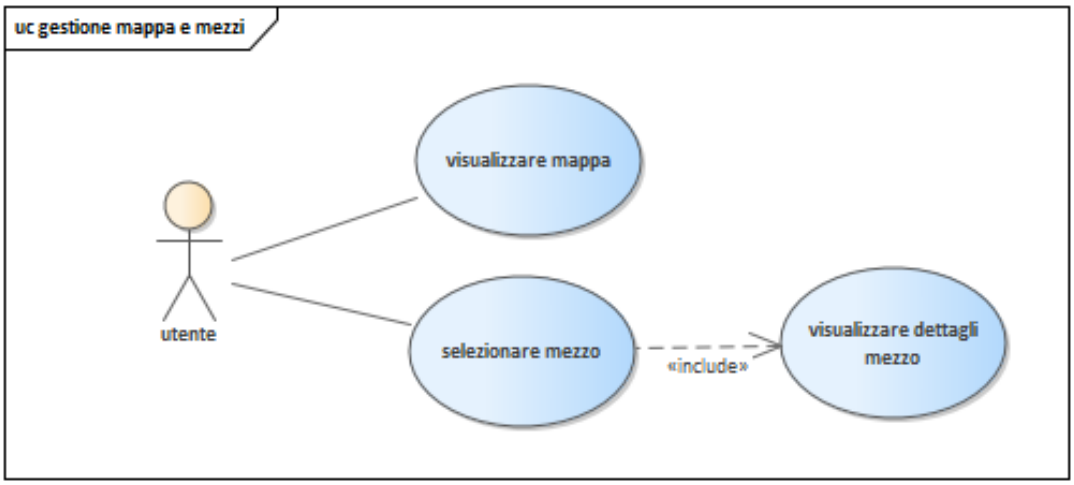


Figure 5: Gestione Mappa e Mezzi

Questi casi d'uso riguardano l'area di funzionalità per la consultazione della mappa e per la scelta del mezzo. In quest'area Firebase non interviene: i mezzi provengono da una sorgente dati locale simulata.

Casi d'uso: Visualizzare mappa

Questo caso d'uso si verifica quando l'utente vuole vedere i mezzi disponibili nelle vicinanze.

Pre-condizioni: L'utente è autenticato.

Post-condizioni: L'utente visualizza la mappa con i mezzi disponibili, distinti per tipologia.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente apre la schermata della mappa, che è la prima schermata dopo l'accesso.

1. Il sistema carica i mezzi disponibili.
2. Il sistema mostra la mappa centrata su Ancona.
3. Il sistema mostra un marker per ogni mezzo: verde per le biciclette classiche, arancione per le biciclette elettriche.
4. Il sistema chiede all'utente il permesso di accedere alla posizione.
5. Se l'utente concede il permesso, il sistema mostra la sua posizione sulla mappa.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 1 se il caricamento dei mezzi non riesce: il sistema mostra un messaggio con la possibilità di riprovare. Inizia al punto 5 se l'utente nega il permesso: il sistema mostra un messaggio e la mappa resta su Ancona con tutti i mezzi visibili. L'utente può richiedere di nuovo il permesso con il pulsante della posizione.

Casi d'uso: Selezionare mezzo

Questo caso d'uso si verifica quando l'utente vuole conoscere un mezzo toccandone il marker sulla mappa.

Pre-condizioni: L'utente visualizza la mappa con almeno un mezzo.

Post-condizioni: Il dettaglio del mezzo selezionato è visualizzato.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente tocca un marker.

1. Il sistema evidenzia il marker, mostrando il nome del mezzo e una sintesi: tariffa al minuto e, per le biciclette elettriche, anche batteria e autonomia.
2. include(Visualizzare dettagli mezzo).

Casi d'uso: Visualizzare dettagli mezzo

Questo caso d'uso mostra le informazioni complete del mezzo scelto. È incluso in Selezionare mezzo.

Pre-condizioni: L'utente ha selezionato un mezzo sulla mappa.

Post-condizioni: L'utente visualizza tariffe e, per le biciclette elettriche, batteria e autonomia, e può proseguire con lo sblocco.

Sequenza di eventi principali:

Il caso d'uso inizia quando il sistema apre il pannello di dettaglio.

1. Il sistema mostra la tariffa di sblocco e la tariffa al minuto.
2. Se il mezzo è una bicicletta elettrica, il sistema mostra la percentuale di batteria, colorata in base al livello, e l'autonomia in chilometri.
3. L'utente può proseguire con lo sblocco del mezzo (caso d'uso Sbloccare mezzo) oppure chiudere il pannello e tornare alla mappa.

3.2.4 UC: Gestione Corsa

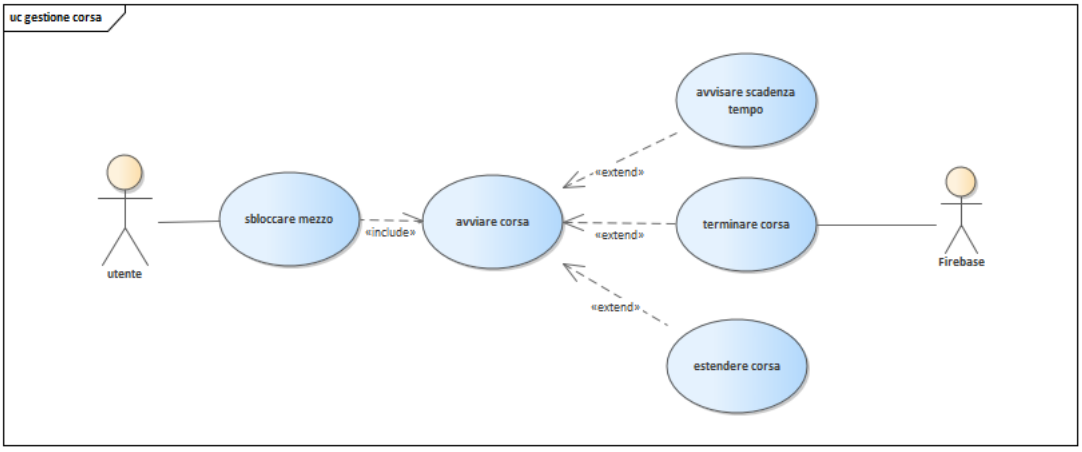


Figure 6: Gestione Corsa

Questi casi d'uso riguardano l'area di funzionalità per la gestione della corsa, dallo sblocco del mezzo alla sua conclusione. L'attore secondario Firebase interviene solo nel salvataggio

della corsa conclusa: durante la corsa i dati sono mantenuti in locale.

Casi d'uso: Sbloccare mezzo

Questo caso d'uso si verifica quando l'utente, dopo aver consultato i dettagli di un mezzo, decide di usarlo.

Pre-condizioni: L'utente è autenticato e visualizza il dettaglio di un mezzo.

Post-condizioni: La corsa è avviata.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente seleziona il pulsante di sblocco nel dettaglio del mezzo.

1. Il sistema apre la schermata della corsa relativa al mezzo scelto.
2. include(Avviare corsa).

Casi d'uso: Avviare corsa

Questo caso d'uso avvia la corsa con il tempo prepagato scelto dall'utente. È incluso in Sbloccare mezzo.

Pre-condizioni: L'utente ha scelto di sbloccare un mezzo e non ha altre corse attive.

Post-condizioni: La corsa è attiva: il conto alla rovescia procede e lo stato della corsa è salvato in locale.

Sequenza di eventi principali:

Il caso d'uso inizia quando il sistema apre la schermata della corsa.

1. Il sistema propone le durate prepagate (15, 30 o 60 minuti), indicando per ciascuna il costo, pari alla tariffa di sblocco più i minuti per la tariffa al minuto.
2. L'utente sceglie una durata.
3. Il sistema registra l'istante di inizio e calcola l'istante di scadenza.
4. Il sistema salva lo stato della corsa nel database locale.
5. Il sistema avvia il conto alla rovescia e mostra, aggiornati ogni secondo, il tempo rimanente e il costo maturato.
6. extend(Avvisare scadenza tempo), quando il tempo termina.
7. extend(Estendere corsa), se l'utente vuole acquistare altro tempo.
8. extend(Terminare corsa), se l'utente vuole concludere la corsa.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 1 se il sistema trova una corsa interrotta, per esempio perchè l'applicazione è stata chiusa dal sistema operativo: se la scadenza non è stata superata da più di 30 minuti, il sistema la ripristina e riprende il conto alla rovescia senza chiedere la durata. Se invece la scadenza è stata superata da più di 30 minuti, la corsa è considerata abbandonata, il sistema elimina lo stato salvato e prosegue dal punto 1.

Casi d'uso: Avvisare scadenza tempo

Questo caso d'uso si verifica quando il tempo prepagato termina. Il tempo è calcolato confrontando la scadenza con l'orologio, quindi resta corretto anche se l'applicazione è rimasta in background.

Pre-condizioni: La corsa è attiva e il tempo rimanente è zero.

Post-condizioni: L'utente ha scelto se estendere o terminare la corsa.

Sequenza di eventi principali:

Il caso d'uso inizia quando il conto alla rovescia raggiunge lo zero.

1. Il sistema mostra un avviso che non può essere ignorato.
2. L'utente sceglie una delle due opzioni offerte dall'avviso.
3. extend(Estendere corsa), se l'utente sceglie di estendere.
4. extend(Terminare corsa), se l'utente sceglie di terminare.

Casi d'uso: Estendere corsa

Questo caso d'uso si verifica quando l'utente vuole acquistare altro tempo per la corsa in corso.

Pre-condizioni: La corsa è attiva.

Post-condizioni: La scadenza è spostata in avanti del tempo scelto e il conto alla rovescia prosegue.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente seleziona il pulsante di estensione, oppure sceglie di estendere dall'avviso di scadenza.

1. Il sistema propone le estensioni (5, 15 o 30 minuti), indicando per ciascuna il costo, pari ai minuti per la tariffa al minuto.
2. L'utente sceglie un'estensione.
3. Il sistema sposta la scadenza in avanti, partendo dalla scadenza attuale oppure, se è già trascorsa, dall'istante corrente.
4. Il sistema aggiorna i minuti prepagati e salva lo stato nel database locale.
5. Il sistema riavvia il conto alla rovescia.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 2 se l'utente annulla: la corsa prosegue invariata.

Casi d'uso: Terminare corsa

Questo caso d'uso si verifica quando l'utente vuole concludere la corsa e registrarla nello storico.

Pre-condizioni: La corsa è attiva.

Post-condizioni: La corsa è salvata nello storico locale e su Firebase, lo stato della corsa attiva è eliminato e l'utente visualizza la mappa.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente seleziona il pulsante di termine oppure il tasto Indietro.

1. Il sistema chiede conferma, mostrando il costo attuale.
2. L'utente conferma.
3. Il sistema ferma il conto alla rovescia e calcola il costo finale: tariffa di sblocco più minuti trascorsi, arrotondati per eccesso, per la tariffa al minuto.

4. Il sistema salva la corsa nel database locale (mezzo, istanti di inizio e fine, minuti prepagati, tariffe e costo totale).
5. Il sistema chiede a Firebase di salvare la corsa.
6. Il sistema elimina lo stato della corsa attiva e riporta l'utente alla mappa.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 1 se l'utente annulla: la corsa prosegue. Se l'utente ha scelto di terminare dall'avviso di scadenza, la conferma non è richiesta e la sequenza prosegue dal punto 3. Inizia al punto 5 se Firebase non conferma il salvataggio, ad esempio per rete assente: la corsa resta comunque nello storico locale.

3.2.5 UC: Gestione Storico

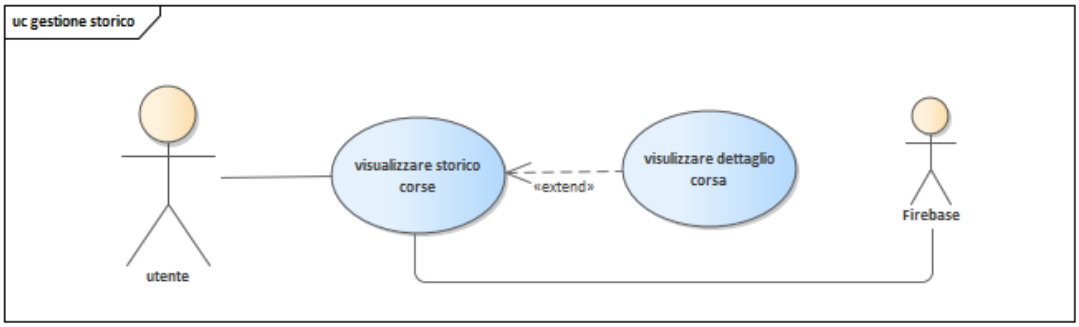


Figure 7: Gestione Storico

Questi casi d'uso riguardano l'area di funzionalità per la consultazione delle corse concluse. L'attore secondario Firebase interviene nel recupero delle corse salvate, che sono sincronizzate nel database locale.

Casi d'uso: Visualizzare storico corse

Questo caso d'uso si verifica quando l'utente vuole consultare le corse che ha concluso.

Pre-condizioni: L'utente è autenticato.

Post-condizioni: L'utente visualizza l'elenco delle proprie corse, dalla più recente, aggiornato con i dati di Firebase.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente apre la sezione Storico.

1. Il sistema mostra subito le corse presenti nel database locale, dalla più recente: per ciascuna data e ora, durata, costo e tipo di mezzo.
2. Il sistema richiede a Firebase le corse dell'utente.
3. Firebase restituisce le corse.
4. Il sistema aggiorna il database locale e l'elenco si aggiorna di conseguenza.
5. extend(Visualizzare dettaglio corsa).

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 1 se non ci sono corse: il sistema mostra lo stato

di elenco vuoto. Inizia al punto 3 se Firebase non risponde, ad esempio per rete assente: se il database locale contiene già delle corse, l'elenco resta quello locale e non compare alcun errore; altrimenti il sistema mostra un messaggio con la possibilità di riprovare.

Casi d'uso: Visualizzare dettaglio corsa

Questo caso d'uso si verifica quando l'utente vuole vedere nel dettaglio una corsa conclusa.

Pre-condizioni: L'utente visualizza l'elenco delle corse e ne ha almeno una.

Post-condizioni: L'utente visualizza il dettaglio della corsa selezionata.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente seleziona una corsa dall'elenco.

1. Il sistema mostra costo totale, durata, data e ora, tipo di mezzo, tariffa di sblocco e tariffa al minuto della corsa.
2. L'utente torna all'elenco.

3.2.6 UC: Gestione Community

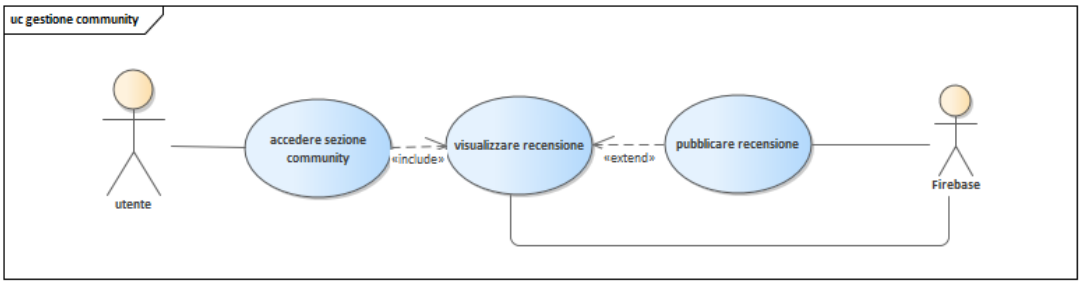


Figure 8: Gestione Community

Questi casi d'uso riguardano l'area di funzionalità per la consultazione e la pubblicazione delle recensioni. L'attore secondario Firebase interviene nella lettura in tempo reale delle recensioni e nel salvataggio di quelle nuove.

Casi d'uso: Accedere alla sezione community

Questo caso d'uso si verifica quando l'utente vuole consultare le recensioni condivise dagli altri utenti.

Pre-condizioni: L'utente è autenticato.

Post-condizioni: L'utente visualizza la sezione community con le recensioni.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente seleziona la sezione Community dalla navigazione dell'applicazione.

1. include(Visualizzare recensione).

Casi d'uso: Visualizzare recensione

Questo caso d'uso mostra le recensioni pubblicate dagli utenti. È incluso in Accedere alla sezione community.

Pre-condizioni: L'utente ha aperto la sezione community.

Post-condizioni: L'utente visualizza le recensioni più recenti, aggiornate in tempo reale; uscendo dalla sezione l'ascolto è interrotto.

Sequenza di eventi principali:

Il caso d'uso inizia quando il sistema apre la sezione community.

1. Il sistema si mette in ascolto delle recensioni su Firebase.
2. Firebase invia le ultime 100 recensioni, dalla più recente.
3. Il sistema mostra per ciascuna recensione autore, tipo (guasto, opinione o consiglio) e testo, con un aspetto diverso a seconda del tipo.
4. Quando un altro utente pubblica una recensione, Firebase notifica la modifica e il sistema aggiorna l'elenco senza intervento dell'utente.
5. extend(Pubblicare recensione).

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 2 se non ci sono recensioni: il sistema mostra lo stato di elenco vuoto. Se Firebase non è raggiungibile, l'elenco non viene aggiornato e non compare alcun messaggio.

Casi d'uso: Pubblicare recensione

Questo caso d'uso si verifica quando l'utente vuole condividere un guasto, un'opinione o un consiglio.

Pre-condizioni: L'utente visualizza l'elenco delle recensioni.

Post-condizioni: La recensione è salvata su Firebase e compare in cima all'elenco; l'utente è tornato all'elenco.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente seleziona il pulsante per scrivere una nuova recensione.

1. Il sistema mostra il modulo di pubblicazione.
2. L'utente sceglie il tipo di recensione (guasto, opinione o consiglio; per impostazione predefinita opinione) e scrive il testo.
3. L'utente conferma la pubblicazione.
4. Il sistema verifica che il testo non sia vuoto e che abbia almeno 10 caratteri.
5. Il sistema recupera il nome dell'autore dal profilo dell'utente.
6. Il sistema chiede a Firebase di salvare la recensione con autore, tipo, testo e istante di pubblicazione.
7. Firebase conferma l'operazione.
8. Il sistema riporta l'utente all'elenco, dove la nuova recensione compare in cima.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 4 se il testo è vuoto o troppo corto: il sistema mostra un messaggio e l'utente resta nel modulo. Al punto 5, se il nome non è disponibile, il sistema usa il nome generico "Utente GoRide". Inizia al punto 7 se Firebase non conferma:

il sistema mostra il messaggio “Pubblicazione non riuscita. Riprova.” e l’utente resta nel modulo.

3.3 Mappa dell’Architettura

Per l’architettura di GoRide è stato adottato il pattern MVVM, affiancato dal pattern Repository. I Fragment (View) si limitano a mostrare i dati e a raccogliere gli input dell’utente; i ViewModel espongono lo stato tramite `LiveData` e coordinano le operazioni asincrone con le coroutine; i Repository sono l’unico punto di accesso ai dati e nascondono le sorgenti utilizzate (Room, Firestore, Firebase Authentication). Le dipendenze vanno in un’unica direzione, dalla UI verso i dati, e i tipi delle librerie Firebase (`FirebaseUser`, `DocumentSnapshot`) non escono mai dai Repository: ai ViewModel e ai Fragment arrivano solo i modelli di dominio `Vehicle`, `Ride`, `Review` e `User`. L’app ha una sola Activity, che ospita il NavHost del Navigation Component; tutte le schermate sono Fragment e i layout si collegano ai ViewModel tramite Data Binding.

La Figura 9 riporta la mappa complessiva dell’architettura: la `MainActivity` ospita la navigazione; i Fragment, con i rispettivi layout e ViewModel, dipendono dai quattro Repository (`UserRepository`, `VehicleRepository`, `RideRepository` e `ReviewRepository`), che sono gli unici componenti a conoscere le sorgenti di dati: Firebase Authentication, Firestore, il database Room (`GoRideDatabase`) e la sorgente di esempio `MockVehicleDataSource`. `MapFragment` usa inoltre le Google Maps API. Nella mappa complessiva le relazioni sono indicate con semplici frecce, dal componente che usa a quello usato.

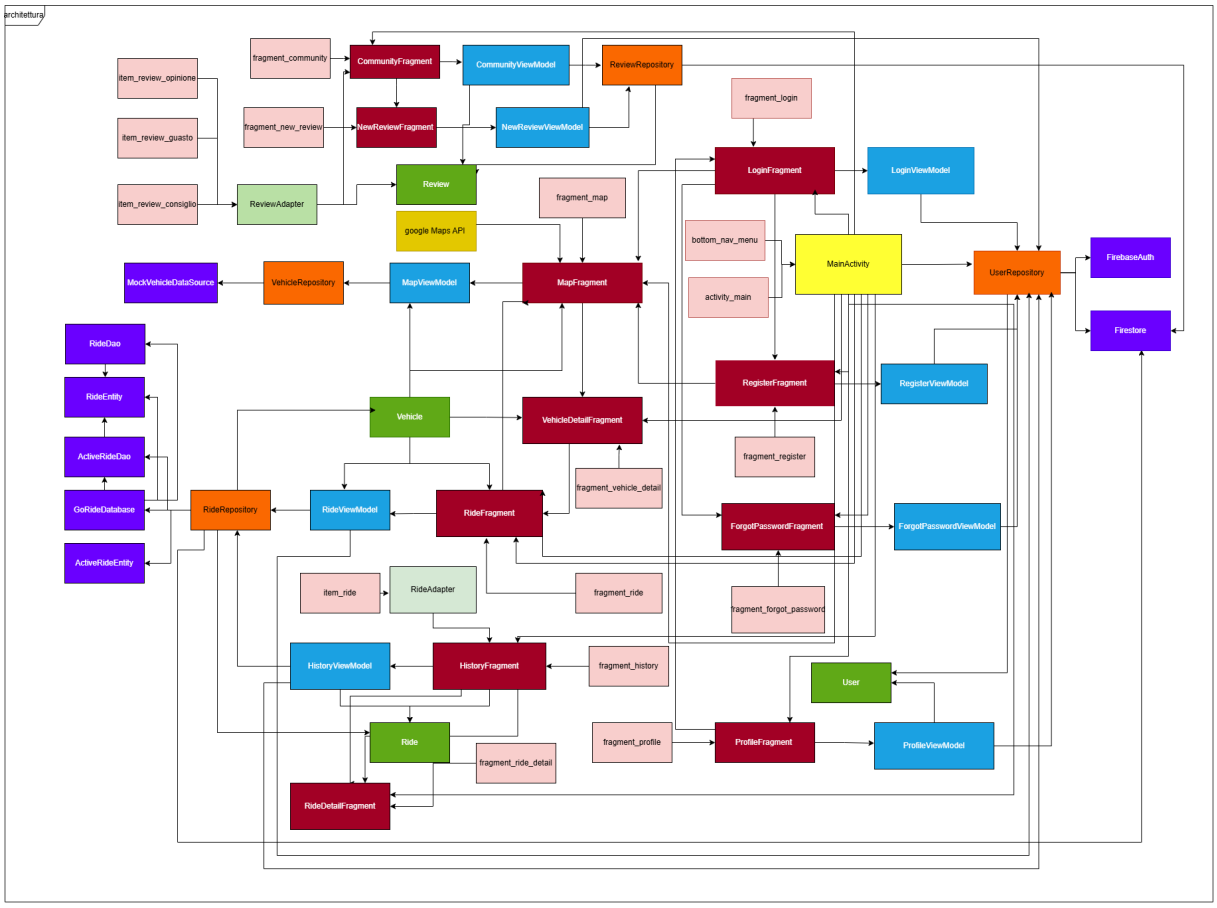


Figure 9: Mappa dell'architettura

Per maggiore leggibilità la mappa è poi suddivisa in sei diagrammi di dettaglio, uno per area funzionale, nei quali si usa la seguente notazione:

- freccia continua: associazione, la classe mantiene un riferimento all'altra (ad esempio da Fragment a ViewModel e da ViewModel a Repository);
- freccia tratteggiata: dipendenza, con uno stereotipo che ne indica il tipo: `«navigate»` per la navigazione tra schermate, `«inflate»` per il caricamento di un layout, `«create»` per l'istanziazione di un oggetto; senza stereotipo indica un semplice utilizzo;
- rombo pieno: composizione (il database contiene le proprie entità);
- triangolo vuoto: generalizzazione (ereditarietà).

I colori distinguono i ruoli: rosso scuro per i Fragment, azzurro per i ViewModel, arancione per i Repository, verde per i modelli, verde chiaro per gli Adapter, rosa per layout e menu, viola per le sorgenti di dati e i servizi esterni, giallo per `MainActivity` e per le Google Maps API.

3.3.1 Architettura dell'Avvio e della Navigazione

La `MainActivity` è l'unica Activity dell'app e non contiene logica di business: carica il layout `activity_main`, ospita il NavHost associato al grafo `nav_graph`, la cui destinazione iniziale è `LoginFragment`, e collega la barra di navigazione inferiore (`bottom_nav_menu`) al NavController, rendendo raggiungibili mappa, storico, community e profilo. All'avvio crea un'istanza di `UserRepository` solo per verificare se esiste già una sessione: in tal caso naviga direttamente alla mappa, altrimenti l'utente parte dal login. La barra di navigazione è nascosta nelle schermate di login, registrazione e recupero password.

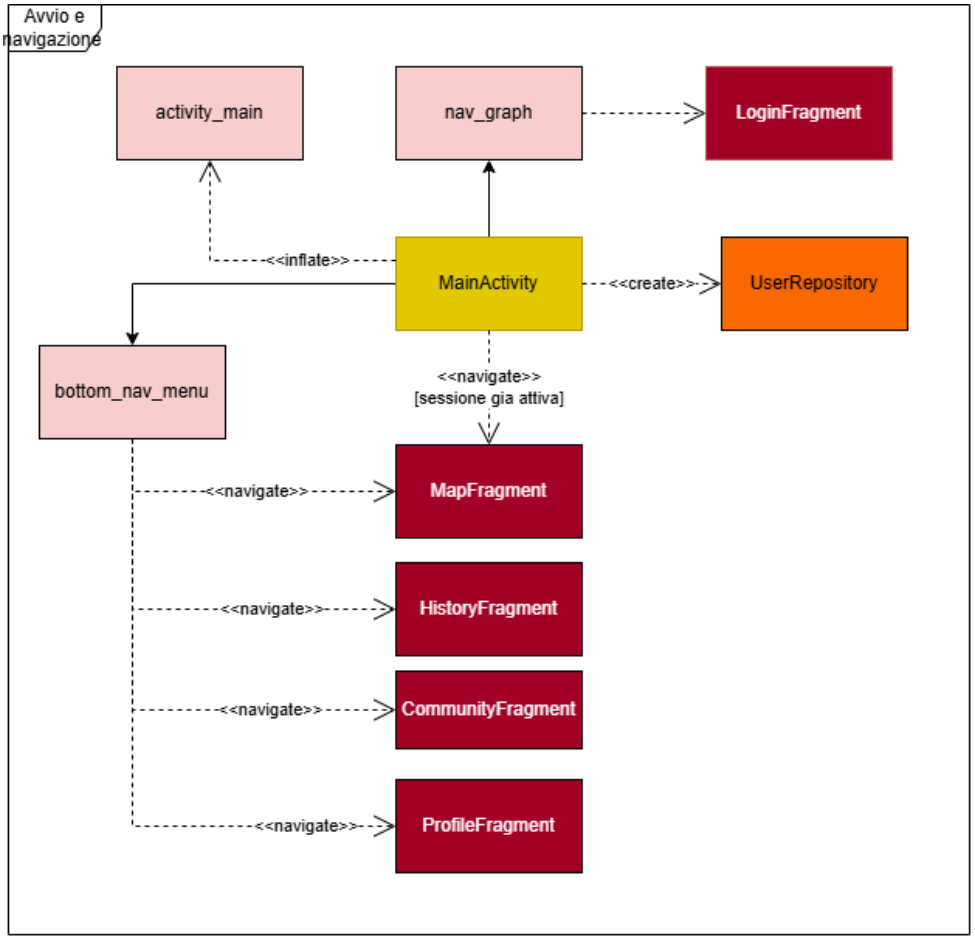


Figure 10: Architettura dell'avvio e della navigazione

3.3.2 Architettura di Autenticazione e Profilo

Le quattro schermate di quest'area (`LoginFragment`, `RegisterFragment`, `ForgotPasswordFragment` e `ProfileFragment`) hanno ciascuna il proprio layout e il proprio ViewModel (`LoginViewModel`, `RegisterViewModel`, `ForgotPasswordViewModel` e `ProfileViewModel`). Tutti i ViewModel si rivolgono a `UserRepository`, l'unica classe che comunica con Firebase Authentication, per login, registrazione, recupero della password e logout, e con Firestore, dove è salvato il profilo dell'utente (collezione `users`). La password è gestita esclusivamente da Firebase Authentication: non viene salvata in Firestore né inclusa nel modello `User`. Solo `ProfileViewModel` conosce il modello `User`, che `UserRepository` crea a partire dal profilo letto da Firestore. Dopo un login o una

registrazione riusciti l'utente viene portato alla mappa. Quest'area realizza i requisiti RF7, RF7.1, RF8 e RF8.1.

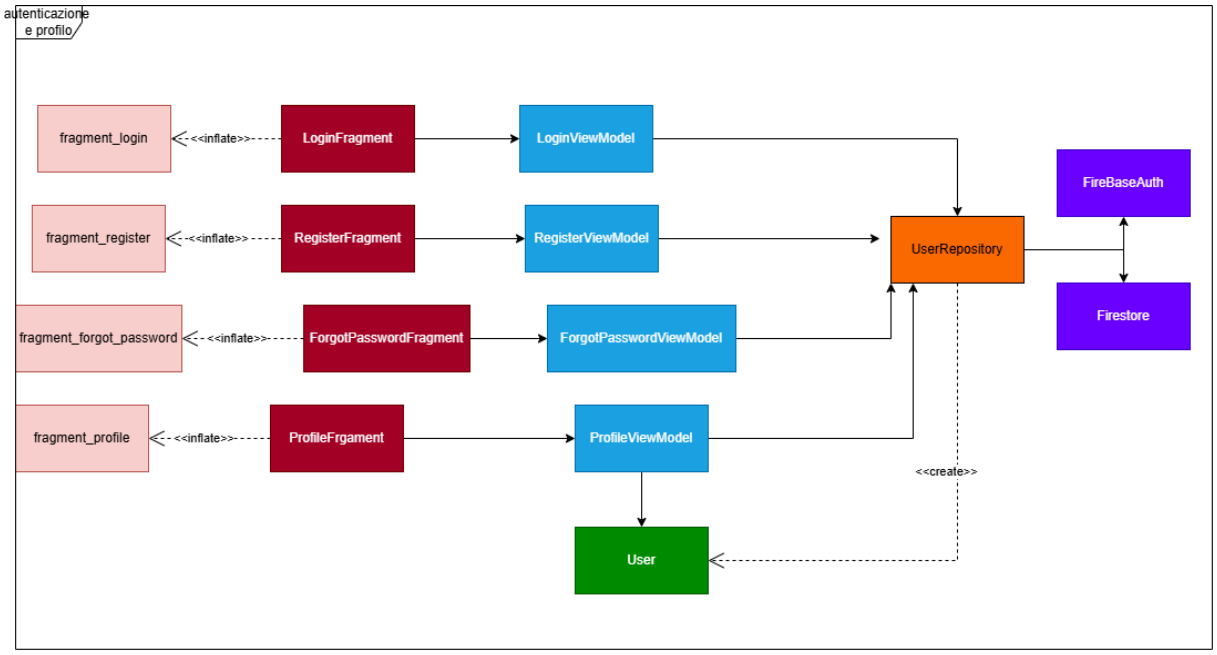


Figure 11: Architettura di autenticazione e profilo

3.3.3 Architettura della Mappa e dei Mezzi

`MapFragment` mostra la mappa tramite le Google Maps API e disegna un marker per ogni mezzo, distinguendo biciclette ed e-bike; è l'unico a conoscere gli oggetti della mappa, mentre `MapViewModel` espone soltanto la lista di `Vehicle`. I mezzi sono forniti da `VehicleRepository`, che attualmente li ottiene da `MockVehicleDataSource`, una sorgente in memoria con mezzi di esempio: sostituirla con un servizio remoto richiederebbe di modificare solo il Repository. Toccando un marker si apre `VehicleDetailFragment`, un bottom sheet (estende `BottomSheetDialogFragment`) che riceve il `Vehicle` tramite Safe Args e mostra le tariffe e, per le e-bike, batteria e autonomia; da qui l'utente passa a `RideFragment` per sbloccare il mezzo. Quest'area realizza i requisiti da RF1 a RF4.

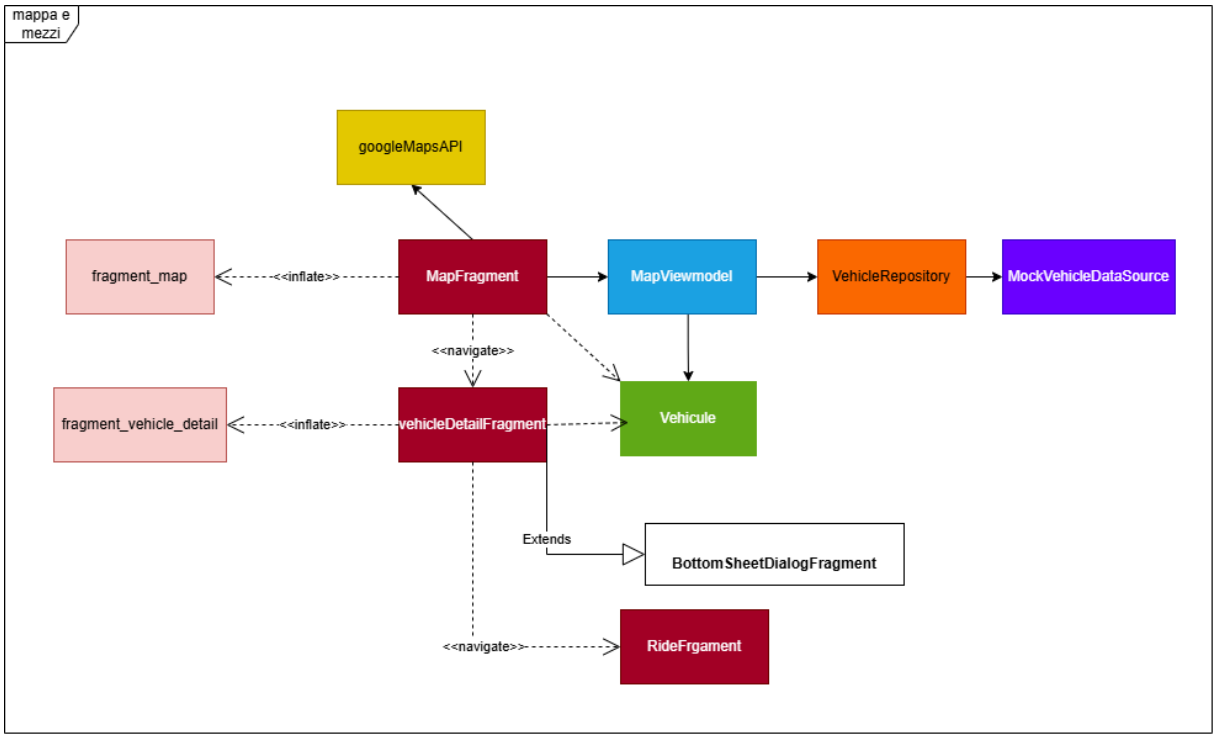


Figure 12: Architettura della mappa e dei mezzi

3.3.4 Architettura della Corsa Attiva

`RideFragment` mostra il timer e il costo della corsa e gestisce l'avviso di scadenza, con la scelta tra estendere e terminare. Il suo `RideViewModel` è condiviso a livello di Activity (`activityViewModels`), così da mantenere lo stato della corsa quando si cambia schermata, ed è un `AndroidViewModel` perché `RideRepository` ha bisogno del `Context` per aprire il database Room. Il ViewModel usa `UserRepository` per ottenere l'utente corrente e `RideRepository` per la persistenza: lo stato della corsa in corso è salvato in Room (`ActiveRideDao` e `ActiveRideEntity`), in modo da poterlo ripristinare se il sistema chiude l'app, mentre a fine corsa viene creato un oggetto `Ride`, salvato prima in Room (`RideDao` e `RideEntity`) e poi su Firestore (collezione `rides`). `GoRideDatabase` contiene le due entità (composizione). Al termine, `RideFragment` riporta l'utente alla mappa. Quest'area realizza i requisiti RF5, RF6, RF6.1 e RF6.2.

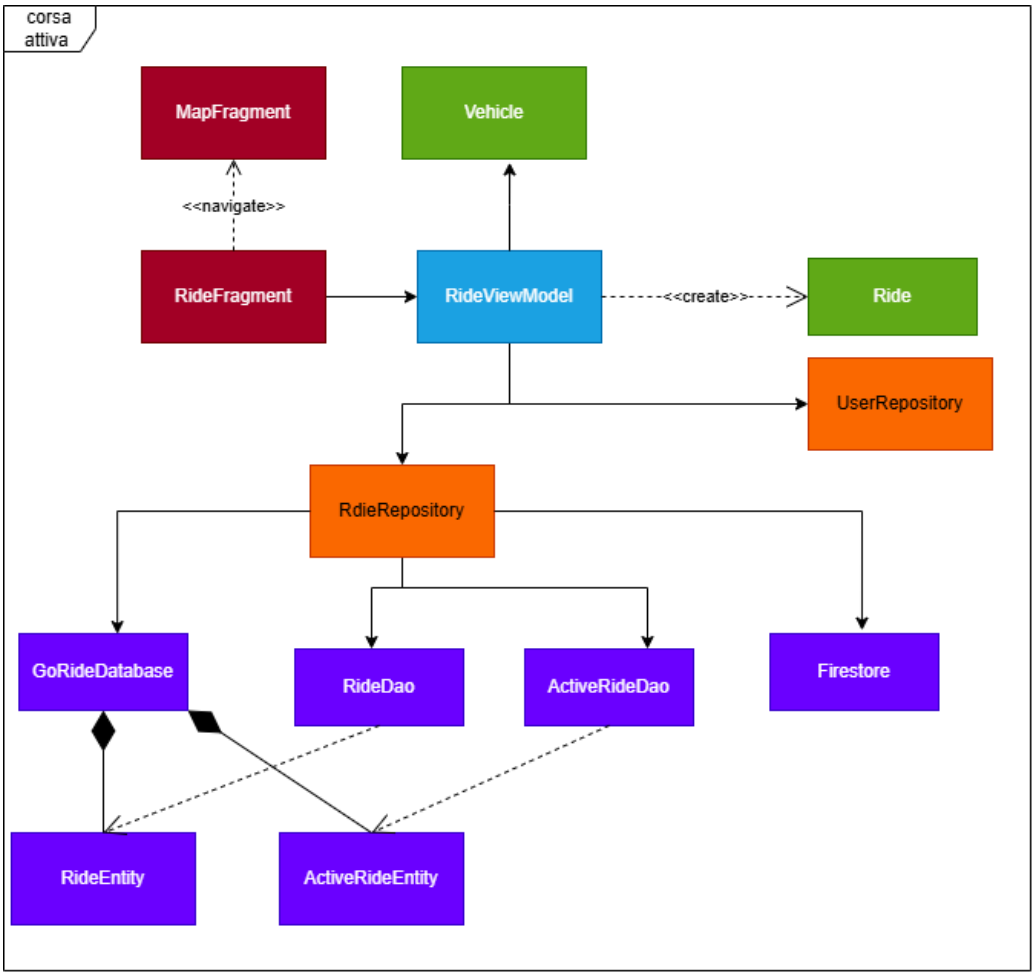


Figure 13: Architettura della corsa attiva

3.3.5 Architettura dello Storico Corse

`HistoryFragment` mostra l'elenco delle corse tramite `RideAdapter`, che per ogni riga usa il layout `item_ride`. `HistoryViewModel` (anch'esso un `AndroidViewModel`) ottiene da `RideRepository` la lista dei `Ride` dell'utente corrente: il Repository la legge dal database Room attraverso `RideDao`, che funge da cache locale e da unica fonte per l'interfaccia, e la mantiene aggiornata sincronizzandola con Firestore. In questo modo lo storico è consultabile anche senza connessione. Selezionando una corsa si apre `RideDetailFragment`, che non ha un ViewModel proprio ma riceve il `Ride` tramite Safe Args e ne mostra durata, mezzo e costo. Quest'area realizza i requisiti RF10 e RF11.

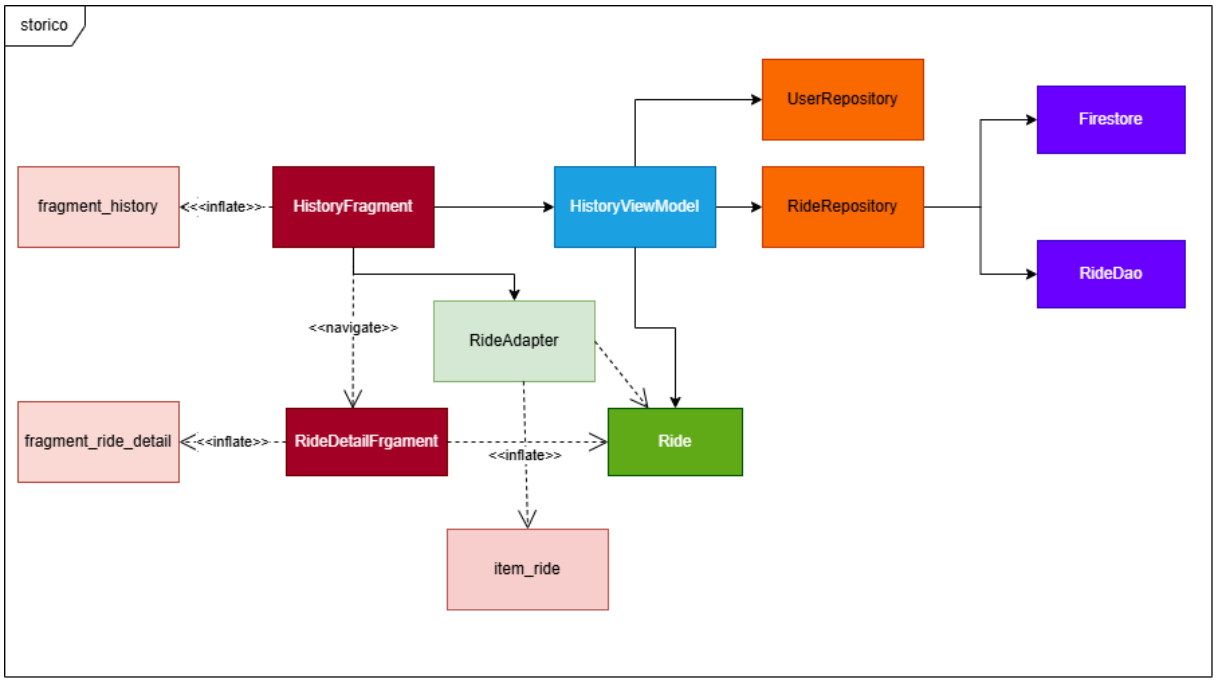


Figure 14: Architettura dello storico corse

3.3.6 Architettura della Community

`CommunityFragment` mostra le recensioni tramite `ReviewAdapter`, che sceglie tra tre layout (`item_review_opinione`, `item_review_guasto` e `item_review_consiglio`) in base al tipo di recensione. `CommunityViewModel` ottiene le recensioni da `ReviewRepository`, che si mette in ascolto della collezione `reviews` di Firestore con un listener in tempo reale: quando un altro utente pubblica una recensione, la lista si aggiorna senza ricaricare la schermata. Per scrivere una nuova recensione `CommunityFragment` porta a `NewReviewFragment`, il cui `NewReviewViewModel` pubblica tramite `ReviewRepository` e recupera i dati dell'autore da `UserRepository`. Quest'area realizza i requisiti RF12, RF13 e RF14.

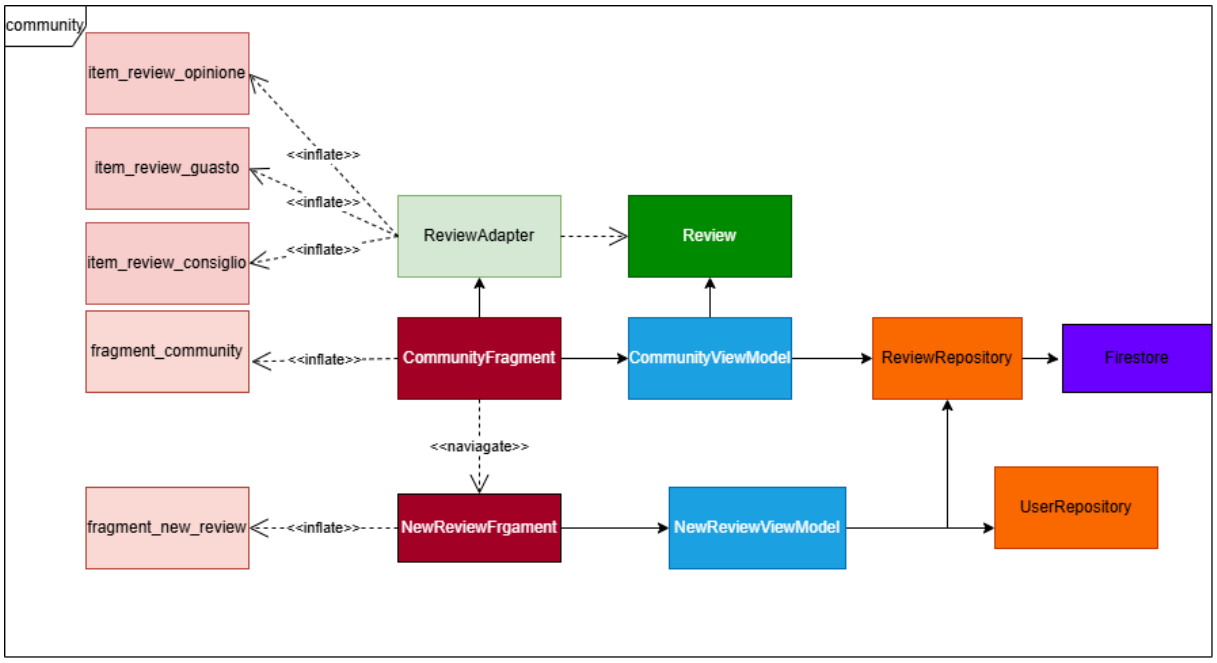


Figure 15: Architettura della community

3.4 Data Storage

I dati di GoRide sono distribuiti su tre sorgenti, ciascuna con un compito preciso: il database locale Room per la cache dello storico e per la corsa in corso, Firebase per account, profili, corse e recensioni, e una sorgente di esempio in memoria per i mezzi. Tutte e tre sono accessibili solo attraverso i Repository.

3.4.1 Database

Per la memorizzazione locale viene utilizzato il database SQLite di Android, gestito tramite Room. Il database serve a due scopi: mantenere una copia locale dello storico, in modo che le corse siano consultabili anche senza connessione, e salvare la corsa in corso, in modo da ripristinare il timer se il sistema termina il processo dell'app. Il database `GoRideDatabase` contiene due entità, riportate nella Tabella 1; l'accesso ai dati avviene tramite i rispettivi DAO, `RideDao` e `ActiveRideDao`.

Entità	Tabella	Attributi
`RideEntity`	`corse`	`id`: String (chiave primaria) `userId`: String `vehicleId`: String `vehicleNome`: String `vehicleTipo`: String `startTimestamp`: Long `endTimestamp`: Long (nullable) `minutiPrepagati`: Int `tariffaSbloccoEuro`: Double `tariffaAlMinutoEuro`: Double `costoTotaleEuro`: Double
`ActiveRideEntity`	`corsa_attiva`	`id`: Int (chiave primaria, sempre 1) `vehicleId`: String `vehicleNome`: String `vehicleTipo`: String `startTimestamp`: Long `scadenzaTimestamp`: Long `minutiPrepagati`: Int `tariffaSbloccoEuro`: Double `tariffaAlMinutoEuro`: Double

Table 1: Struttura delle entità nel database Room

`RideEntity` è la cache dello storico: la UI legge sempre da questa tabella, mentre i dati vengono aggiornati dalla sincronizzazione con Firestore. `ActiveRideEntity` contiene al massimo una riga, perché la chiave primaria è fissa a 1 e ogni salvataggio sostituisce il precedente. Il tipo del mezzo è salvato come stringa, perché Room non gestisce direttamente le enumerazioni.

3.4.2 Firebase

Firebase gestisce la parte online dell'app. Firebase Authentication si occupa degli account (registrazione, accesso e recupero password), mentre Cloud Firestore memorizza i profili degli utenti, le corse concluse e le recensioni della community. La password è conservata solo in Authentication e non compare mai in Firestore.

Firestore contiene tre collezioni:

```
users/{uid}
    nome: String
    cognome: String
    email: String
    telefono: String
    dataRegistrazione: Long

rides/{rideId}
    userId: String
    vehicleId: String
    vehicleNome: String
```

```
    vehicleTipo: String
    startTimestamp: Long
    endTimestamp: Long
    minutiPrepagati: Int
    tariffaSbloccoEuro: Double
    tariffaAlMinutoEuro: Double
    costoTotaleEuro: Double

reviews/{reviewId}
    userId: String
    autoreNome: String
    tipo: String
    testo: String
    timestamp: Long
    vehicleId: String (nullable)
```

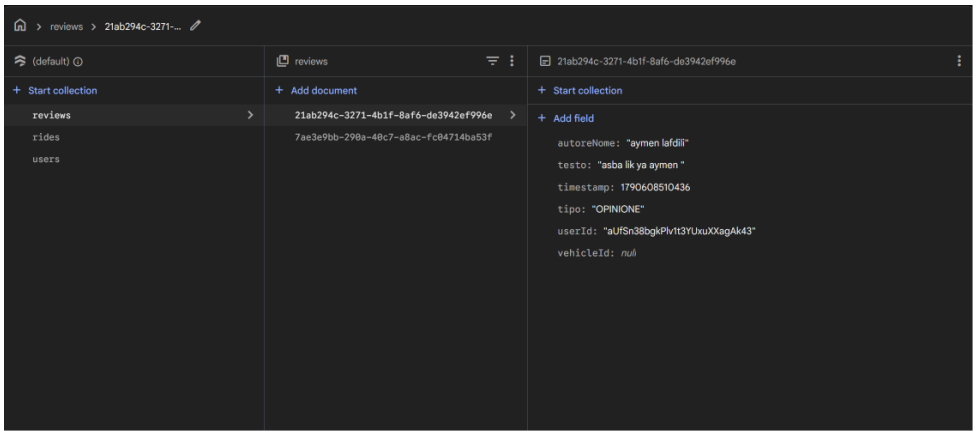


Figure 16: Esempio Struttura della Raccolta e dei Documenti in Firestore

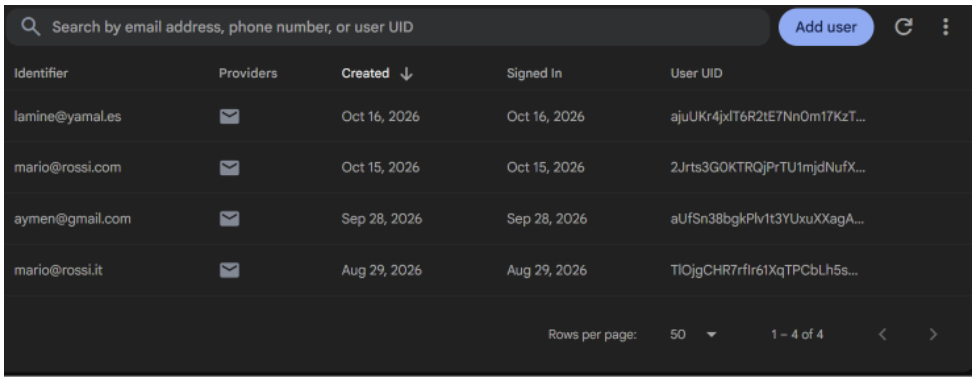


Figure 17: Utenti registrati in Firebase Authentication

3.4.3 Sorgente Dati Mock

I mezzi non sono memorizzati né su Firestore né su Room: la flotta è fornita da `MockVehicleDataSource`, un oggetto singleton che contiene un elenco fisso di biciclette distribuite attorno ad Ancona. Il `VehicleRepository` accede alla sorgente senza conoscerne

l'origine, quindi se in futuro i mezzi provenissero da un server, andrebbe modificato solo questo livello. Ogni mezzo è descritto da identificativo, nome, tipo, coordinate, tariffe e disponibilità; batteria e autonomia sono presenti solo per le e-bike.

Esempio di bici classica:

```
Vehicle(
            id = "BK-001",
            nome = "Bici 001 - Piazza Cavour",
            tipo = VehicleType.CLASSIC,
            lat = 43.6155, lon = 13.5175,
            batteriaPercentuale = null,      // le classiche non hanno batteria
            autonomiaKm = null,
            tariffaSbloccoEuro = 0.50,
            tariffaAlMinutoEuro = 0.15,
            disponibile = true
        ),
```

Esempio di e-bike:

```
 Vehicle(
            id = "EB-001",
            nome = "E-Bike 001 - Corso Garibaldi",
            tipo = VehicleType.EBIKE,
            lat = 43.6142, lon = 13.5133,
            batteriaPercentuale = 87,
            autonomiaKm = 52.0,
            tariffaSbloccoEuro = 1.00,
            tariffaAlMinutoEuro = 0.28,
            disponibile = true
        ),
```

3.5 MockUp UI

3.5.1 NavBar

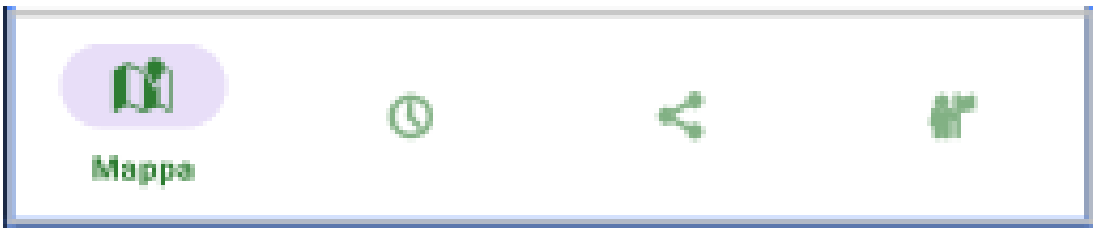


Figure 18: Navbar

3.5.2 Sezione Account

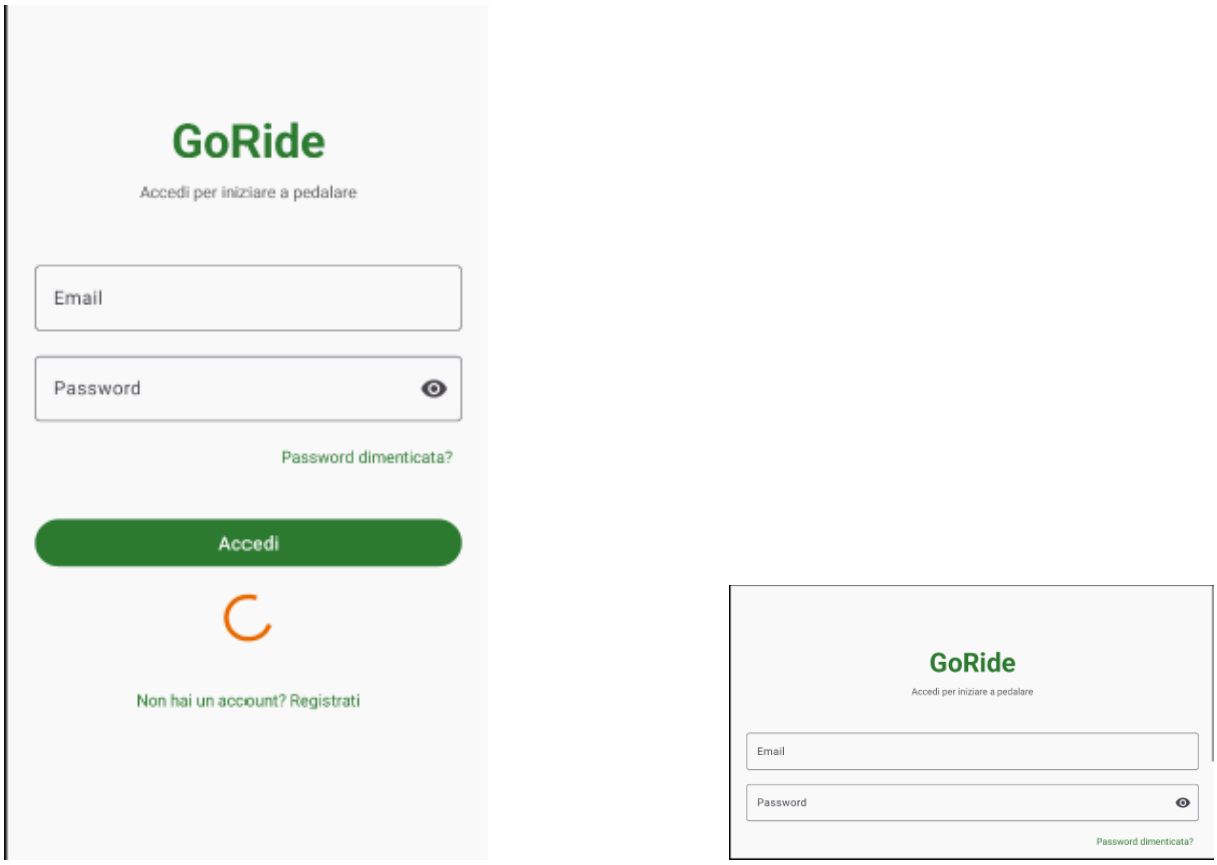


Figure 19: Accesso Portrait e Landscape

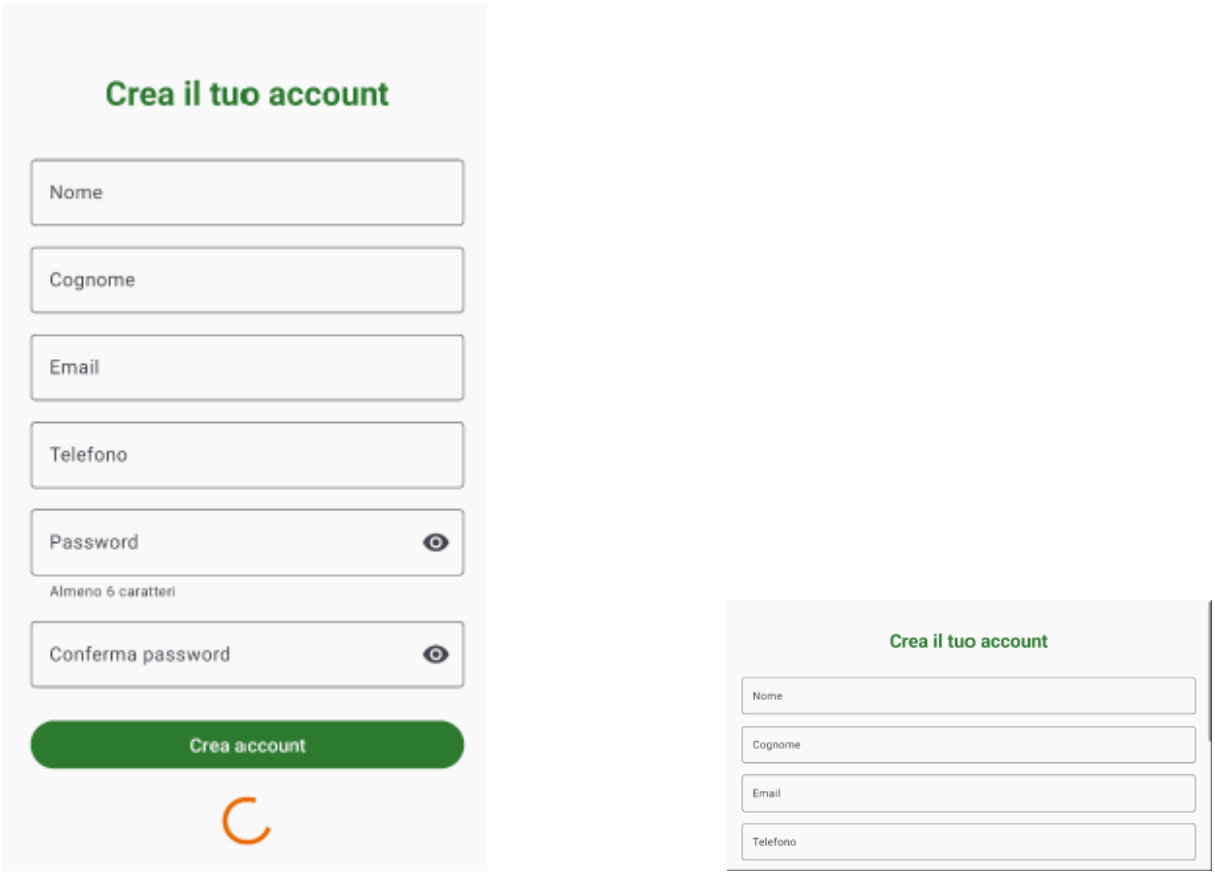


Figure 20: Registrazione Portrait e Landscape

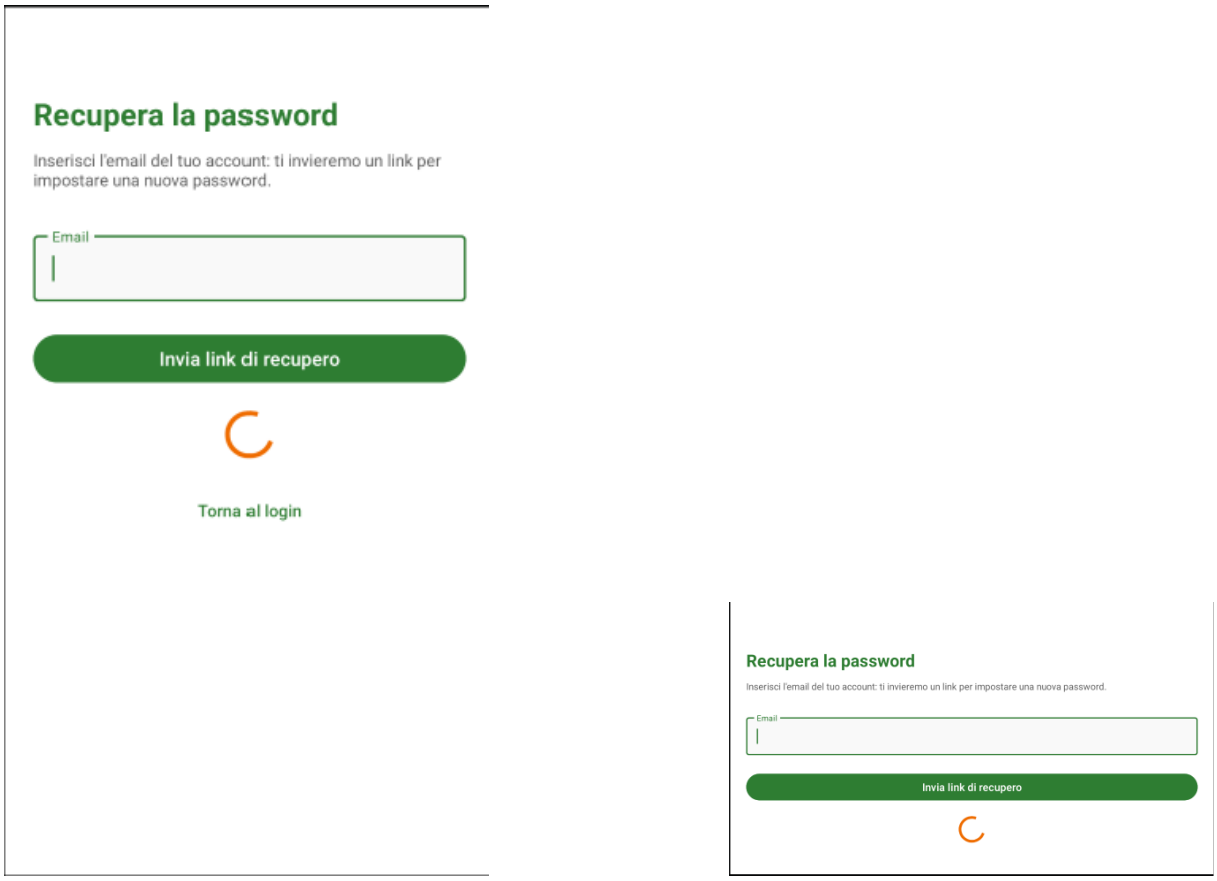


Figure 21: Recupero Password Portrait e Landscape

3.5.3 Sezione Mappa

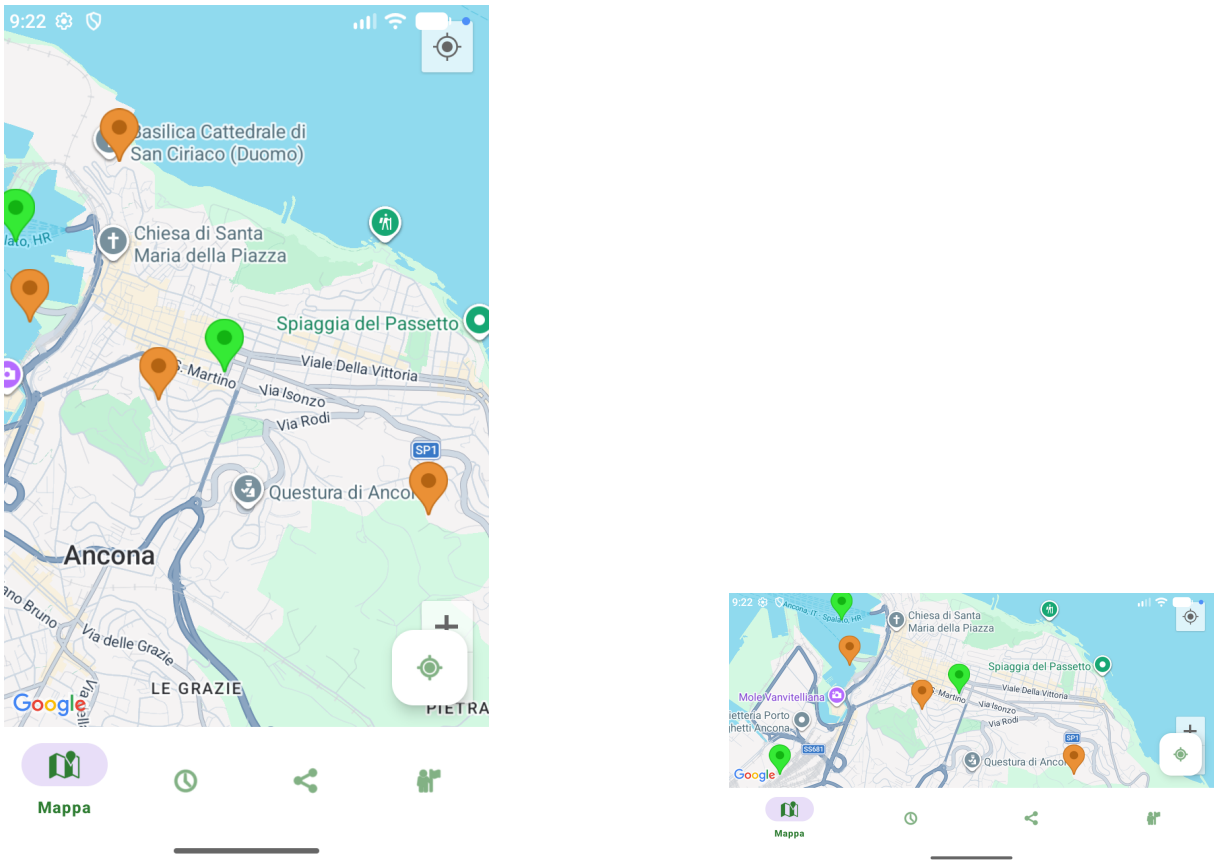


Figure 22: Mappa Portrait e Landscape

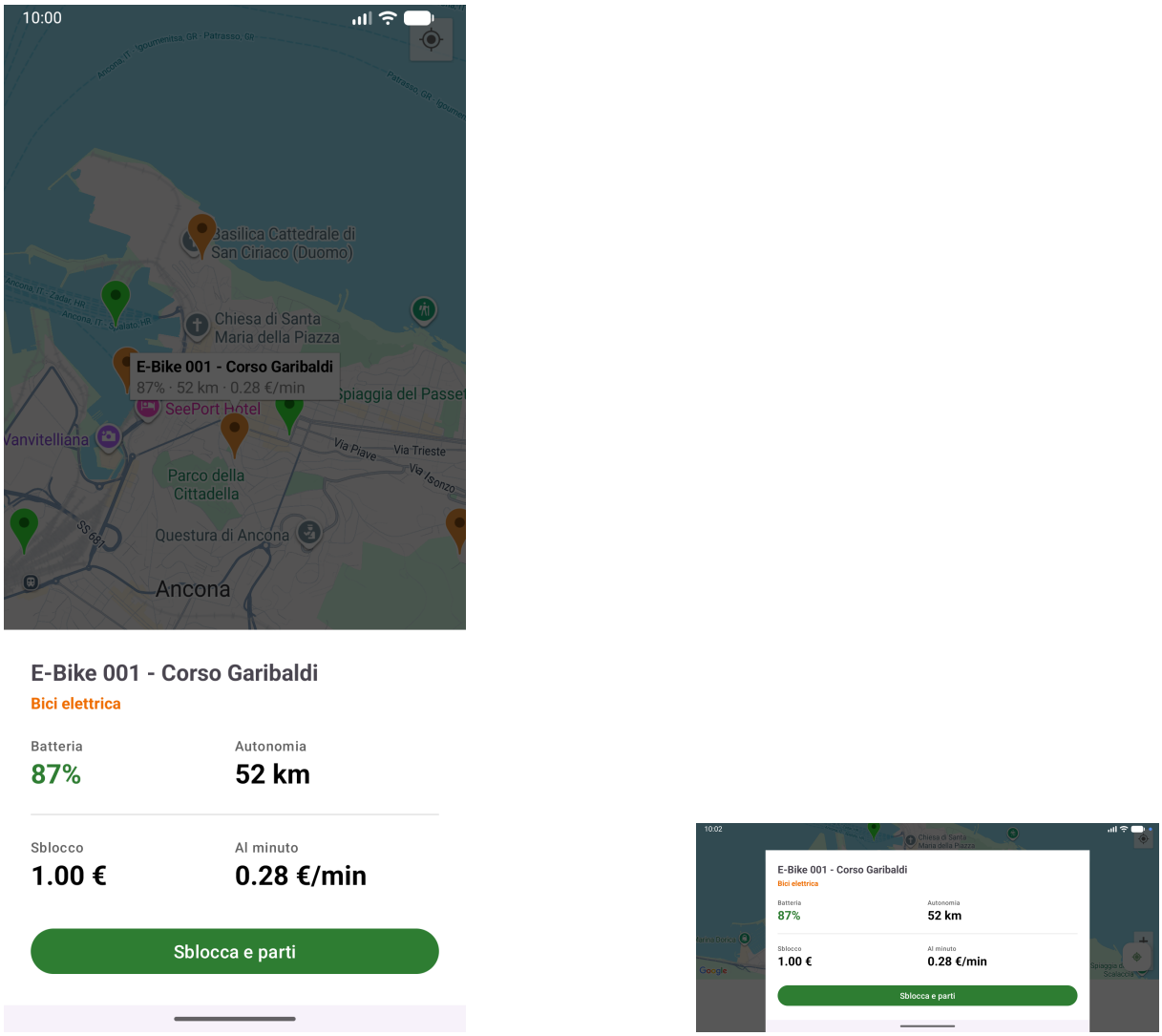


Figure 23: Dettaglio Mezzo Portrait e Landscape

3.5.4 Sezione Corsa

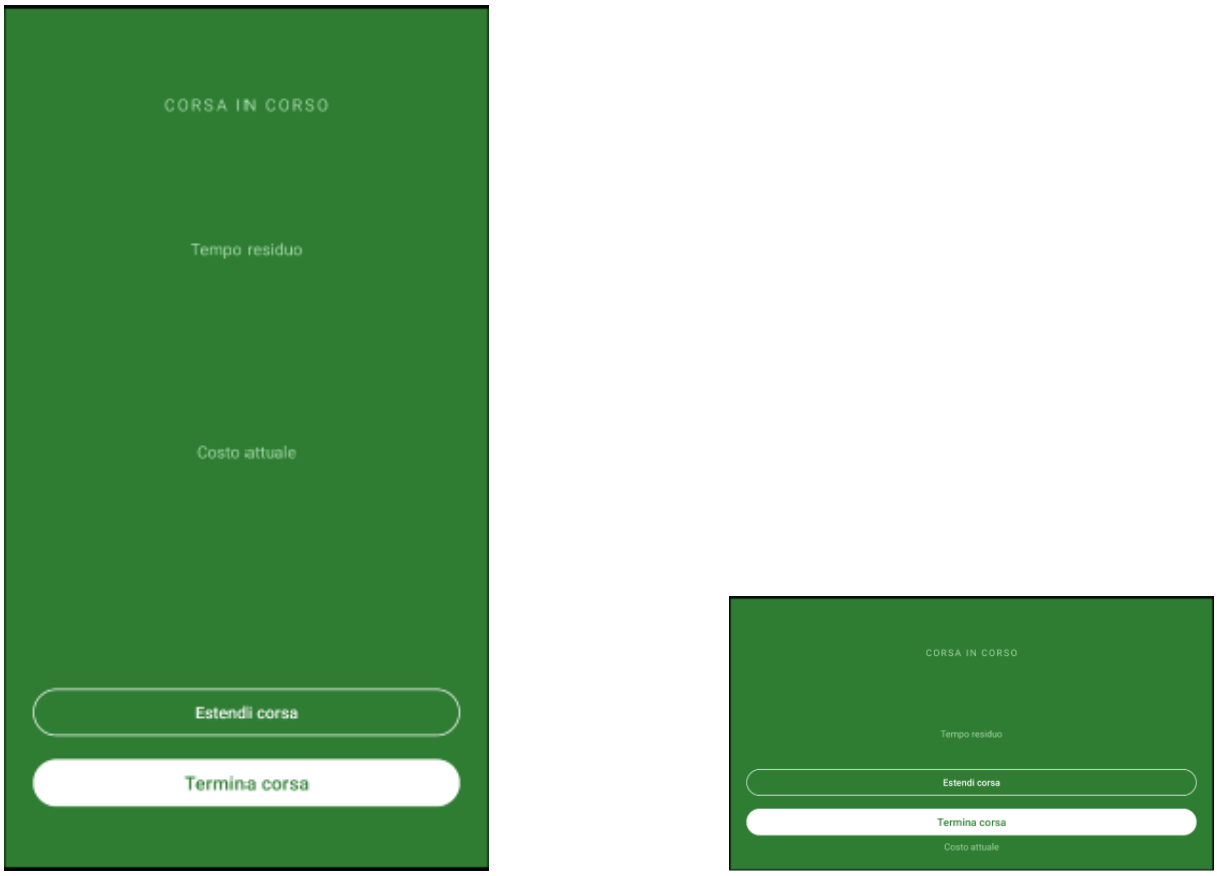


Figure 24: Corsa Portrait e Landscape

3.5.5 Sezione Storico

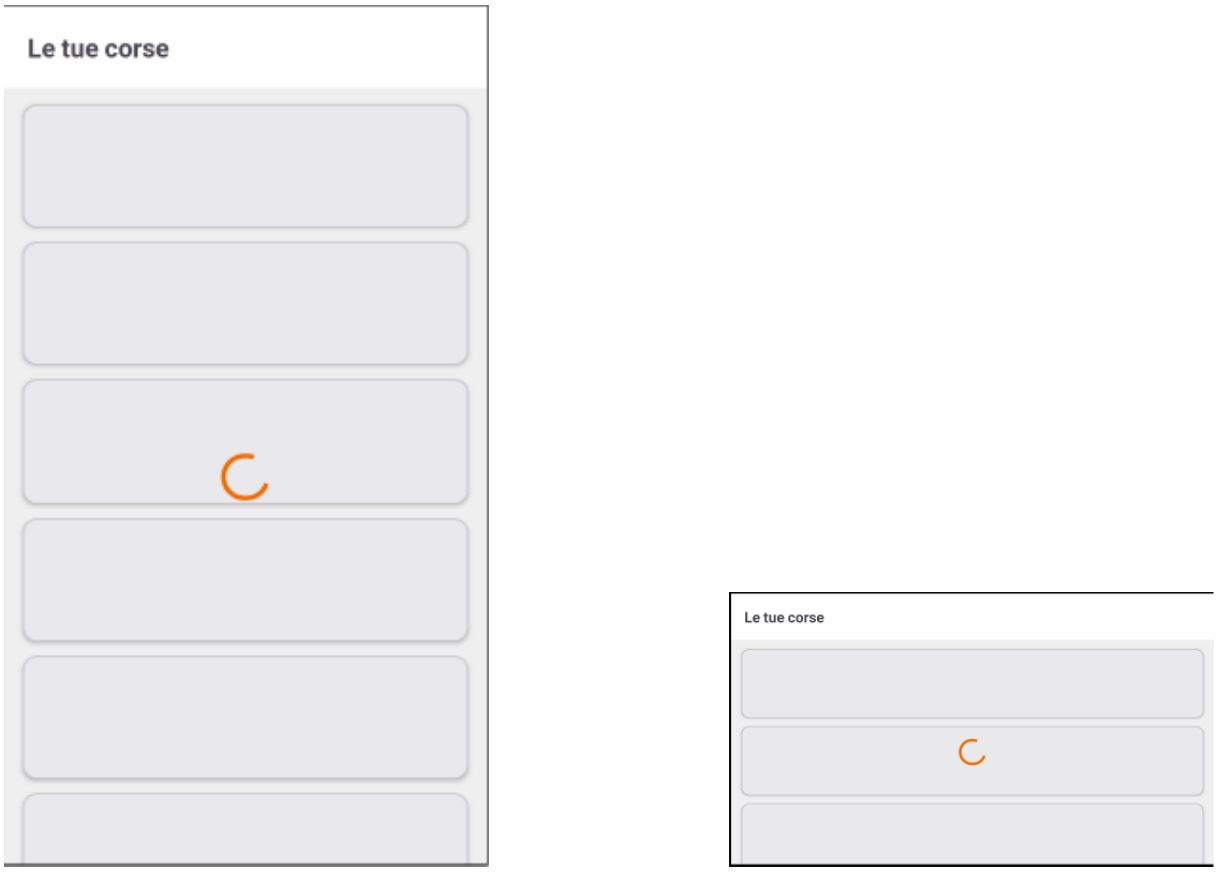


Figure 25: Storico Portrait e Landscape

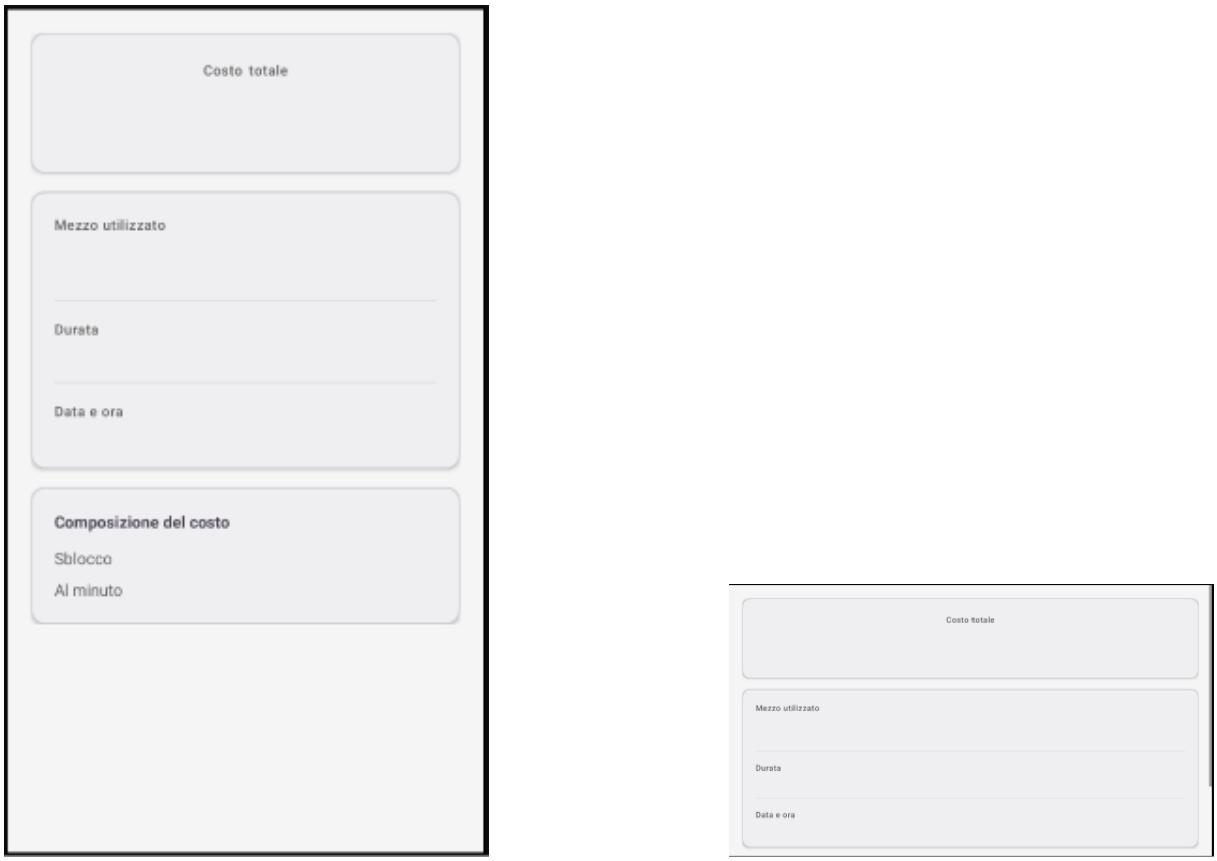


Figure 26: Dettaglio Corsa Portrait e Landscape

3.5.6 Sezione Community

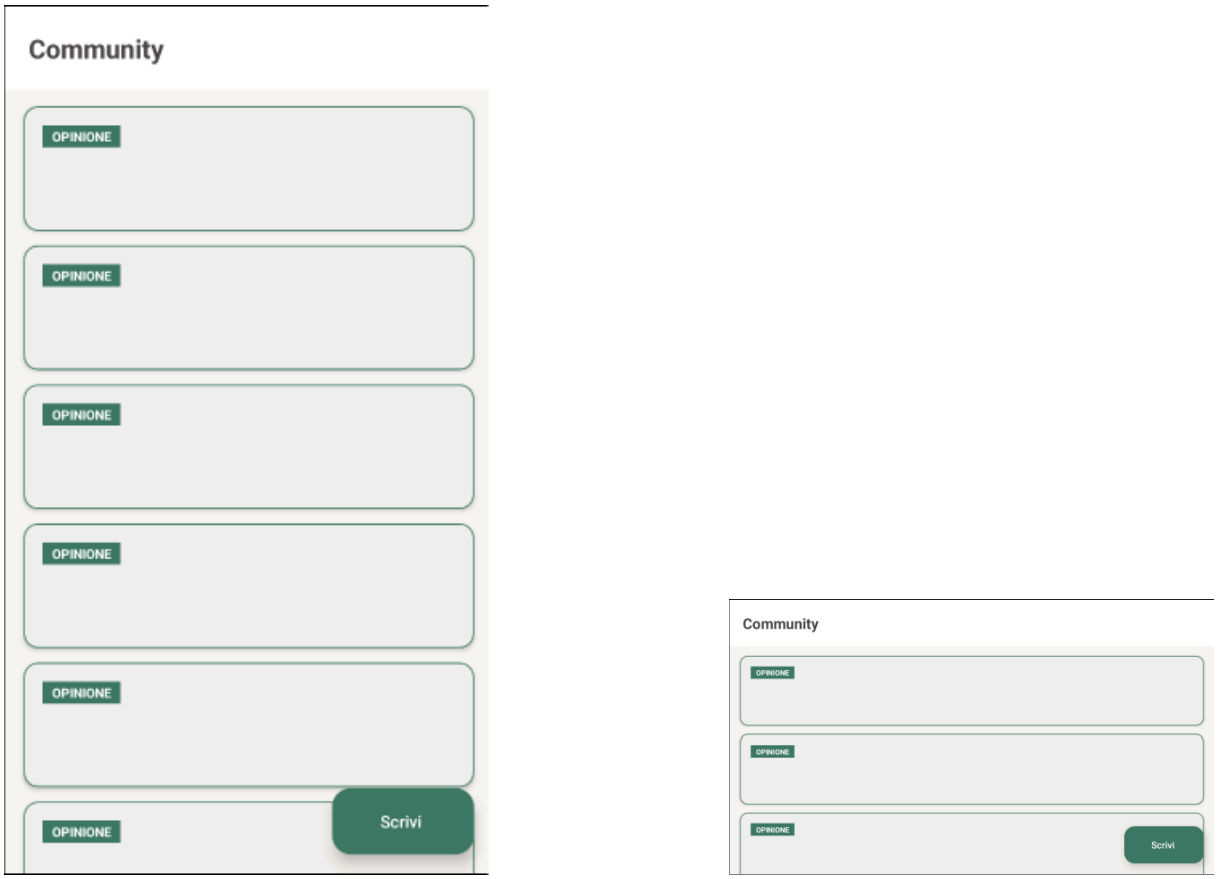


Figure 27: Community Portrait e Landscape

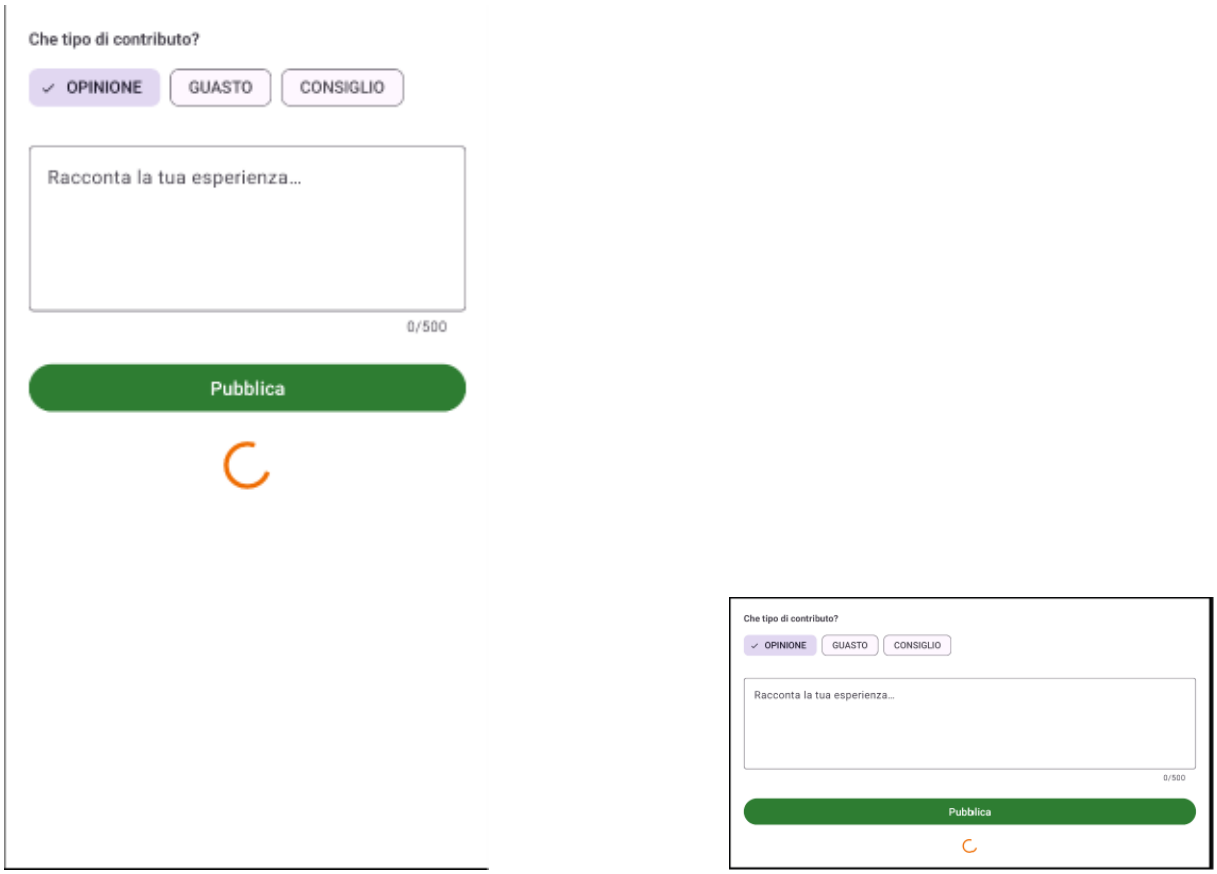


Figure 28: Nuova Recensione Portrait e Landscape

3.5.7 Sezione Profilo

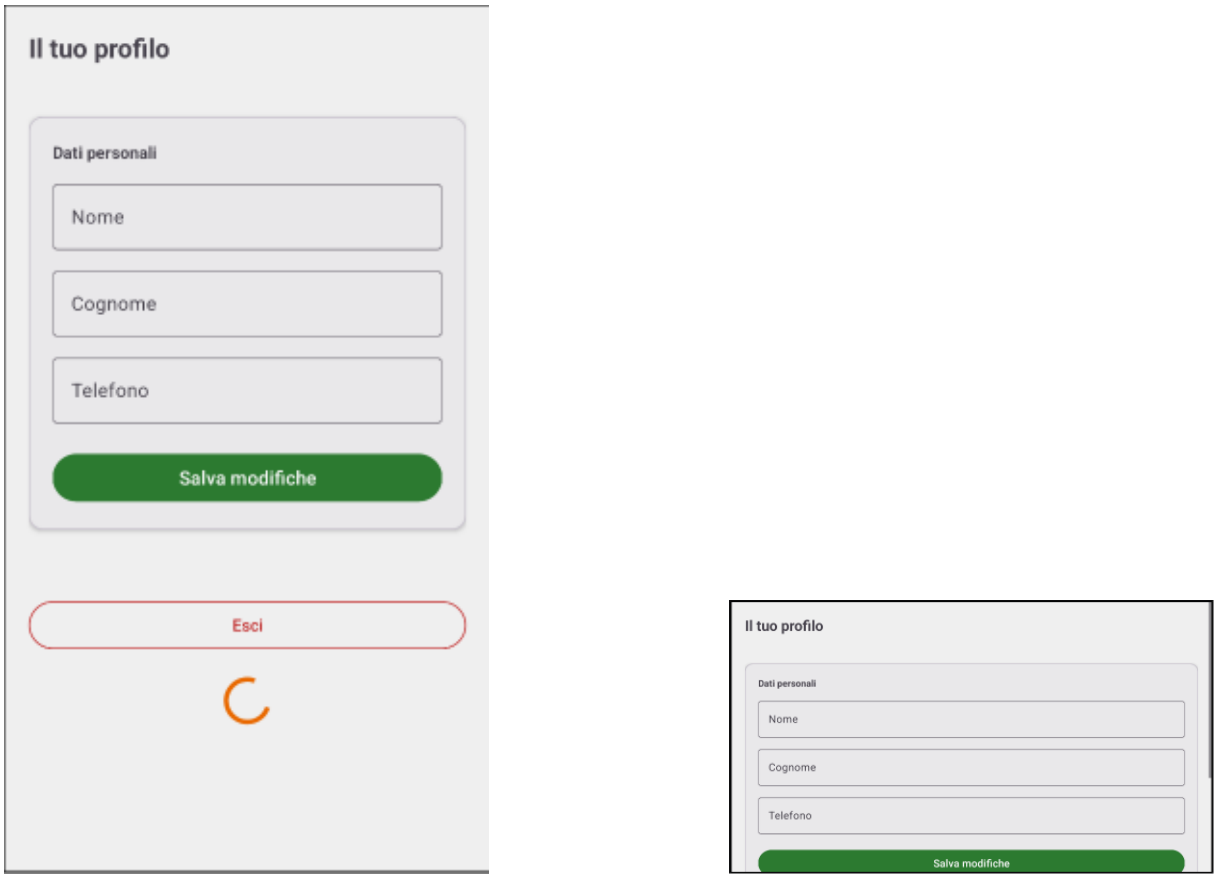


Figure 29: Profilo Portrait e Landscape

3.6 Rappresentazione Grafica dei Casi d'Uso

3.6.1 Acesso e creazione Account

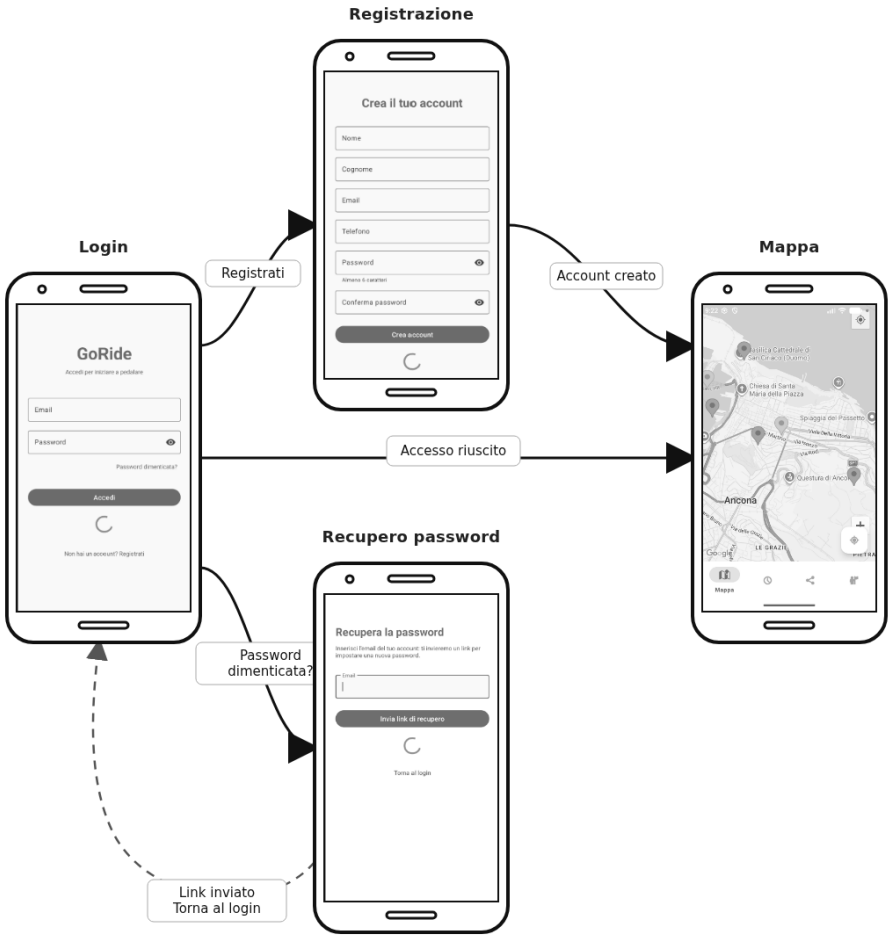


Figure 30: Acesso e creazione Account

3.6.2 Avviamento di una corsa

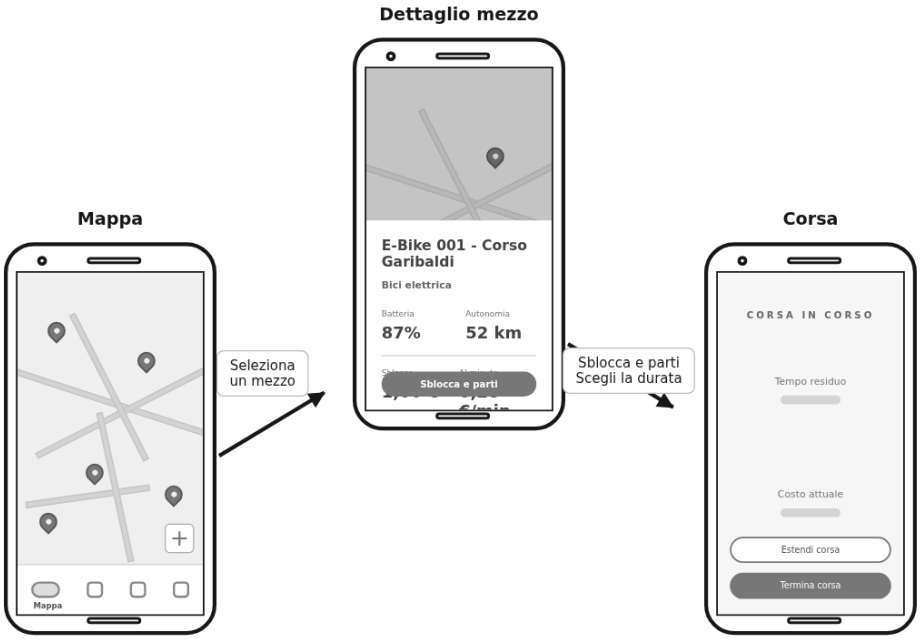


Figure 31: Avviamento di una corsa

3.6.3 Estensioe e terminazione corsa

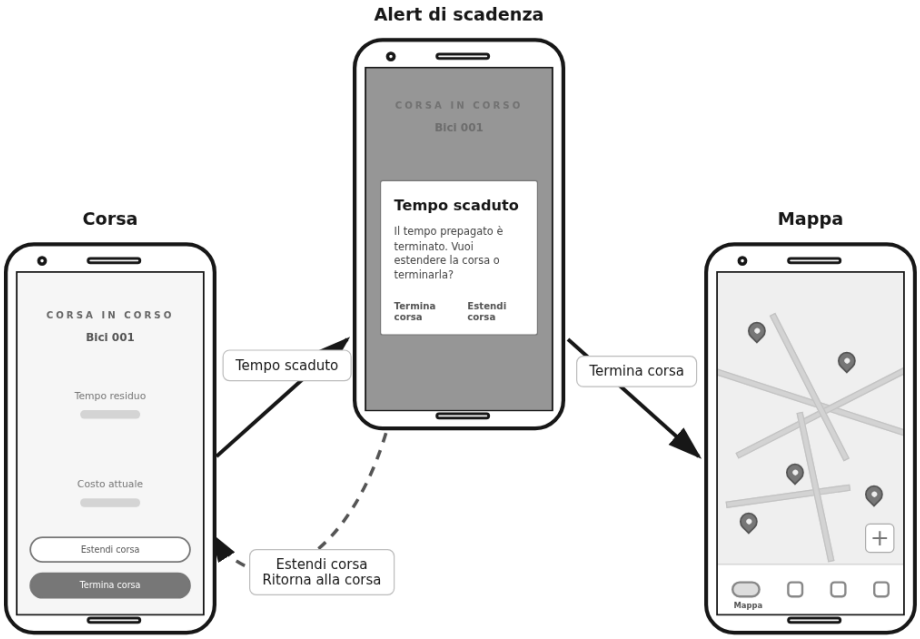


Figure 32: Estensioe e terminazione corsaEstensioe e terminazione corsa

3.6.4 Consultazione storico

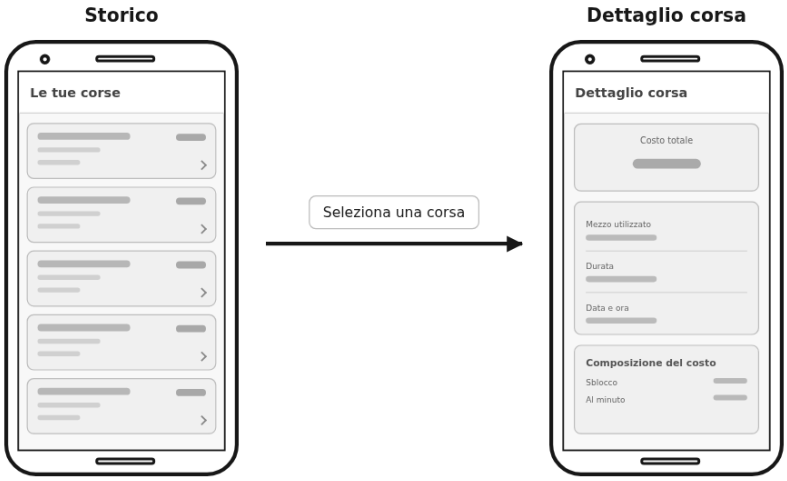


Figure 33: Consultazione storico

3.6.5 Pubblicazione recensione

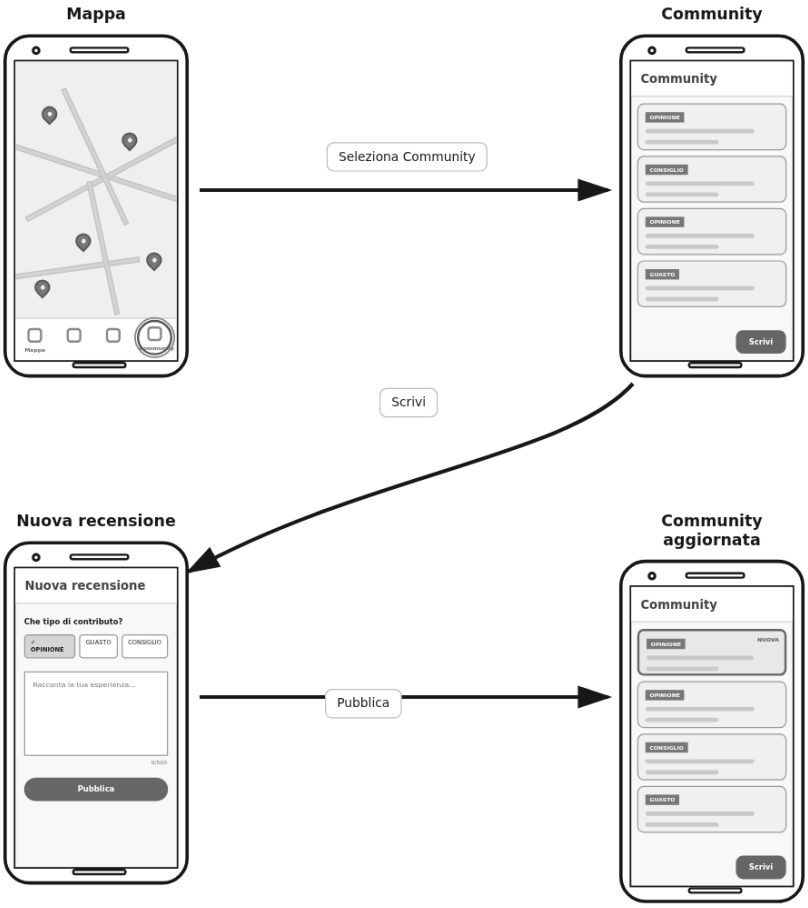


Figure 34: Pubblicazione recensione

3.6.6 Modificazione profilo e logout

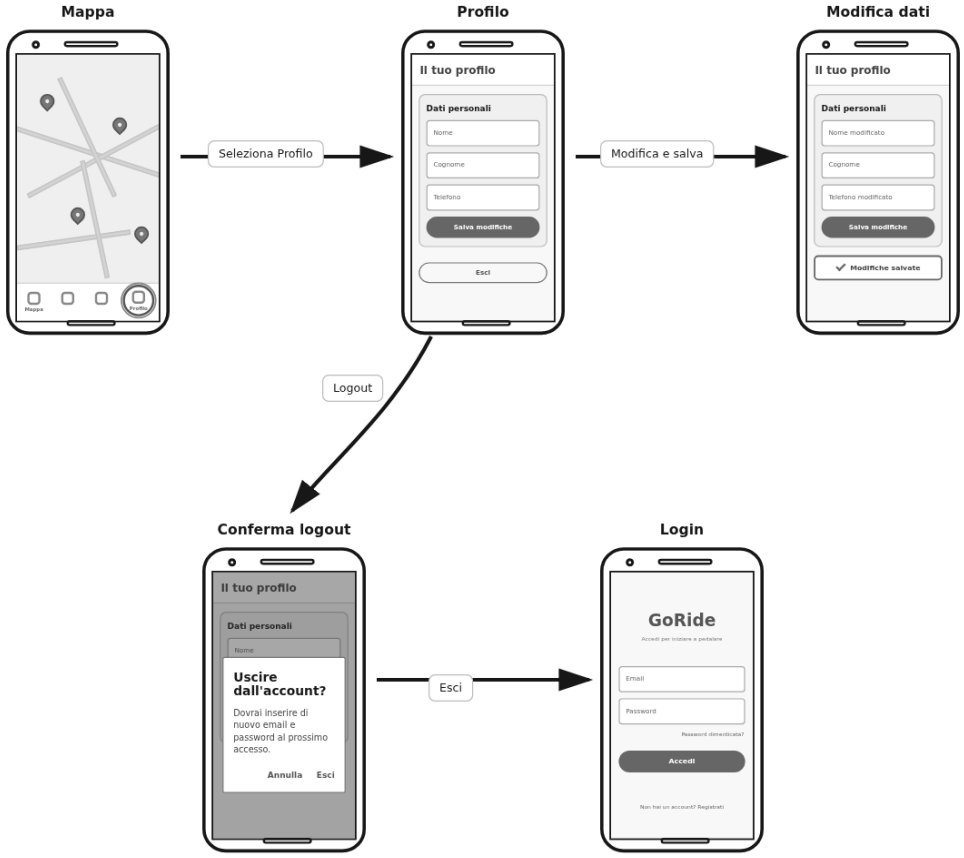


Figure 35: Modificazione profilo e logout

3.7 Approccio e Problematiche di Sviluppo

3.7.1 Bug Risolti

- **Il timer della corsa perdeva secondi quando l'app andava in background:** contando un secondo alla volta, le pause della coroutine in background facevano restare indietro il conteggio. Per risolverlo, all'avvio della corsa si salva l'istante di scadenza (`scadenzaTimestamp`) e a ogni ciclo il tempo residuo viene calcolato come differenza tra la scadenza e l'ora corrente (`System.currentTimeMillis()`). Il timer è quindi sempre corretto, anche dopo molti cicli saltati.
- **Dopo l'estensione della corsa il cronometro scorreva più velocemente:** ogni estensione avviava un secondo timer che scriveva nella stessa `LiveData`. Ora la funzione che avvia il timer cancella sempre il `Job` precedente con `cancel()` prima di crearne uno nuovo. Inoltre, nell'estensione la nuova scadenza parte dal maggiore tra la scadenza attuale e l'ora corrente, così il tempo residuo non va perso e non si ottiene tempo negativo.

- **La corsa andava persa se il sistema terminava il processo:** lo stato vive solo nel ViewModel. Per questo la corsa in corso viene salvata in Room (tabella `corsa_attiva`) e, alla riapertura della schermata, il timer viene ripristinato dalla scadenza salvata. Se la scadenza è passata da più di trenta minuti, la corsa viene scartata.
- **Tornando indietro dal dettaglio del mezzo si veniva riportati subito al dettaglio:** l'observer si riattaccava e rileggeva il mezzo selezionato, ancora presente nella `LiveData`. Dopo la navigazione il ViewModel azzera lo stato (`deselezionaMezzo()`); lo stesso schema (`navigazioneCompletata()`) vale per la navigazione al termine della corsa e dopo il logout.
- **La finestra di scelta della durata compariva durante una corsa già avviata:** ruotando lo schermo, `onViewCreated` veniva rieseguito. Il Fragment ora mostra la scelta solo se nel ViewModel non c'è una corsa attiva e se il ripristino da Room non ne ha trovata una; viene controllato anche `isAdded`, perché la lettura da Room è asincrona e il Fragment potrebbe essere stato nel frattempo chiuso.
- **Lo storico non era visibile senza connessione:** la UI osserva sempre Room, mentre la sincronizzazione con Firestore aggiorna la tabella in background. Al termine di una corsa, inoltre, il salvataggio avviene prima in Room e poi su Firestore, così la corsa compare subito nello storico.

3.7.2 Bug

- La tabella `corsa_attiva` non salva l'identificativo dell'utente: se un altro utente accede dallo stesso dispositivo dopo una chiusura forzata dell'app, potrebbe trovarsi ripristinata la corsa del precedente.
- L'uscita dall'account esegue solo il `signOut` di Firebase e non svuota la cache di Room: lo storico non è visibile ad altri utenti, perché filtrato per `userId`, ma i dati restano sul dispositivo.
- Al termine della corsa l'esito del salvataggio su Firestore non viene controllato: in assenza di rete la corsa resta solo in Room e non viene segnalato alcun errore.
- Se lo schema di Room cambia, il database viene cancellato e ricreato (`fallbackToDestructiveMigration`): accettabile in sviluppo, ma in produzione servirebbero le migrazioni.

3.7.3 Librerie di Terze Parti

Oltre alle librerie di Android Jetpack utilizzate nel corso, sono state aggiunte due dipendenze esterne.

Google Maps SDK, per la visualizzazione della mappa e dei marker dei mezzi in `MapFragment`. La chiave API non è nel codice ma in `local.properties`, e viene inserita nel manifest tramite un placeholder di Gradle.

```
implementation 'com.google.android.gms:play-services-maps:19.0.0'
implementation 'com.google.android.gms:play-services-location:21.3.0'
```

Firebase (Authentication e Cloud Firestore), per account, profili, corse e recensioni. Il file `google-services.json` generato dalla console Firebase è inserito nella cartella `app`. Le API a callback di Firebase sono state collegate alle coroutine con `suspendCancellableCoroutine`, senza aggiungere altre librerie.

```
implementation platform('com.google.firebase:firebase-bom:33.7.0')
implementation 'com.google.firebase:firebase-auth'
implementation 'com.google.firebase:firebase-firestore'
```

3.8 Descrizione Componenti

Tutti i Fragment seguono lo stesso schema: in `onCreateView()` fanno l'inflate del layout con il Data Binding e, se hanno un ViewModel, assegnano `binding.viewModel` e `binding.lifecycleOwner`; in `onDestroyView()` azzerano il binding, perché la View viene distrutta prima del Fragment. Gli observer usano sempre `viewLifecycleOwner`. Per questo, nelle descrizioni sotto, questi metodi non vengono ripetuti.

3.8.1 MainActivity

`class MainActivity : AppCompatActivity()`

Unica Activity dell'app: ospita il NavHost del Navigation Component e la barra di navigazione. Non contiene logica.

`onCreate()` : Recupera il `NavController` dal `NavHostFragment`. Se `UserRepository.isLoggato()` è vero naviga direttamente alla mappa, svuotando il back stack del login. Collega la `BottomNavigationView` al `NavController` e, con un `OnDestinationChangedListener`, nasconde la barra nelle schermate di login, registrazione e recupero password.

3.8.2 Modelli

`data class Vehicle(...) : Parcelable`

Mezzo disponibile al noleggio: id, nome, tipo, coordinate, tariffe, disponibilità e, solo per le e-bike, batteria e autonomia (nullable). È `Parcelable` perché viaggia tra i Fragment con Safe Args.

`val isEbike: Boolean` : Vero se il tipo è `VehicleType.EBIKE`; usato nei layout e nel dettaglio del mezzo.

`enum class VehicleType`

Tipo di mezzo: `CLASSIC` o `EBIKE`.

`data class Ride(...) : Parcelable`

Corsa, in corso o conclusa: id, utente, mezzo, istanti di inizio e fine, minuti prepagati, tariffe e costo totale.

`val isAttiva: Boolean` : Vero finché `endTimestamp` è nullo.

`val durataSecondi: Long` : Durata della corsa in secondi, calcolata da inizio e fine.

`data class Review(...) : Parcelable`

Recensione della community: id, autore, tipo, testo, istante di pubblicazione e mezzo (opzionale).

`enum class ReviewType`

Tipo di recensione: `GUASTO`, `OPINIONE` o `CONSIGLIO`; determina il layout nella lista.

`data class User(...) : Parcelable`

Profilo utente: uid, nome, cognome, email, telefono e data di registrazione. La password non compare mai.

`val nomeCompleto: String` : Nome e cognome uniti.

3.8.3 GoRideDatabase

`abstract class GoRideDatabase : RoomDatabase()`

Database Room dell'app, con le entità `RideEntity` e `ActiveRideEntity`. È un singleton.

`getDatabase(context: Context)` : Restituisce l'unica istanza, creandola con `applicationContext` se non esiste. L'istanza è `@Volatile` e la creazione è protetta da `synchronized`, per evitare due istanze create da thread diversi. Con `fallbackToDestructiveMigration()` il database viene ricreato a ogni cambio di schema.

`rideDao() / activeRideDao()` : Restituiscono i due DAO, generati da Room.

3.8.4 RideDao

`interface RideDao`

Accesso alla tabella `corse`.

`getCorseUtente(userId: String): LiveData<List<RideEntity>>` : Corse dell'utente dalla più recente. Non è `suspend`: Room riesegue la query a ogni modifica della tabella e notifica gli observer.

`getCorsaById(id: String)` : Restituisce una corsa o `null`.

`inserisci(corsa) / inserisciTutte(corse)` : Inserimento con `OnConflictStrategy.REPLACE`.

`cancellaCorseUtente(userId: String)` : Elimina le corse di un utente.

3.8.5 ActiveRideDao

`interface ActiveRideDao`

Accesso alla tabella `corsa_attiva`.

`getCorsaAttiva()` : Legge l'unica riga, o `null`.

`salva(corsa: ActiveRideEntity)` : Salva la corsa in corso, sostituendo la precedente.

`cancella()` : Svuota la tabella.

3.8.6 RideEntity e ActiveRideEntity

`data class RideEntity(...)`

Riga della tabella `corse`. Il tipo del mezzo è salvato come stringa.

`toRide() / fromRide(ride)` : Conversione tra entità e modello di dominio `Ride`, in modo che Room non esca dal livello dati.

`data class ActiveRideEntity(...)`

Riga della tabella `corsa_attiva`, con chiave primaria fissa a 1 (`RIGA_UNICA`) e istante di scadenza.

3.8.7 VehicleRepository e MockVehicleDataSource

`class VehicleRepository`

Unica porta d'accesso ai mezzi. I metodi sono `suspend` e girano su `Dispatchers.IO`; un ritardo di 600 ms simula la rete.

`getMezziDisponibili(): List<Vehicle>` : Restituisce i mezzi dalla sorgente.

`getMezzoById(id: String): Vehicle?` : Restituisce un mezzo o `null`.

`sbloccaMezzo(vehicleId: String): Result<Vehicle>` : Verifica che il mezzo esista e sia libero; restituisce un errore altrimenti.

`object MockVehicleDataSource`

Singleton con l'elenco fisso dei mezzi attorno ad Ancona. Contiene anche le coordinate del centro e lo zoom iniziale della mappa.

`getMezziDisponibili() / getMezzoById(id)` : Restituiscono i mezzi o il singolo mezzo.

3.8.8 UserRepository

`class UserRepository`

Unica classe che usa Firebase Authentication e la collezione `users`. Le API a callback di Firebase sono convertite in funzioni `suspend` con `suspendCancellableCoroutine`, e il risultato è un `Result`.

`isLoggato() / getUidCorrente()` : Dicono se c'è una sessione attiva e ne restituiscono l'uid.

`login(email, password)` : Accesso con email e password.

`registra(nome, cognome, email, telefono, password)` : Crea l'account in Authentication e poi salva il profilo in Firestore, senza la password.

`inviaEmailRecupero(email)` : Chiede a Firebase di inviare la mail di recupero.

`getProfilo()` : Legge il profilo dell'utente corrente come `User`.

`aggiornaProfilo(nome, cognome, telefono)` : Aggiorna i dati anagrafici; email e password non sono modificabili.

`logout()` : Chiude la sessione.

3.8.9 RideRepository

`class RideRepository(context: Context)`

Gestisce le corse con due sorgenti: Firestore come fonte remota e Room come cache. La UI legge sempre da Room.

`getStoricoCorse(userId): LiveData<List<Ride»` : Storico osservabile da Room, con le entità convertite in `Ride`.

`sincronizzaDaFirestore(userId): Result<Unit>` : Scarica le corse dell'utente da Firestore e le inserisce in Room; i documenti malformati vengono scartati.

`salvaCorsa(ride: Ride): Result<String>` : Scrive la corsa prima in Room e poi su Firestore.

`getCorsaById(id)` : Legge una corsa dalla cache.

`salvaCorsaAttiva(...) / getCorsaAttiva() / cancellaCorsaAttiva()` : Salvano, leggono e cancellano la corsa in corso, per il ripristino dopo la chiusura del processo.

3.8.10 ReviewRepository

`class ReviewRepository`

Gestisce la collezione `reviews` di Firestore. Non usa Room: la lista si aggiorna in tempo reale.

`recensioni: LiveData<List<Review»` : Lista osservabile, scritta dal listener.

`iniziaAscolto() / fermaAscolto()` : Avviano e rimuovono il listener sulle ultime 100 recensioni, ordinate per data. Prima di avviarne uno, quello esistente viene rimosso.

`pubblicaRecensione(userId, autoreNome, tipo, testo, vehicleId)` : Salva una nuova recensione con id casuale e istante corrente.

3.8.11 ACCOUNT: LoginFragment e LoginViewModel

`class LoginFragment : Fragment()`

Schermata di accesso. Non contiene logica: i campi sono legati al ViewModel con il two-way binding.

`onViewCreated()` : Dà il focus al campo email e apre la tastiera, e collega i link verso registrazione e recupero password.

`osservaViewModel()` : Mostra gli errori in una `Snackbar` e, a login riuscito, naviga alla mappa.

`class LoginViewModel : ViewModel()`

Stato del login: `email` e `password` (two-way binding), `isLoading`, `errore` e `loginRiuscito`.

`login()` : Valida i campi (vuoti, formato email) e, in una coroutine, chiama il repository. Gestisce l'esito con `fold()`.

`traduciErrore(eccezione)` : Converte i messaggi di Firebase in testi leggibili in italiano.

`erroreMostrato()` / `navigazioneCompletata()` : Azzerano l'errore e l'evento di navigazione dopo l'uso, perché la `LiveData` conserva l'ultimo valore.

3.8.12 ACCOUNT: RegisterFragment e RegisterViewModel

`class RegisterFragment : Fragment()`

Schermata di registrazione. Osserva errori e riuscita; il link "Accedi" torna al login con `navigateUp()`.

`class RegisterViewModel : ViewModel()`

Stato del form: nome, cognome, email, telefono, password e conferma, più `isLoading`, `errore` e `registrazioneRiuscita`.

`registra()` : Valida i dati e, se corretti, chiama `UserRepository.registra()` in una coroutine.

`valida(...)` : Controlla campi vuoti, formato email, lunghezza del telefono (almeno 8 cifre), lunghezza della password (almeno 6 caratteri) e corrispondenza con la conferma. Restituisce il messaggio d'errore o `null`.

`traduciErrore(eccezione)` : Traduce gli errori di Firebase, per esempio email già in uso.

3.8.13 ACCOUNT: ForgotPasswordFragment e ForgotPasswordViewModel

`class ForgotPasswordFragment : Fragment()`

Schermata di recupero password. Dopo l'invio della mail attende 2,5 secondi e torna al login, controllando `isAdded`.

`class ForgotPasswordViewModel : ViewModel()`

Stato con `email`, `isLoading`, `messaggio` ed `emailInviata`.

`inviaEmail()` : Valida l'email e chiama il repository. Il messaggio di successo è volutamente generico, per non rivelare se l'email è registrata.

3.8.14 PROFILO: ProfileFragment e ProfileViewModel

`class ProfileFragment : Fragment()`

Schermata del profilo. Mostra email e data di iscrizione e permette di modificare i dati.

`confermaLogout()` : Mostra un `AlertDialog` di conferma prima di uscire dall'account.

`class ProfileViewModel : ViewModel()`

Stato con `nome`, `cognome` e `telefono` modificabili, più `utente`, `isLoading`, `messaggio` e `logoutEseguito`.

`caricaProfilo()` : Legge il profilo all'avvio e precompila i campi.

`salvaModifiche()` : Valida i campi, aggiorna il profilo e ricarica i dati.

`logout()` : Chiude la sessione e segnala l'evento al Fragment, che torna al login.

3.8.15 MAPPA: MapFragment

`class MapFragment : Fragment()`

Schermata della mappa. È l'unica classe che usa `GoogleMap`: il ViewModel conosce solo la lista di `Vehicle`.

`permessoPosizione` : Launcher dell'Activity Result API per il permesso `ACCESS_FINE_LOCATION`. È una proprietà della classe perché va registrato alla creazione del Fragment. Se il permesso viene negato l'app non si blocca: la mappa resta su Ancona e i mezzi sono visibili.

`inizializzaMappa()` : Con `getMapAsync()` ottiene la mappa, centra la camera su Ancona, imposta il click sui marker (il `tag` del marker contiene il `Vehicle`) e disegna i mezzi già arrivati.

`osservaViewModel()` : Disegna i marker alla ricezione dei mezzi, mostra gli errori con l'azione "Riprova" e, quando un mezzo viene selezionato, naviga al dettaglio con Safe Args e azzera la selezione.

`disegnaMarker(mezzi)` : Cancella i marker e ne crea uno per mezzo: verde per le bici classiche, arancio per le e-bike. Il sottotitolo riporta tariffa e, per le e-bike, batteria e autonomia.

`richiediPermessoPosizione()` : Controlla il permesso con tre rami: già concesso, da motivare con un messaggio, oppure da richiedere.

`abilitaPosizioneUtente()` : Attiva il punto di posizione sulla mappa, gestendo `SecurityException`.

3.8.16 MAPPA: MapViewModel

`class MapViewModel : ViewModel()`

Stato della mappa: `mezzi`, `isLoading`, `errore` e `mezzoSelezionato`.

`caricaMezzi()` : Nell'`init` e al "Riprova", carica i mezzi in una coroutine di `viewModelScope`, gestendo l'errore.

`selezionaMezzo(vehicle) / deselezionaMezzo()` : Impostano e azzerano il mezzo selezionato; l'azzeramento evita che il ritorno al dettaglio si ripeta.

`erroreMostrato()` : Azzera l'errore dopo la visualizzazione.

3.8.17 MAPPA: VehicleDetailFragment

`class VehicleDetailFragment : BottomSheetDialogFragment()`

Dettaglio del mezzo in un bottom sheet, dichiarato nel grafo come `<dialog>`. Riceve il `Vehicle` con Safe Args e non ha un ViewModel.

`onViewCreated()` : Mostra le tariffe; per le e-bike mostra anche batteria e autonomia, con il colore della batteria in base alla carica (alta da 60%, media da 25%). Il pulsante "Sblocca" naviga alla corsa passando il mezzo.

3.8.18 CORSA: RideFragment

`class RideFragment : Fragment()`

Schermata della corsa. Usa `activityViewModels()`, perché il `RideViewModel` deve sopravvivere alla navigazione e non solo alla rotazione.

`onViewCreated()` : Se non c'è una corsa attiva prova a ripristinarne una da Room; se non c'è, mostra la scelta della durata. Collega i pulsanti "Estendi" e "Termina" e intercetta il tasto Indietro, che chiede conferma.

`osservaViewModel()` : Aggiorna il costo, mostra l'alert alla scadenza e, a corsa terminata, torna alla mappa.

`mostraSceltaDurata()` : `AlertDialog` non annullabile con le durate disponibili e il costo di ciascuna.

`mostraAlertScadenza()` : `AlertDialog` non annullabile con "Estendi corsa" e "Termina corsa": l'utente deve scegliere.

`mostraSceltaEstensione()` : Dialog con i minuti aggiuntivi e il relativo costo.

`confermaTermine()` : Dialog di conferma con il costo attuale.

`onDestroyView()` : Chiude l'alert di scadenza, per non lasciare una finestra orfana.

3.8.19 CORSA: RideViewModel

`class RideViewModel(application: Application) : AndroidViewModel(application)`

Cuore dell'app. È un `AndroidViewModel` perché `RideRepository` ha bisogno del `Context` per aprire Room. Il timer non decrementa un contatore ma ricalcola il tempo residuo dalla scadenza.

`avviaCorsa(vehicle, minutiPrepagati)` : Se non c'è già una corsa, imposta inizio e scadenza, salva lo stato in Room e avvia il timer.

`ripristinaCorsaAttiva(): Boolean` : Legge la corsa salvata in Room. La scarta se scaduta da più di 30 minuti; altrimenti ricostruisce lo stato e riavvia il timer.

`avviaTimer()` : Cancella il timer precedente e lancia una coroutine che ogni secondo calcola il tempo residuo e il costo. Alla scadenza imposta `tempoScaduto` ed esce dal ciclo.

`tempoFormattato` : `LiveData` derivata con `map` dai millisecondi rimanenti, in formato mm:ss.

`aggiornaCosto()` : Calcola il costo come tariffa di sblocco più i minuti trascorsi, arrotondati per eccesso, per la tariffa al minuto.

`estendiCorsa(minutiAggiuntivi)` : Sposta la scadenza partendo dal maggiore tra la scadenza attuale e l'ora corrente, salva lo stato e riavvia il timer.

`terminaCorsa()` : Ferma il timer, fissa il costo, crea la `Ride`, la salva con `salvaCorsa()`, cancella la corsa attiva ed emette l'evento di fine.

`salvaStatoSuRoom()` : Salva la corsa attiva in Room.

`alertMostrato() / navigazioneCompletata()` : Azzerano gli eventi dopo l'uso.

`DURATE_DISPONIBILI / ESTENSIONI_DISPONIBILI` : Liste dei minuti selezionabili: 15, 30, 60 e 5, 15, 30.

3.8.20 STORICO: HistoryFragment e HistoryViewModel

`class HistoryFragment : Fragment()`

Elenco delle corse. Il tocco su una corsa apre il dettaglio con Safe Args.

`onViewCreated()` : Crea `RideAdapter` e `LinearLayoutManager`; la lista si aggiorna con `submitList()` e, se vuota, compare lo stato vuoto.

`onDestroyView()` : Sgancia l'adapter dalla `RecyclerView`.

`class HistoryViewModel(application: Application) : AndroidViewModel(application)`

Espone `corse` (da Room), `isLoading` ed `errore`.

`sincronizza()` : Eseguito nell'`init` e al "Riprova": scarica le corse da Firestore. In caso di errore segnala il problema solo se la cache locale è vuota.

3.8.21 STORICO: RideAdapter

`class RideAdapter(onClick: (Ride) -> Unit) : ListAdapter<Ride, RideViewHolder>`

Adapter dello storico. `ListAdapter` calcola le differenze con `DiffUtil`.

`RideViewHolder.bind(ride, onClick)` : Mostra costo, data, durata e tipo di mezzo (colorato) e imposta il click.

`RideViewHolder.from(parent)` : Crea il ViewHolder facendo l'inflate di `item_ride`.

`DiffCallback` : `areItemsTheSame` confronta gli id; `areContentsTheSame` confronta tutti i campi.

3.8.22 STORICO: RideDetailFragment

`class RideDetailFragment : Fragment()`

Dettaglio di una corsa, ricevuta con Safe Args.

`onViewCreated()` : Mostra costo totale, durata, data e ora, tipo di mezzo e tariffe di sblocco e al minuto.

3.8.23 COMMUNITY: CommunityFragment e CommunityViewModel

`class CommunityFragment : Fragment()`

Elenco delle recensioni. Si aggiorna da solo quando un altro utente pubblica.

`onViewCreated()` : Crea `ReviewAdapter`, osserva le recensioni, mostra lo stato vuoto e apre la schermata di nuova recensione con il pulsante flottante.

`class CommunityViewModel : ViewModel()`

Espone `recensioni` da `ReviewRepository`.

`init / onCleared()` : Nell'`init` avvia il listener di Firestore; in `onCleared()` lo ferma, per non lasciare una connessione aperta.

3.8.24 COMMUNITY: NewReviewFragment e NewReviewViewModel

`class NewReviewFragment : Fragment()`

Schermata di scrittura di una recensione. Il tipo si sceglie con un gruppo di chip.

`onViewCreated()` : Traduce il chip selezionato in `ReviewType`, mostra gli errori e, a pubblicazione riuscita, torna indietro.

`class NewReviewViewModel : ViewModel()`

Stato con `testo`, `tipoSelezionato`, `isLoading`, `errore` e `pubblicata`.

`selezionaTipo(tipo)` : Memorizza il tipo scelto.

`pubblica()` : Controlla che il testo non sia vuoto e abbia almeno 10 caratteri, legge il profilo per il nome dell'autore e chiama il repository.

3.8.25 COMMUNITY: ReviewAdapter

`class ReviewAdapter : ListAdapter<Review, ReviewViewHolder>`

Adapter delle recensioni, con tre layout diversi nella stessa lista.

`getItemViewType(position)` : Restituisce un intero diverso per guasto, opinione e consiglio.

`onCreateViewHolder(parent, viewType)` : Fa l'inflate del layout corrispondente al tipo.

`ReviewViewHolder.bind(review)` : Assegna la recensione con `setVariable(BR.review, ...)`, perché il ViewHolder è generico sui tre binding, e scrive autore e data.

3.8.26 Formatters

`object Formatters`

Funzioni di formattazione condivise, per non ripetere la stessa logica nei Fragment.

`durata(secondi)` / `euro(importo)` : Durata come "1h 02m" o "2m 05s"; importo come "4,50 €".

`dataOra(timestamp)` / `dataBreve(timestamp)` : Data completa con ora, oppure solo data.

3.9 Testing

3.9.1 ExampleUnitTest

Unit test eseguiti sulla JVM, senza emulatore, per la logica di base dell'app: le funzioni di formattazione di `Formatters`, la sorgente dati `MockVehicleDataSource` e i modelli `Vehicle` e `Ride`.

1. `testFormattazioneDurata()`: verifica che 125 secondi diventino "2m 05s" e che 3725 secondi diventino "1h 02m".

2. `testFormattazioneEuro()`: verifica che 4,5 diventi "4,50 €" e che 0 diventi "0,00 €".
3. `testMezziDisponibiliFiltraNonDisponibili()`: verifica che `getMezziDisponibili()` restituisca solo mezzi con `disponibile = true` e che il mezzo non disponibile "BK-005" non sia presente.
4. `testProprietaIsEbike()`: verifica che `isEbike` sia vero per un mezzo di tipo `EBIKE` e falso dopo averne cambiato il tipo in `CLASSIC`.
5. `testCalcoloDurataEStatoCorsa()`: verifica che una corsa conclusa dopo 300 secondi abbia `durataSecondi` pari a 300 e `isAttiva` falso, e che la stessa corsa senza istante di fine abbia durata 0 e `isAttiva` vero.

4 Flutter App

4.1 Introduzione

La seconda versione di GoRide è stata sviluppata con Flutter, il framework di Google per creare applicazioni multipiattaforma a partire da un unico codice scritto nel linguaggio Dart. L'app offre le stesse sezioni della versione Kotlin (mappa dei mezzi, storico, community e profilo) con le stesse schermate e gli stessi messaggi per l'utente, ma con una gestione del noleggio semplificata: invece di una corsa con timer, l'utente sceglie in anticipo la durata del noleggio, ne vede subito il prezzo e il noleggio viene registrato direttamente nello storico.

Le due app condividono anche il backend. La versione Flutter usa lo stesso progetto Firebase della versione Kotlin, con le stesse collezioni di Firestore (`users`, `rides`, `reviews`) e gli stessi nomi dei campi; anche i valori delle enumerazioni sono salvati con gli stessi codici (`CLASSIC`, `EBIKE`, `GUASTO`, `OPINIONE`, `CONSIGLIO`). Di conseguenza un utente registrato con un'app può accedere con l'altra, e una recensione pubblicata da Kotlin compare in tempo reale nella community della versione Flutter, e viceversa.

Ciò che cambia è il modo in cui l'app è costruita. In Flutter l'interfaccia non è descritta in file XML ma direttamente nel codice, come un albero di widget; le schermate non sono Fragment ospitati da un'Activity ma widget a cui si naviga tramite rotte con nome. Per lo stato è stato scelto un approccio semplice, vicino a quello visto a lezione: ogni schermata è uno `StatefulWidget` che aggiorna la propria interfaccia con `setState()` e mostra i dati asincroni con `FutureBuilder` e `StreamBuilder`; non sono quindi necessari ViewModel. Il pattern Repository resta invece identico: quattro Repository sono l'unica porta verso Firebase, il database locale e la sorgente dei mezzi.

4.2 Analisi dei Requisiti

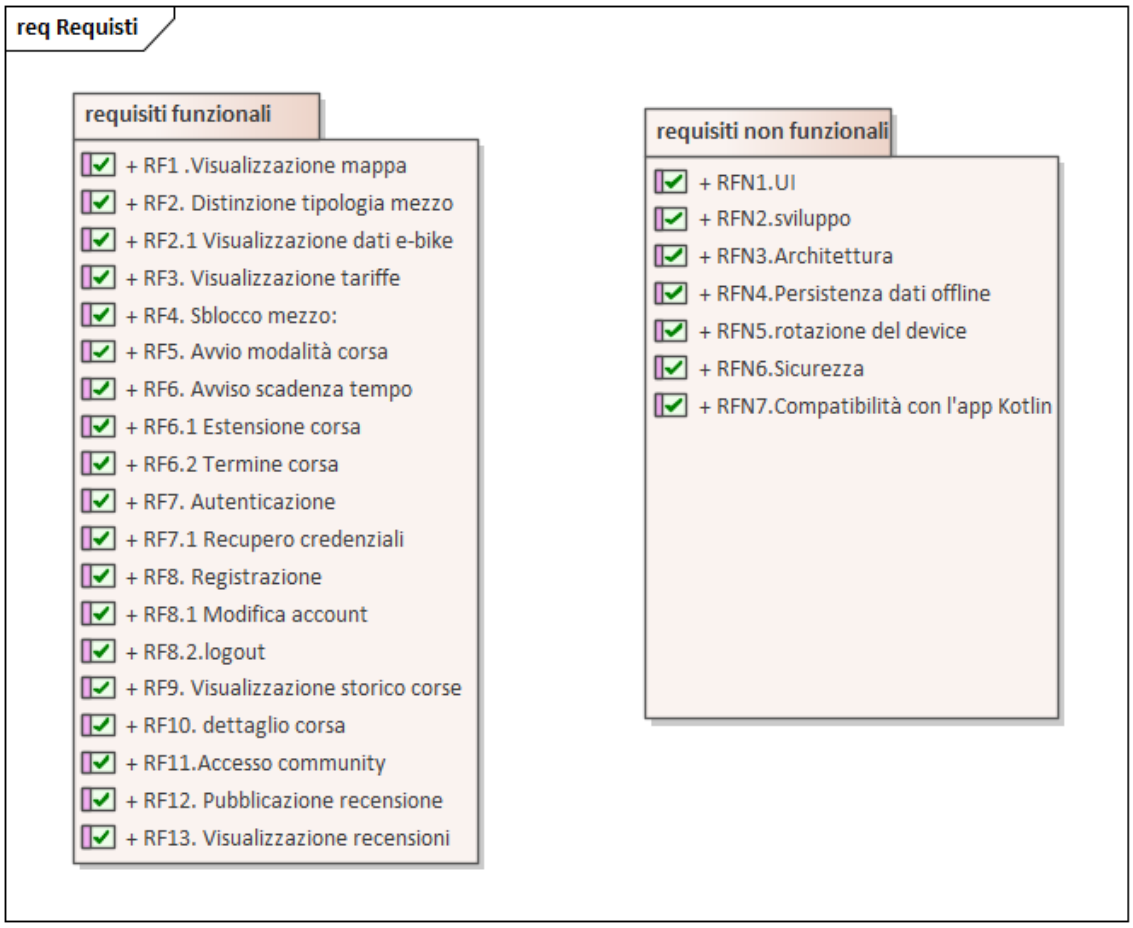


Figure 36: Requisiti di Sistema

I requisiti funzionali coincidono in gran parte con quelli dell'app Kotlin. Cambia la gestione della corsa: al posto di sblocco, timer, avviso di scadenza, estensione e termine, la versione Flutter prevede un noleggio a durata prefissata (RF4–RF6). Cambiano inoltre i requisiti non funzionali legati alla tecnologia.

4.2.1 REQUISITI FUNZIONALI

RF1. Visualizzazione mappa : L'utente deve poter visualizzare su una mappa i mezzi disponibili nelle vicinanze

RF2. Distinzione tipologia mezzo : L'utente deve poter distinguere sulla mappa le biciclette classiche dalle e-bike

RF2.1. Visualizzazione dati e-bike : Per ogni e-bike l'utente deve poter visualizzare la percentuale di batteria e l'autonomia residua in chilometri

RF3. Visualizzazione tariffe : L'utente deve poter visualizzare, prima del noleggio, la tariffa di sblocco e la tariffa al minuto del mezzo selezionato

RF4. Noleggio mezzo : L'utente deve poter avviare il noleggio del mezzo selezionato dalla schermata di dettaglio

RF5. Scelta durata e prezzo : L'utente deve poter scegliere la durata del noleggio (15, 30 o 60 minuti) e visualizzare per ciascuna il prezzo, pari alla tariffa di sblocco più i minuti per la tariffa al minuto

RF6. Registrazione noleggio : Dopo la scelta della durata il sistema deve registrare il noleggio nello storico, con mezzo, durata e prezzo

RF7. Autenticazione : L'utente deve poter autenticarsi inserendo email e password

RF7.1. Recupero credenziali : L'utente deve poter recuperare la password ricevendo una email con il link per impostarne una nuova

RF8. Registrazione : L'utente deve poter registrarsi inserendo nome, cognome, email, telefono e una password di almeno 6 caratteri, da confermare

RF8.1. Modifica account : L'utente deve poter visualizzare i propri dati dal profilo e modificarne nome, cognome e telefono

RF8.2. Uscita dall'account : L'utente deve poter uscire dal proprio account, previa conferma, tornando alla schermata di accesso

RF9. Visualizzazione storico corse : L'utente deve poter visualizzare l'elenco delle proprie corse concluse, dalla più recente, anche senza connessione

RF10. Dettaglio corsa : L'utente deve poter visualizzare per ogni corsa conclusa la durata, il mezzo utilizzato e il costo, con la composizione del costo

RF11. Accesso community : L'utente autenticato deve poter accedere alla sezione community

RF12. Pubblicazione recensione : L'utente deve poter pubblicare una recensione scegliendone il tipo (guasto, opinione o consiglio) e inserendo il testo

RF13. Visualizzazione recensioni : L'utente deve poter visualizzare le recensioni pubblicate dagli utenti, con autore e data, aggiornate in tempo reale

4.2.2 REQUISITI NON FUNZIONALI

RNF1. UI : L'interfaccia deve essere semplice ed intuitiva e riprendere la grafica dell'app Kotlin (colori, icone, disposizione delle sezioni), così che l'utente ritrovi la stessa esperienza

RNF2. Sviluppo : Il linguaggio di sviluppo è Dart, con il framework Flutter; la piattaforma di destinazione è Android

RNF3. Architettura : Le schermate sono `StatefulWidget` che gestiscono il proprio stato con `setState()`, `FutureBuilder` e `StreamBuilder`; i Repository sono l'unica porta verso i dati e vengono condivisi tra le schermate tramite un `InheritedWidget` (`AppScope`); la navigazione usa rotte con nome

RNF4. Persistenza dati offline : Lo storico dei noleggi è salvato in locale in un database SQLite gestito con il pacchetto sqflite, con la stessa tabella `corse` dell'app Kotlin, in modo da essere disponibile anche senza connessione

RNF5. Rotazione del device : Alla rotazione dello schermo e al cambio di sezione l'app deve mantenere il proprio stato; in Flutter lo `State` delle schermate non viene ricreato alla rotazione, e le quattro sezioni principali restano in vita grazie all'`IndexedStack` di HomeShell

RNF6. Sicurezza : La password è gestita solo da Firebase Authentication e non viene mai salvata in Firestore; la chiave di Google Maps è tenuta fuori dal codice versionato, nel file `local.properties`

RNF7. Compatibilità con l'app Kotlin : L'app usa lo stesso progetto Firebase e lo stesso formato dei dati della versione Kotlin, in modo che account, corse e recensioni siano condivisi tra le due app

4.3 Casi d'Uso

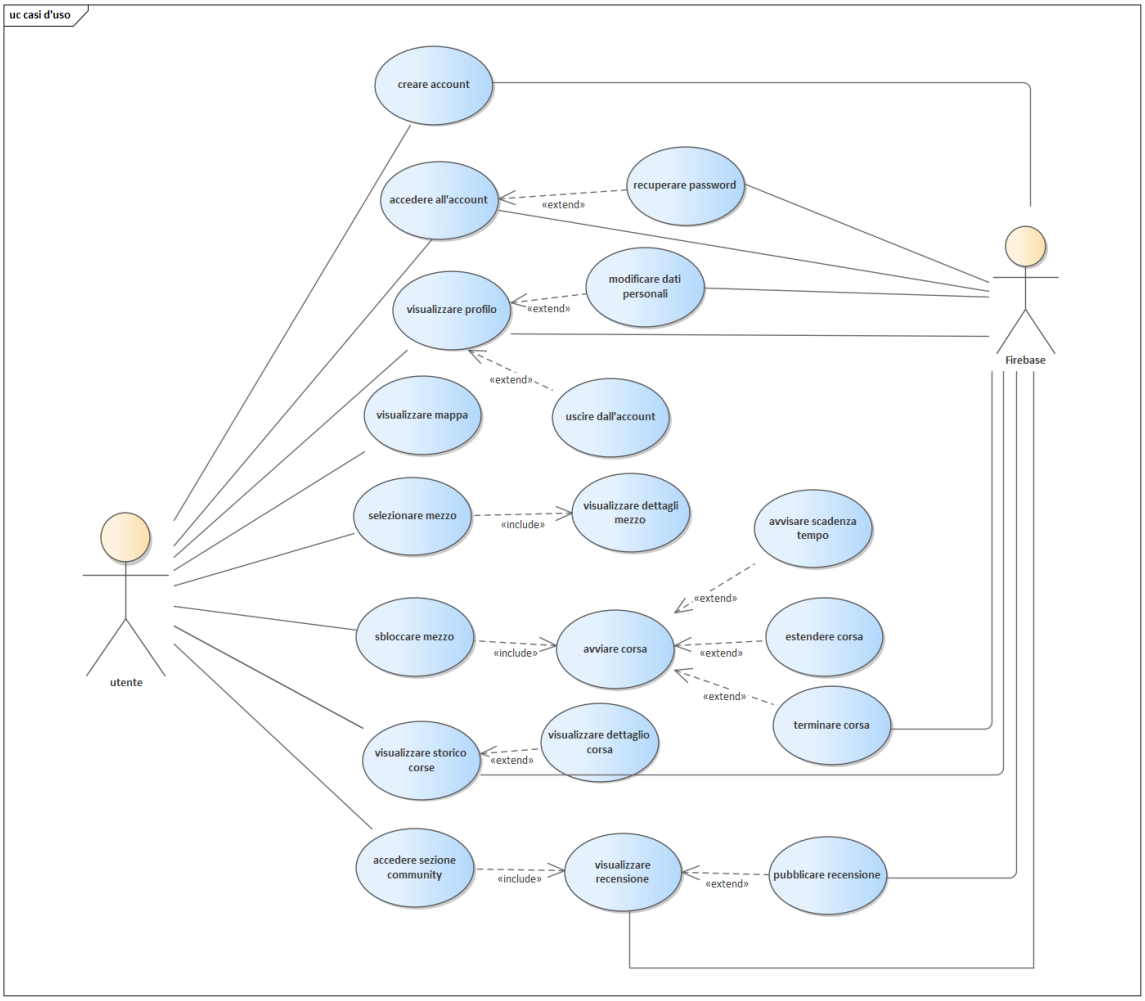


Figure 37: Casi d'Uso

I casi d'uso della versione Flutter coincidono in gran parte con quelli dell'app Kotlin, descritti nella sezione 3.2: le aree Gestione Account, Gestione Profilo, Gestione Storico e Gestione Community hanno gli stessi casi d'uso, con gli stessi attori, le stesse relazioni e le stesse sequenze di eventi. Cambia la gestione della corsa, sostituita da un noleggio a durata prefissata, e la mappa non usa più la posizione dell'utente. Per le aree invariate si riportano solo i diagrammi; i casi d'uso nuovi sono descritti per intero.

Cambia inoltre il modo in cui i passi sono realizzati: le finestre di dialogo sono `AlertDialog` e `SimpleDialog` mostrati con `showDialog()`, il dettaglio del mezzo è un bottom sheet aperto con `showModalBottomSheet()` e i passaggi tra le schermate avvengono con il `Navigator` e le rotte con nome.

4.3.1 UC: Gestione Account

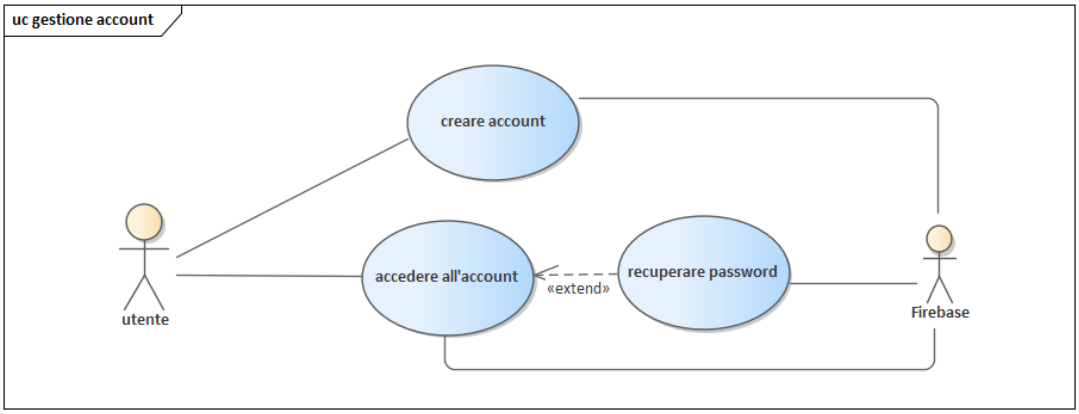


Figure 38: Gestione Account

I casi d'uso Creare account, Accedere all'account e Recuperare password sono uguali a quelli della versione Kotlin (sezione 3.2.1).

4.3.2 UC: Gestione Profilo

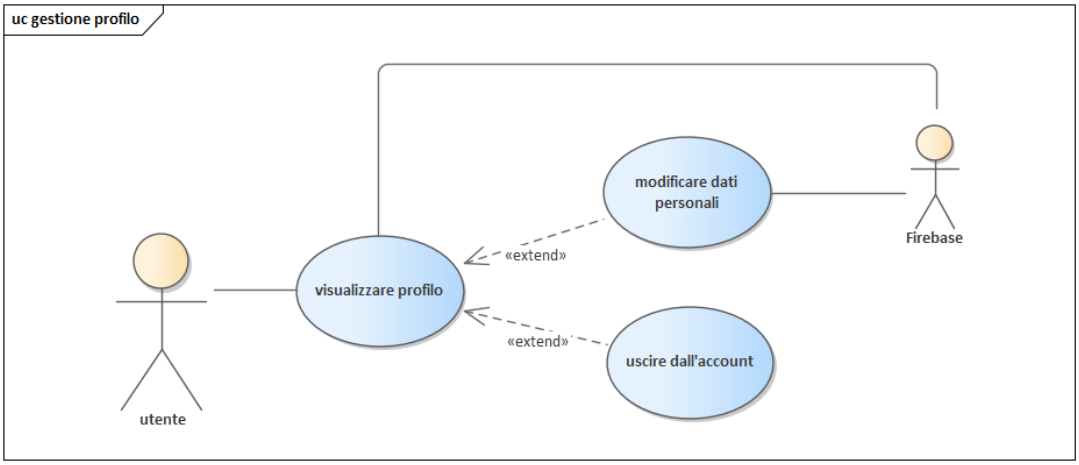


Figure 39: Gestione Profilo

I casi d'uso Visualizzare profilo, Modificare dati personali e Uscire dall'account sono uguali a quelli della versione Kotlin (sezione 3.2.2).

4.3.3 UC: Gestione Mappa e Mezzi

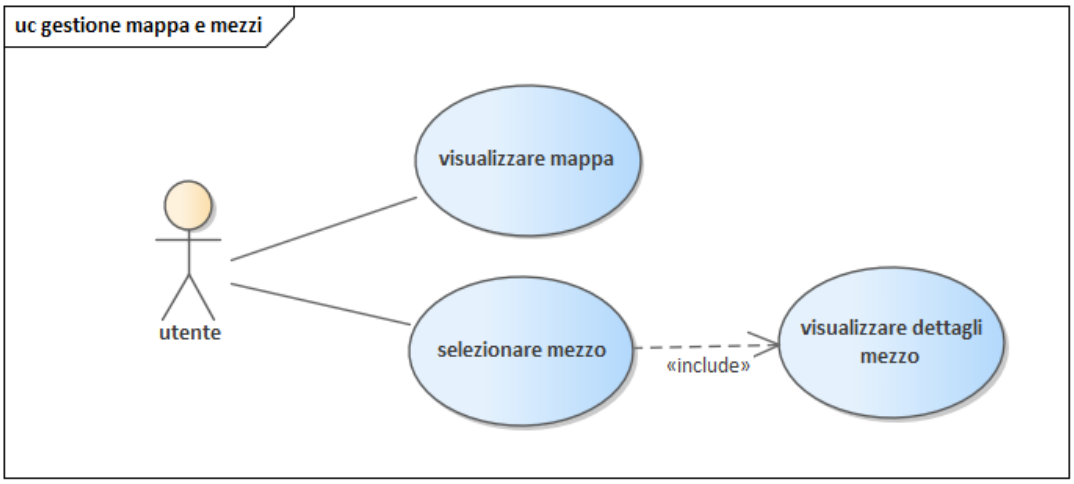


Figure 40: Gestione Mappa e Mezzi

I casi d'uso sono gli stessi della versione Kotlin (sezione 3.2.3), con due differenze: in Visualizzare mappa il sistema non chiede il permesso di accedere alla posizione e la mappa resta centrata su Ancona con tutti i mezzi visibili; in Visualizzare dettagli mezzo il pulsante del pannello porta al caso d'uso Noleggiare mezzo invece che allo sblocco.

4.3.4 UC: Gestione Corsa

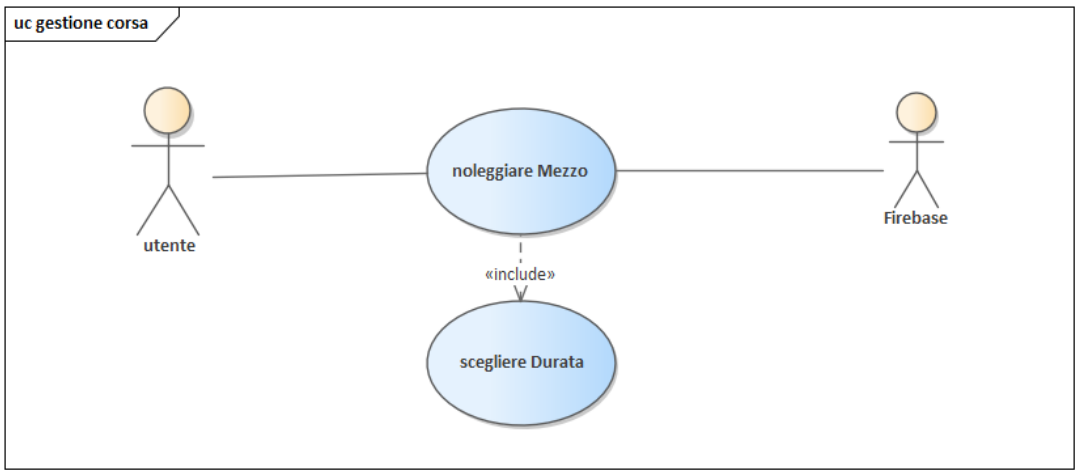


Figure 41: Gestione Corsa

Questi casi d'uso sostituiscono quelli della gestione della corsa della versione Kotlin (sbloccare mezzo, avviare corsa, avvisare scadenza tempo, estendere corsa e terminare

corsa): invece di una corsa con timer, l'utente sceglie in anticipo la durata e il noleggio viene registrato subito. L'attore secondario Firebase interviene nel salvataggio del noleggio.

Casi d'uso: Noleggiare mezzo

Questo caso d'uso si verifica quando l'utente, dopo aver consultato il dettaglio di un mezzo, decide di noleggiarlo.

Pre-condizioni: L'utente è autenticato e visualizza il dettaglio di un mezzo.

Post-condizioni: Il noleggio è salvato nello storico locale e su Firebase e l'utente visualizza la mappa.

Sequenza di eventi principali:

Il caso d'uso inizia quando l'utente seleziona il pulsante "Noleggia" nel dettaglio del mezzo.

1. include(Scegliere durata).
2. Il sistema crea il noleggio con mezzo, istante di inizio, durata, tariffe e prezzo.
3. Il sistema salva il noleggio nel database locale.
4. Il sistema chiede a Firebase di salvare il noleggio.
5. Il sistema mostra il messaggio "Noleggio registrato: lo trovi nello storico" e l'utente resta sulla mappa.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 1 se l'utente chiude la finestra senza scegliere una durata: il noleggio non viene creato e l'utente resta sulla mappa. Inizia al punto 4 se Firebase non conferma il salvataggio, ad esempio per rete assente: il noleggio resta comunque nello storico locale.

Casi d'uso: Scegliere durata

Questo caso d'uso mostra le durate acquistabili con il relativo prezzo. È incluso in Noleggiare mezzo.

Pre-condizioni: L'utente ha scelto di noleggiare un mezzo.

Post-condizioni: La durata del noleggio è scelta.

Sequenza di eventi principali:

Il caso d'uso inizia quando il sistema apre la finestra di scelta della durata.

1. Il sistema propone le durate disponibili (15, 30 o 60 minuti), indicando per ciascuna il prezzo, pari alla tariffa di sblocco più i minuti per la tariffa al minuto.
2. L'utente sceglie una durata.

Sequenza degli eventi alternativi

La sequenza alternativa inizia al punto 2 se l'utente chiude la finestra: la scelta è annullata.

4.3.5 UC: Gestione Storico

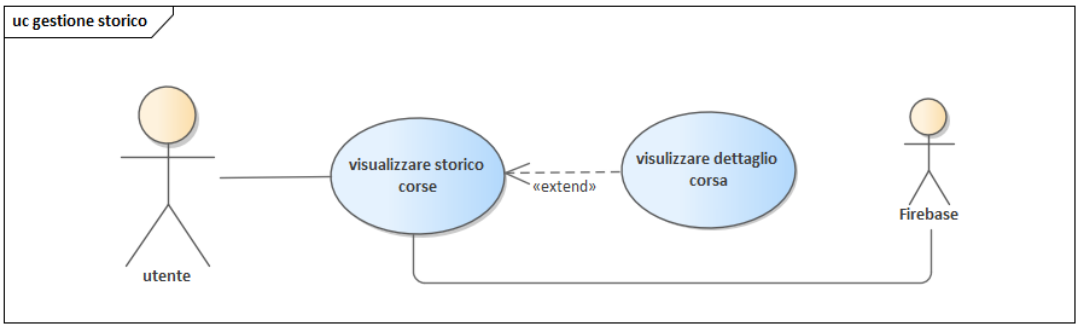


Figure 42: Gestione Storico

I casi d'uso Visualizzare storico corse e Visualizzare dettaglio corsa sono uguali a quelli della versione Kotlin (sezione 3.2.5). L'unica differenza è che l'elenco viene mostrato dopo il tentativo di sincronizzazione con Firebase, invece che prima; i noleggi registrati compaiono nello storico come le corse concluse.

4.3.6 UC: Gestione Community

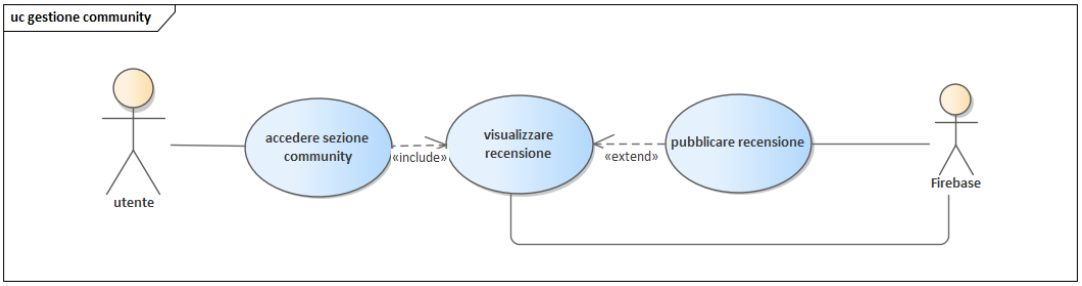


Figure 43: Gestione Community

I casi d'uso Accedere alla sezione community, Visualizzare recensione e Pubblicare recensione sono uguali a quelli della versione Kotlin (sezione 3.2.6).

4.4 Mappa dell'Architettura

Per l'architettura della versione Flutter è stato mantenuto il pattern Repository, mentre lo stato è gestito direttamente dalle schermate. Ogni schermata è uno `StatefulWidget` che raccoglie gli input dell'utente, chiama il Repository giusto e mostra il risultato con `setState()`, `FutureBuilder` o `StreamBuilder`. I quattro Repository (UserRepository, VehicleRepository, RideRepository e ReviewRepository) sono l'unico punto di accesso ai dati e nascondono le sorgenti utilizzate: Firebase Authentication, Firestore, il database locale sqflite (GoRideDatabase) e la sorgente di esempio MockVehicleDataSource. I tipi di Firebase non escono mai dai Repository: alle schermate arrivano solo i modelli Vehicle, Ride, Review e AppUser. I Repository sono creati una sola volta da GoRideApp e resi disponibili a tutte le schermate dall'AppScope.

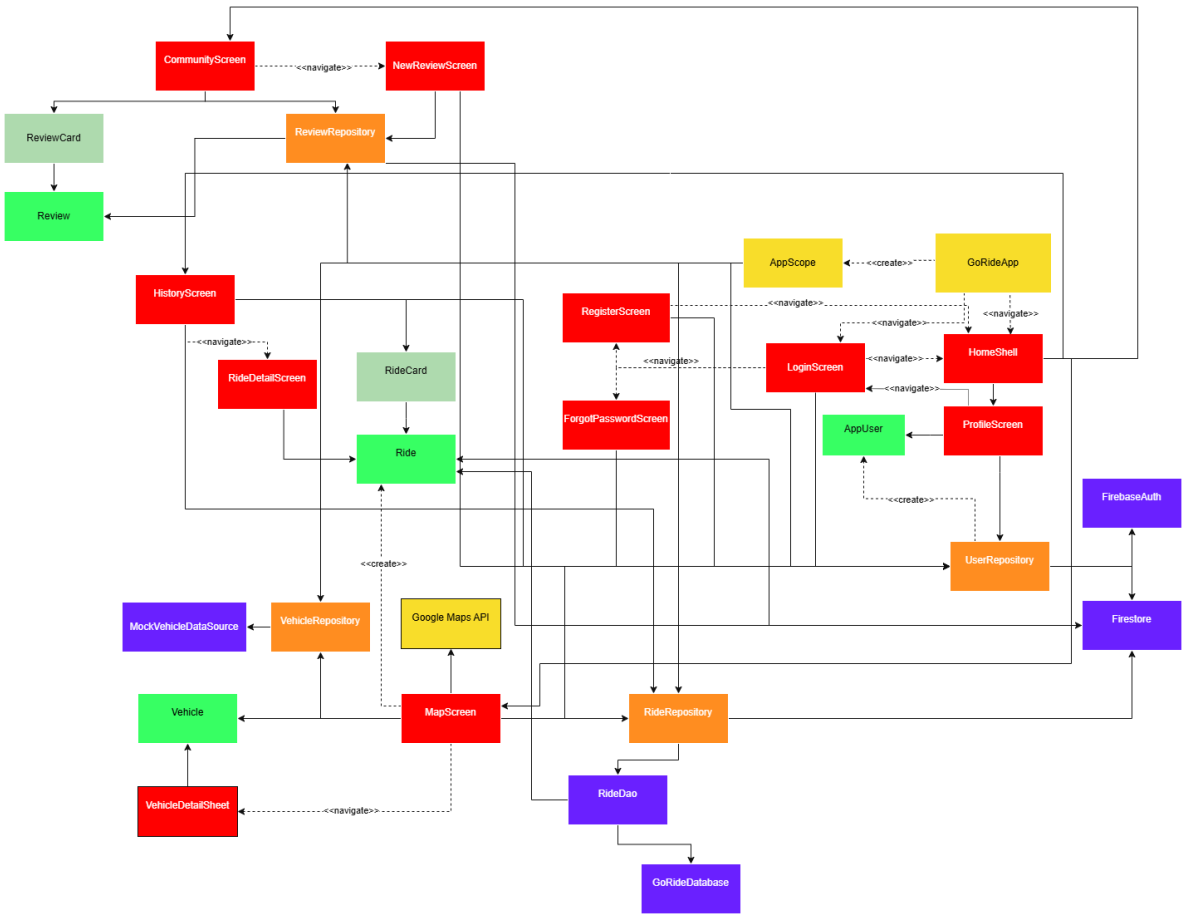


Figure 44: Mappa dell'Architettura

La mappa complessiva è poi suddivisa in cinque diagrammi di dettaglio, uno per area funzionale. Si usa la stessa notazione della versione Kotlin: freccia continua per l'associazione o l'utilizzo, freccia tratteggiata con stereotipo «navigate» per la navigazione tra schermate e «create» per la creazione di un oggetto. I colori distinguono i ruoli: rosso per le schermate, arancione per i Repository, verde per i modelli, verde chiaro per i widget delle liste, viola per le sorgenti di dati e i servizi esterni, giallo per il widget radice, l'AppScope e le Google Maps API.

4.4.1 Architettura dell'Avvio e della Navigazione

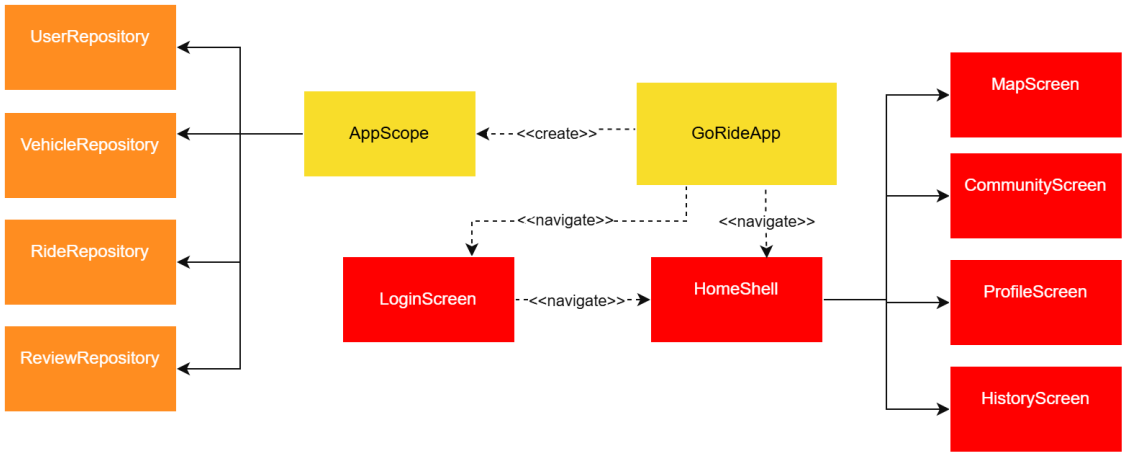


Figure 45: Architettura dell'avvio e della navigazione

GoRideApp è il widget radice dell'applicazione, definito in `main.dart`. All'avvio crea una sola volta i quattro Repository e li inserisce in AppScope, un `InheritedWidget` posto in cima all'albero dei widget: ogni schermata li ottiene con `AppScope.of(context)`, senza doverli ricevere nel costruttore. In questo modo esiste una sola istanza di ogni Repository, condivisa da tutta l'app.

Sotto AppScope si trova il `MaterialApp` con le rotte con nome. La rotta iniziale dipende dalla sessione: se l'utente è già autenticato si apre direttamente HomeShell, altrimenti LoginScreen, che dopo l'accesso porta anch'essa a HomeShell. HomeShell contiene la `BottomNavigationBar` e le quattro sezioni principali (mappa, storico, community e profilo), mostrate con un `IndexedStack` che le mantiene in vita quando l'utente cambia scheda. A differenza della versione Kotlin, la barra non va nascosta nelle schermate di accesso e registrazione: queste sono rotte separate, esterne a HomeShell.

4.4.2 Architettura di Autenticazione e Profilo

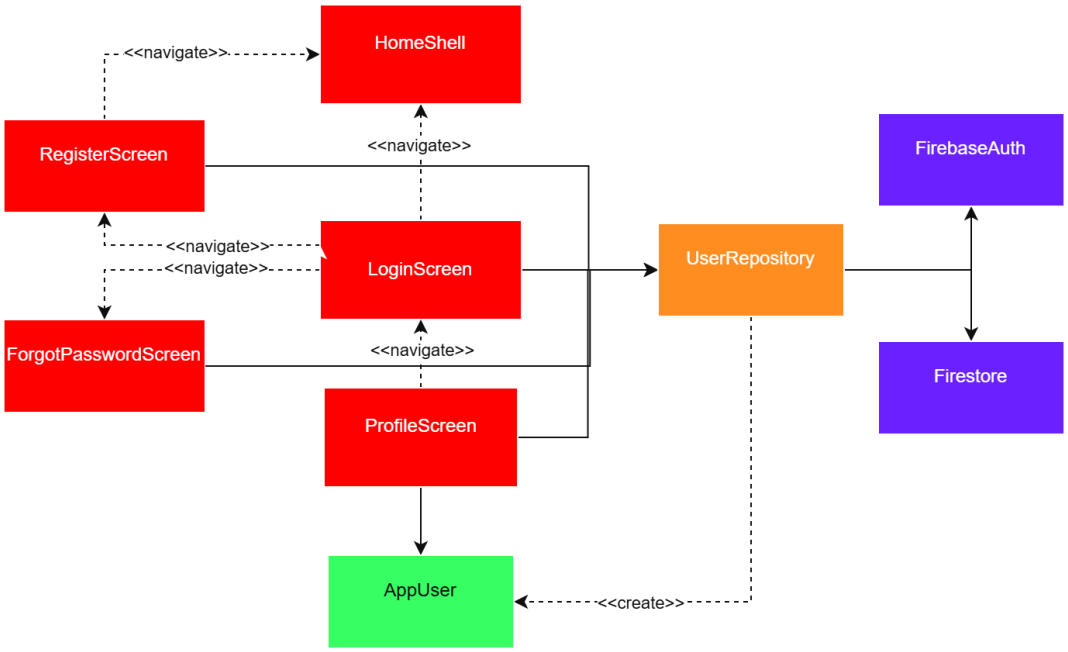


Figure 46: Architettura di autenticazione e profilo

Le quattro schermate di quest'area (LoginScreen, RegisterScreen, ForgotPasswordScreen e ProfileScreen) non hanno un ViewModel: la validazione dei campi e le chiamate asincrone si trovano nello State della schermata, che aggiorna l'interfaccia con `setState()`. Tutte si rivolgono a UserRepository, l'unica classe che comunica con Firebase Authentication, per login, registrazione, recupero della password e logout, e con Firestore, dove è salvato il profilo dell'utente (collezione `users`). La password è gestita esclusivamente da Firebase Authentication: non viene salvata in Firestore né inclusa nel modello AppUser. Solo ProfileScreen conosce il modello AppUser, che UserRepository crea a partire dal profilo letto da Firestore.

Gli errori di Firebase vengono tradotti da UserRepository in una `GoRideException` con un messaggio in italiano, che la schermata mostra in una SnackBar. Dopo un login o una registrazione riusciti l'utente viene portato a HomeShell, mentre il logout dal profilo riporta a LoginScreen svuotando la pila delle schermate. Quest'area realizza i requisiti RF7, RF7.1, RF8, RF8.1 e RF8.2.

4.4.3 Architettura della Mappa e del Noleggio

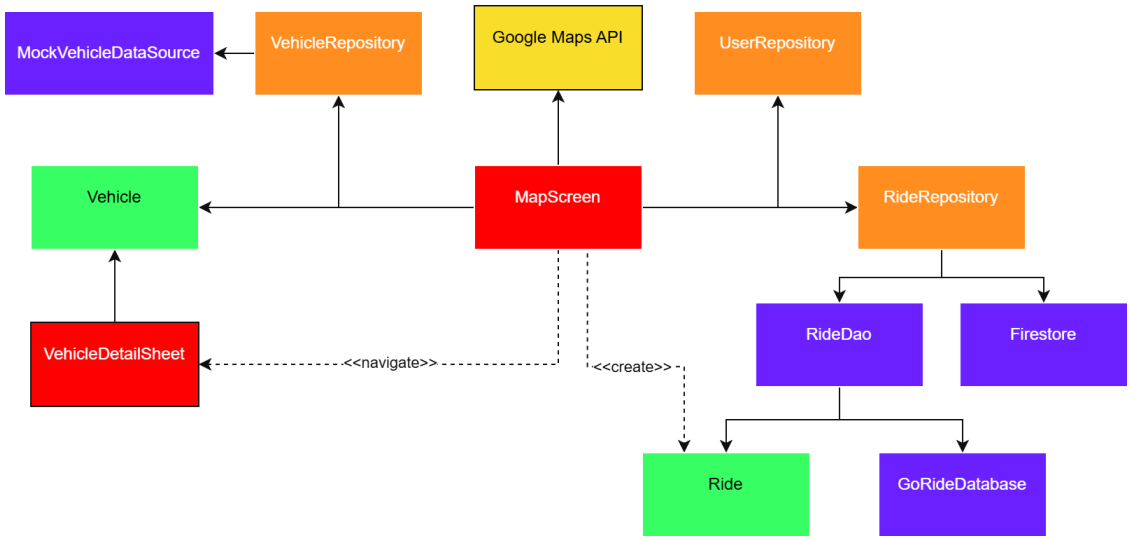


Figure 47: Architettura della mappa e del noleggio

MapScreen mostra la mappa tramite le Google Maps API e disegna un marker per ogni mezzo, verde per le biciclette classiche e arancione per le e-bike. I mezzi sono forniti da VehicleRepository, che li ottiene da MockVehicleDataSource, una sorgente in memoria con mezzi di esempio attorno ad Ancona; la lista arriva come `Future` ed è mostrata con un `FutureBuilder`. Toccando un marker si apre VehicleDetailSheet, un bottom sheet che riceve il Vehicle nel costruttore e mostra le tariffe e, per le e-bike, batteria e autonomia.

Se l'utente sceglie di noleggiare il mezzo, il bottom sheet si chiude restituendo `true` e MapScreen mostra la scelta della durata, con il prezzo di ciascuna. Confermata la durata, MapScreen crea un oggetto Ride già concluso, con istante di inizio, durata, tariffe e prezzo, e ottiene l'identificativo dell'utente da UserRepository. Il Ride viene passato a RideRepository, che lo salva prima nel database locale tramite RideDao (tabella `corse` di GoRideDatabase) e poi su Firestore (collezione `rides`). Rispetto alla versione Kotlin non esistono quindi una schermata della corsa né un ViewModel: il noleggio è gestito interamente da MapScreen. Quest'area realizza i requisiti da RF1 a RF6.

4.4.4 Architettura dello Storico Corse

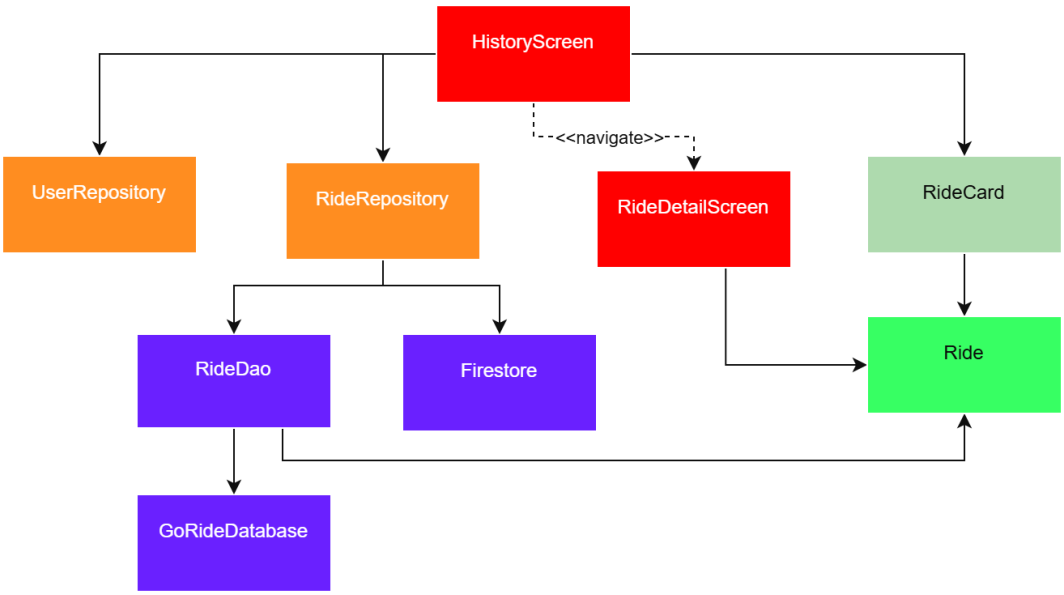


Figure 48: Architettura dello storico corse

HistoryScreen mostra l'elenco delle corse dell'utente, di cui ottiene l'identificativo da UserRepository. All'apertura chiede a RideRepository di sincronizzare le corse da Firestore (collezione `rides`) nel database locale, e poi le legge tramite RideDao dalla tabella `corse` di GoRideDatabase, che funge da cache: per questo lo storico è consultabile anche senza connessione. Il risultato è un `Future` mostrato con un `FutureBuilder`. HomeShell ricrea la schermata ogni volta che l'utente apre la sezione Storico, così i noleggi appena registrati compaiono subito.

Ogni corsa è mostrata con un widget RideCard, l'equivalente del RideAdapter della versione Kotlin. In Flutter non serve un Adapter: la lista è costruita da `ListView.separated` a partire direttamente dalla lista di Ride. Selezionando una corsa si apre RideDetailScreen, che riceve il Ride come argomento della rotta con nome e ne mostra durata, mezzo, prezzo e composizione del costo. Quest'area realizza i requisiti RF9 e RF10.

4.4.5 Architettura della Community

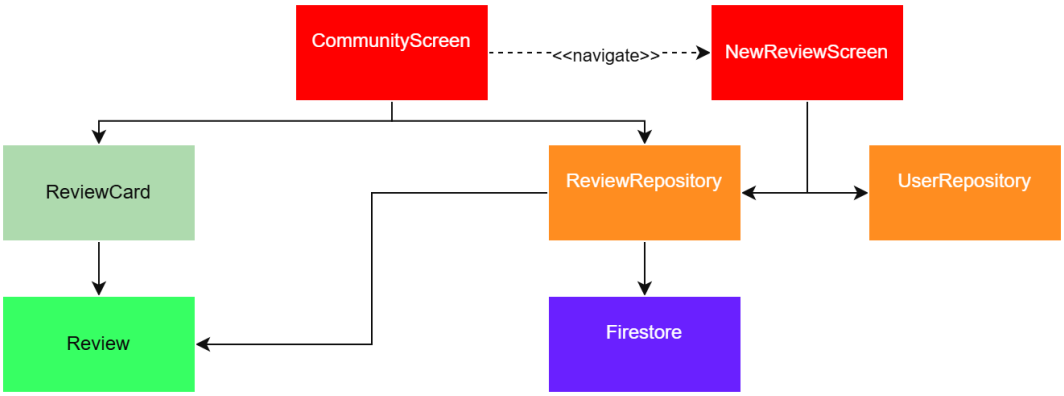


Figure 49: Architettura della community

CommunityScreen mostra le recensioni tramite il widget ReviewCard, che cambia aspetto in base al tipo di recensione (guasto, opinione o consiglio): un solo widget sostituisce i tre layout e l'Adapter della versione Kotlin. Le recensioni arrivano da ReviewRepository, che espone le ultime 100 recensioni della collezione `reviews` di Firestore come `Stream`, ottenuto con `snapshots()`. CommunityScreen lo mostra con uno `StreamBuilder`: quando un altro utente pubblica una recensione, anche dall'app Kotlin, la lista si aggiorna senza ricaricare la schermata. Quando la schermata viene chiusa, lo StreamBuilder cancella l'iscrizione e Firestore smette di ascoltare, come `fermaAscolto()` nella versione Kotlin.

Per scrivere una nuova recensione CommunityScreen porta a NewReviewScreen, che recupera il nome dell'autore da UserRepository e pubblica tramite ReviewRepository. Quest'area realizza i requisiti RF11, RF12 e RF13.

4.5 Database

Per la memorizzazione locale viene utilizzato SQLite tramite il pacchetto sqflite, visto a lezione. A differenza di Room, sqflite non genera codice: la tabella viene creata con un'istruzione SQL nel metodo `onCreate` di `openDatabase`, e le righe si leggono e si scrivono come `Map` con i metodi `query`, `insert` e `delete`. Il database `GoRideDatabase` ha un solo scopo: mantenere una copia locale dello storico, in modo che le corse siano consultabili anche senza connessione. Non serve invece la tabella `corsa_attiva` della versione Kotlin: senza timer non c'è una corsa in corso da ripristinare.

`GoRideDatabase` è un singleton. Il database viene aperto una sola volta e il `Future` dell'apertura viene conservato, così se due parti dell'app lo richiedono insieme ricevono la stessa connessione: è il ruolo di `@Volatile` e `synchronized` nella versione Kotlin. L'accesso alla tabella avviene tramite `RideDao`. La tabella `corse` ha le stesse colonne della tabella Room, riportate nella Tabella 2.

Tabella	Attributi
`corse`	`id`: TEXT (chiave primaria) `userId`: TEXT `vehicleId`: TEXT `vehicleNome`: TEXT `vehicleTipo`: TEXT `startTimestamp`: INTEGER `endTimestamp`: INTEGER (nullable) `minutiPrepagati`: INTEGER `tariffaSbloccoEuro`: REAL `tariffaAlMinutoEuro`: REAL `costoTotaleEuro`: REAL

Table 2: Struttura della tabella nel database sqflite

Non esiste una classe Entity separata: è il modello `Ride` a convertirsi da e verso una riga con `toMap()` e `fromMap()`, gli stessi metodi usati anche per i documenti di Firestore. Il tipo del mezzo è salvato come stringa (`CLASSIC` o `EBIKE`), perché SQLite non gestisce le enumerazioni. Gli inserimenti usano `ConflictAlgorithm.replace`, così una corsa già presente viene sovrascritta, e la sincronizzazione con Firestore scrive tutte le corse in un'unica operazione con un `batch`.

4.5.1 Firebase

L'utilizzo di Firebase è lo stesso dell'app Kotlin (sezione 3.4.2): stesse collezioni `users`, `rides` e `reviews`, con gli stessi campi. Per la configurazione, la versione Flutter usa il file `firebase_options.dart`, generato con i dati dello stesso progetto, al posto di `google-services.json`.

4.6 Mockup UI

Le schermate della versione Flutter riprendono la grafica dell'app Kotlin (colori, icone e disposizione degli elementi). Si riportano in orientamento verticale; la schermata della corsa con il timer è sostituita dalla finestra di scelta della durata del noleggio.

4.6.1 NavBar

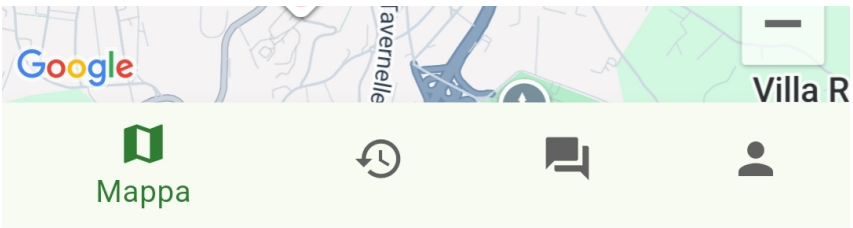


Figure 50: Navbar

4.6.2 Sezione Account

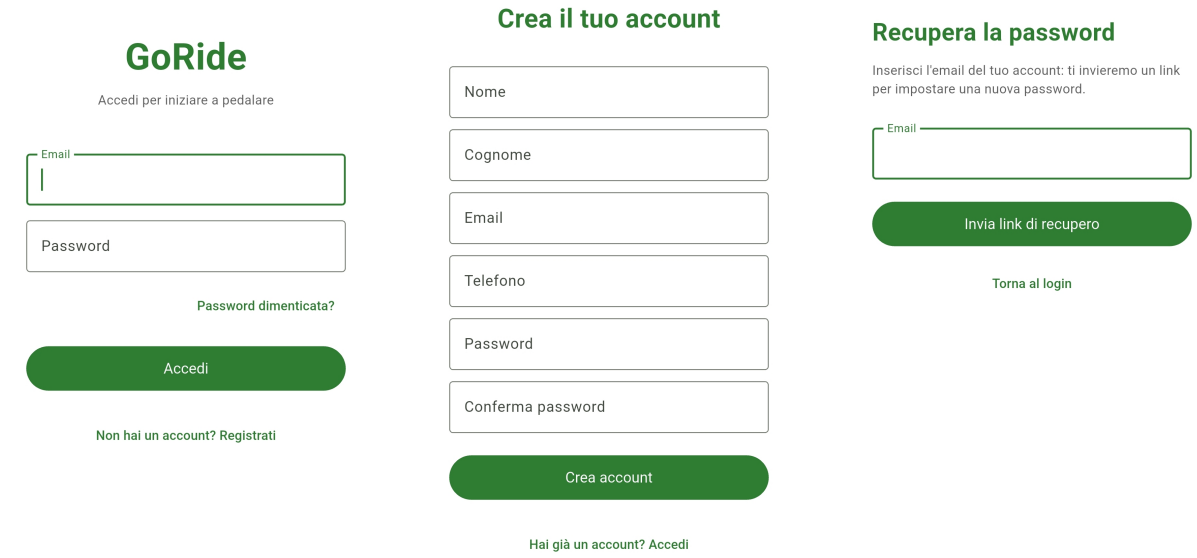


Figure 51: Accesso, Registrazione e Recupero Password

4.6.3 Sezione Mappa

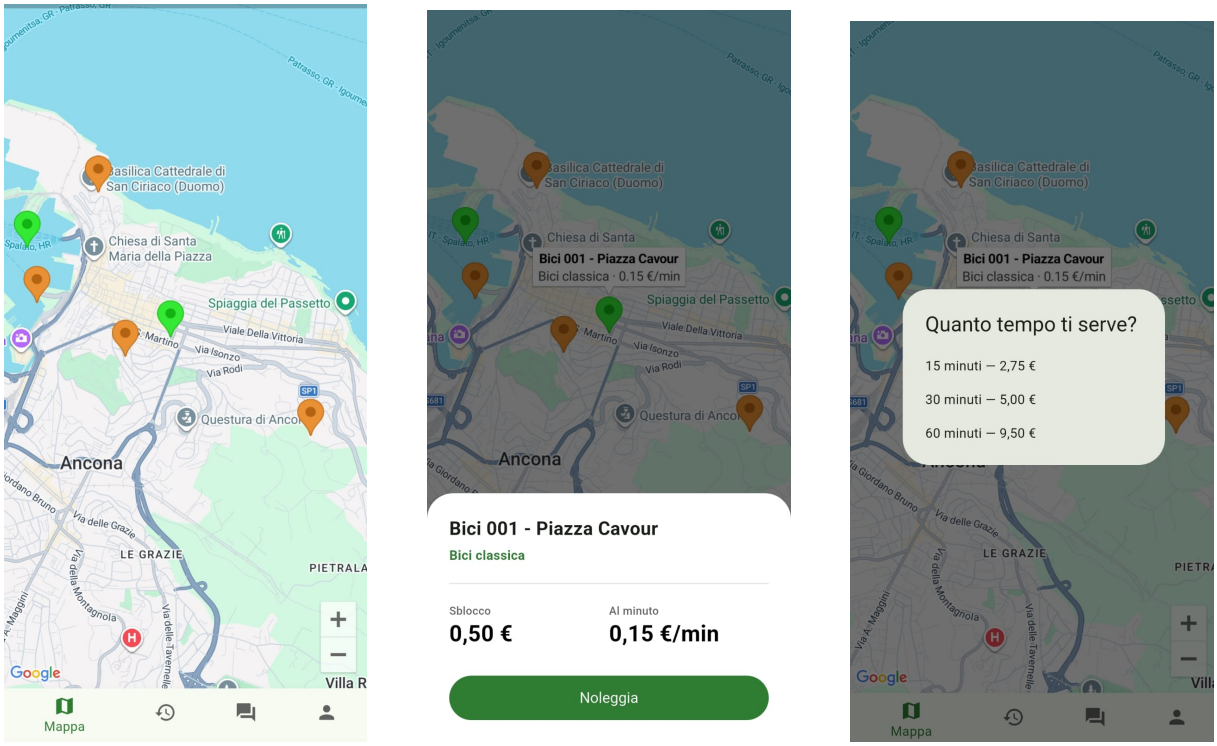


Figure 52: Mappa, Dettaglio Mezzo e Scelta della Durata del Noleggio

4.6.4 Sezione Storico

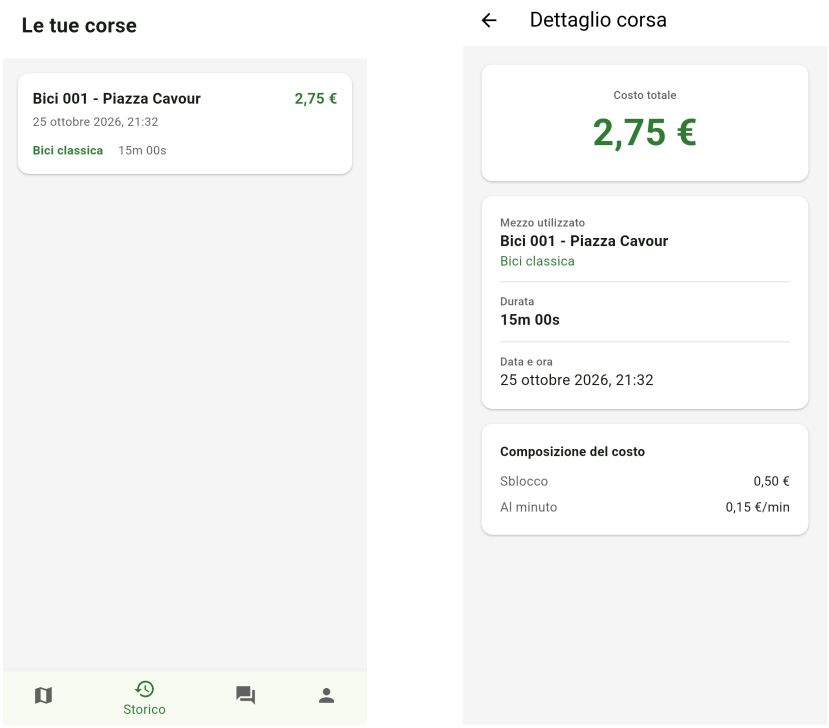


Figure 53: Storico e Dettaglio Corsa

4.6.5 Sezione Community

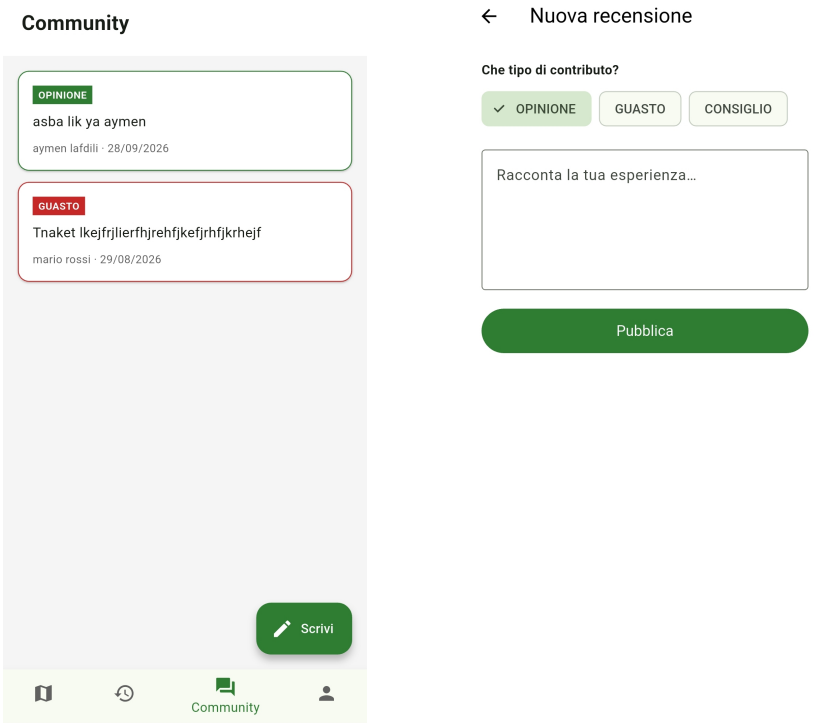


Figure 54: Community e Nuova Recensione

4.6.6 Sezione Profilo

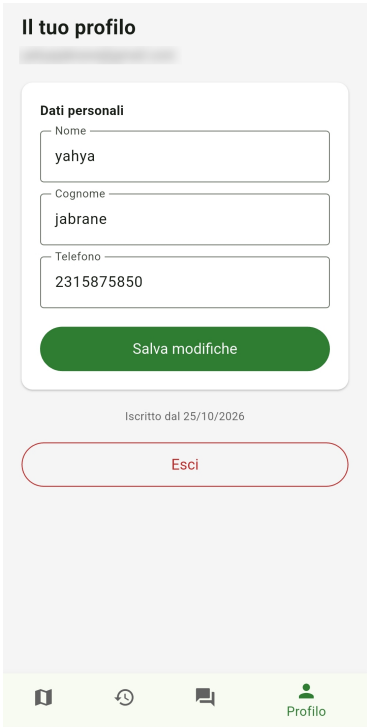


Figure 55: Profilo

4.7 Rappresentazione Grafica dei Casi d'Uso

4.7.1 Accesso e creazione account

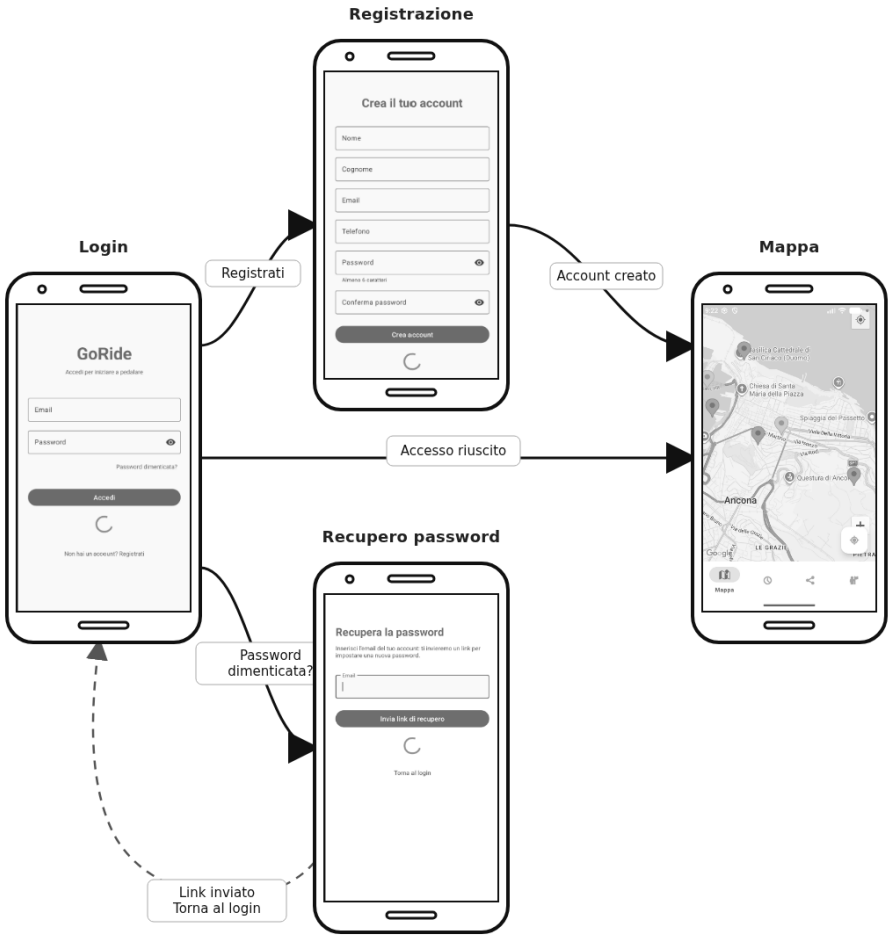


Figure 56: Accesso e creazione account

4.7.2 Noleggio di un mezzo

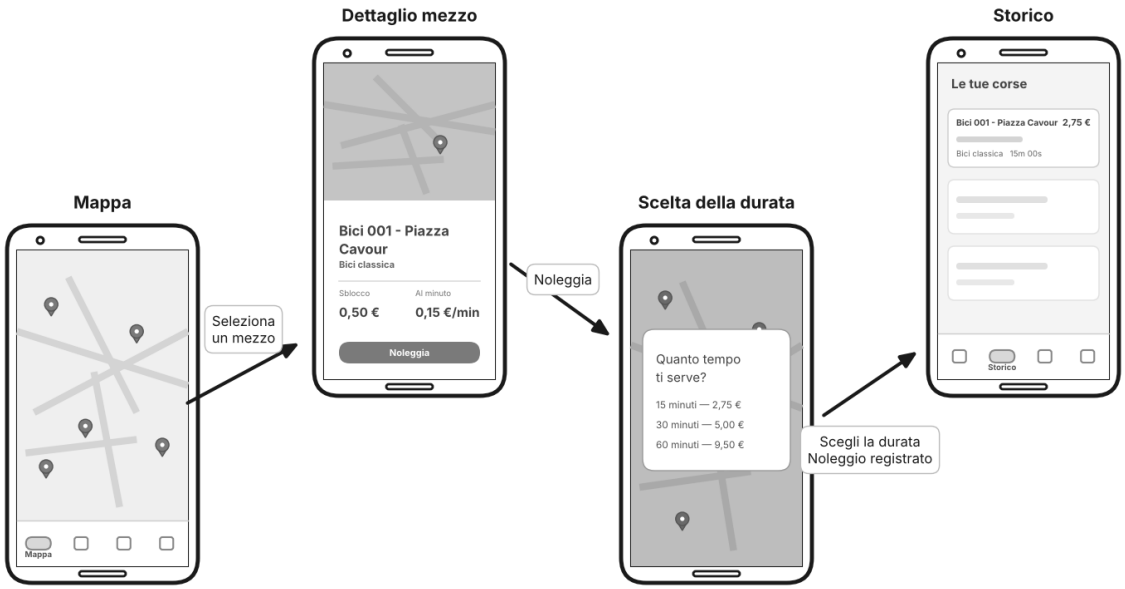


Figure 57: Noleggio di un mezzo

4.7.3 Consultazione storico

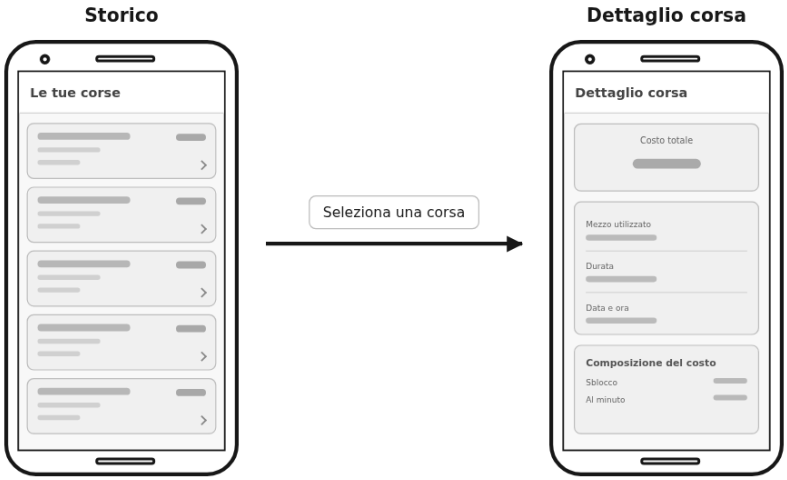


Figure 58: Consultazione storico

4.7.4 Pubblicazione recensione

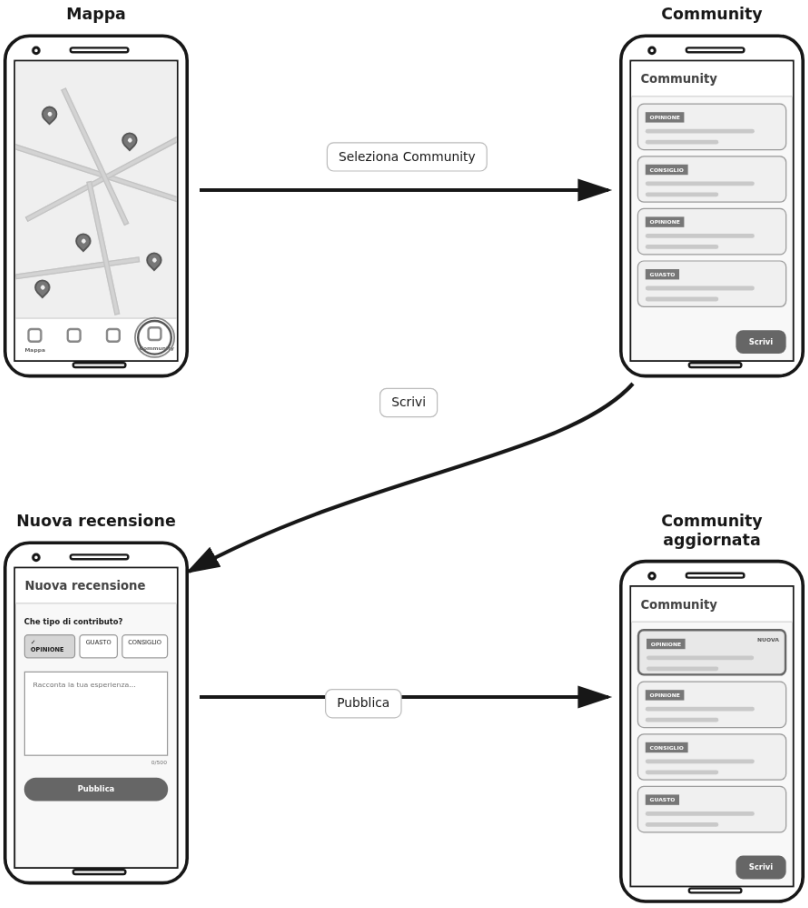


Figure 59: Pubblicazione recensione

4.7.5 Modifica profilo e logout

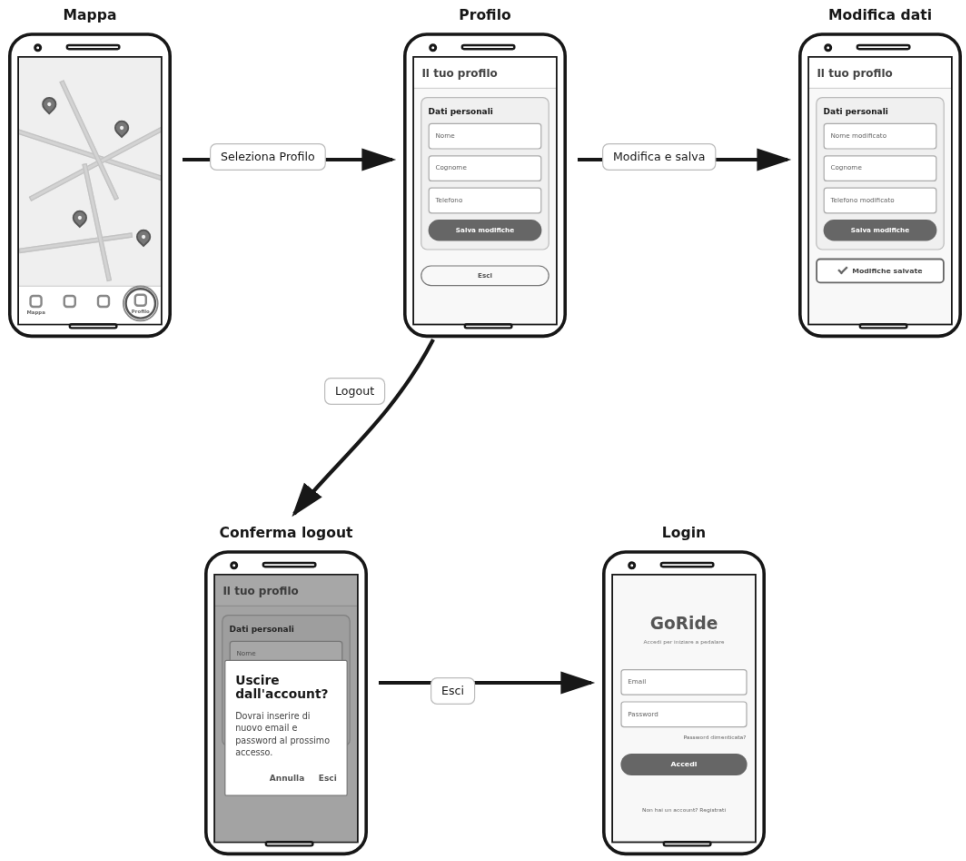


Figure 60: Modifica profilo e logout

4.8 Commenti sull'Approccio di Sviluppo

4.8.1 Componenti Importate

Oltre ai widget del framework Flutter, sono stati aggiunti alcuni pacchetti esterni, scaricati da pub.dev con il comando `flutter pub add` e dichiarati nel file `pubspec.yaml`. Sqflite e path sono quelli visti a lezione per il database locale; Firebase e Google Maps sono le stesse librerie usate nella versione Kotlin, nella loro versione per Flutter.

sqflite e **path**, per il database locale SQLite che contiene lo storico delle corse. `path` serve a costruire il percorso del file del database con la funzione `join`.

```
flutter pub add sqflite path
```

Google Maps per Flutter, per la visualizzazione della mappa e dei marker dei mezzi in MapScreen. Come nella versione Kotlin, la chiave API non è nel codice ma nel file `local.properties`, e viene inserita nel manifest tramite un placeholder di Gradle.

```
flutter pub add google_maps_flutter
```

Firebase (Core, Authentication e Cloud Firestore), per account, profili, corse e recensioni. `firebase_core` inizializza la connessione al progetto Firebase all'avvio dell'app, mentre gli altri due pacchetti corrispondono a `firebase-auth` e `firebase-firestore` della versione Kotlin. Le API di questi pacchetti restituiscono già `Future` e `Stream`, quindi si usano direttamente con `async`/`await` e con `StreamBuilder`, senza dover convertire le callback.

```
flutter pub add firebase_core firebase_auth cloud_firestore
```

Le dipendenze risultanti nel file `pubspec.yaml` sono:

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  firebase_core: ^4.15.0
  firebase_auth: ^6.7.0
  cloud_firestore: ^6.10.0
  google_maps_flutter: ^2.18.2
  sqflite: ^2.4.4
  path: ^1.9.1
```

4.9 Descrizione delle Componenti

Tutte le schermate seguono lo stesso schema: sono `StatefulWidget` e leggono i Repository dall'AppScope in `didChangeDependencies()`, perché in `initState()` l'`InheritedWidget` non è ancora accessibile. Dopo ogni `await` controllano `mounted` prima di usare il `context`, perché nel frattempo la schermata potrebbe essere stata chiusa. Per questo, nelle descrizioni sotto, questi passaggi non vengono ripetuti.

4.9.1 Main e GoRideApp

`Future<void> main()`

Punto di ingresso dell'applicazione.

`main()` : Chiama `WidgetsFlutterBinding.ensureInitialized()`, inizializza Firebase con `Firebase.initializeApp()` e avvia l'app con `runApp()`.

`class GoRideApp extends StatefulWidget`

Widget radice. È Stateful perché crea una sola volta i quattro Repository e li mantiene per tutta la durata dell'app.

`build()` : Restituisce l'AppScope con i Repository, che contiene il `MaterialApp` con il tema e le rotte con nome. La rotta iniziale è la HomeShell se `UserRepository.isLoggato` è vero, altrimenti la schermata di login.

4.9.2 AppScope e Routes

`class AppScope extends InheritedWidget`

Mette a disposizione di tutte le schermate i quattro Repository, senza passarli nei costruttori.

`static AppScope of(context)` : Restituisce l'AppScope più vicino nell'albero dei widget con `dependOnInheritedWidgetOfExactType()`.

`updateShouldNotify()` : Restituisce sempre `false`, perché i Repository non cambiano mai.

`class Routes`

Contiene i nomi delle rotte (`/login`, `/register`, `/forgot-password`, `/home`, `/ride-detail`, `/new-review`): sostituisce il `nav_graph` della versione Kotlin.

4.9.3 HomeShell

`class HomeShell extends StatefulWidget`

Contenitore delle quattro sezioni principali con la `BottomNavigationBar`.

`_onVoceToccata(indice)` : Con `setState()` cambia la sezione visibile. Se l'utente tocca Storico, incrementa un contatore usato come `key` di HistoryScreen, così la schermata viene ricreata e rilegge le corse.

`build()` : Usa un `IndexedStack`, che mantiene in vita le sezioni e ne mostra una sola. Le sezioni vengono costruite solo la prima volta che l'utente le apre.

4.9.4 Modelli

`class Vehicle`

Mezzo disponibile al noleggio: id, nome, tipo, coordinate, tariffe, disponibilità e, solo per le e-bike, batteria e autonomia (nullable). Non serve `Parcelable`: viaggia tra le schermate come argomento della rotta o nel costruttore.

`bool get isEbike` : Vero se il tipo è `VehicleType.ebike`.

`enum VehicleType`

`classic` o `ebike`, con il codice salvato su Firestore (`CLASSIC`, `EBIKE`).

`static daCodice(codice)` : Converte il codice letto dal database nell'enumerazione.

`class Ride`

Corsa conclusa: id, utente, mezzo, istanti di inizio e fine, minuti prepagati, tariffe e costo totale.

`int get durataSecondi` : Durata in secondi, calcolata da inizio e fine.

`toMap() / Ride.fromMap(id, map)` : Conversione da e verso una riga di SQLite o un documento di Firestore.

`class Review e enum ReviewType`

Recensione della community: id, autore, tipo (`guasto`, `opinione`, `consiglio`), testo, istante di pubblicazione e mezzo opzionale.

`class AppUser`

Profilo utente: uid, nome, cognome, email, telefono e data di registrazione. La password non compare mai.

`String get nomeCompleto` : Nome e cognome uniti.

4.9.5 GoRideDatabase e RideDao

`class GoRideDatabase`

Database sqflite dell'app, con la sola tabella `corse`. È un singleton.

`static getDatabase()` : Apre il database alla prima chiamata e conserva il `Future` dell'apertura, così le chiamate successive ricevono la stessa connessione. La tabella viene creata in `onCreate`.

`class RideDao`

Accesso alla tabella `corse`.

`getCorseUtente(userId)` : Corse dell'utente dalla più recente, con `query` e `orderBy: 'startTimestamp DESC'`. Il parametro `?` con `whereArgs` evita la SQL injection.

`inserisci(corsa) / inserisciTutte(corse)` : Inserimento con `ConflictAlgorithm.replace`. `inserisciTutte` usa un `batch` per scrivere tutte le corse in un'unica operazione.

4.9.6 VehicleRepository e MockVehicleDataSource

`class VehicleRepository`

Unica porta d'accesso ai mezzi, forniti da `MockVehicleDataSource`: una sorgente in memoria con l'elenco fisso dei mezzi attorno ad Ancona, le coordinate del centro e lo zoom iniziale.

`getMezziDisponibili()` : Restituisce i mezzi disponibili dopo un ritardo di 600 ms, che simula la rete.

4.9.7 UserRepository

`class UserRepository`

Unica classe che usa Firebase Authentication e la collezione `users`. Le API di Firebase restituiscono già `Future`, quindi si usano con `await`. Gli errori sono tradotti in una `GoRideException` con un messaggio in italiano.

`isLoggato / uidCorrente` : Dicono se c'è una sessione attiva e ne restituiscono l'uid.

`login(email, password)` : Accesso con email e password.

`registra(...)` : Crea l'account in Authentication e poi salva il profilo in Firestore, senza la password.

`inviaEmailRecupero(email)` : Chiede a Firebase di inviare la mail di recupero.

`getProfilo() / aggiornaProfilo(...)` : Leggono il profilo come AppUser e aggiornano nome, cognome e telefono.

`logout()` : Chiude la sessione.

4.9.8 RideRepository

`class RideRepository`

Gestisce le corse con due sorgenti: Firestore come fonte remota e sqflite come cache.

`getStorico(userId)` : Legge lo storico dal database locale.

`sincronizzaDaFirestore(userId)` : Scarica le corse dell'utente da Firestore e le inserisce nel database locale; i documenti malformati vengono scartati.

`salvaCorsa(ride)` : Scrive la corsa prima nel database locale e poi su Firestore.

4.9.9 ReviewRepository

`class ReviewRepository`

Gestisce la collezione `reviews` di Firestore. Non usa il database locale.

`osservaRecensioni()` : Restituisce le ultime 100 recensioni, ordinate per data, come `Stream` ottenuto con `snapshots()`. Il listener resta attivo finché qualcuno è iscritto allo Stream.

`pubblicaRecensione(...)` : Salva una nuova recensione con id casuale e istante corrente.

`class GoRideException implements Exception`

Errore con un messaggio in italiano, lanciato dai Repository al posto delle eccezioni di Firebase, così le schermate non conoscono i tipi di Firebase.

4.9.10 ACCOUNT: LoginScreen

`class LoginScreen extends StatefulWidget`

Schermata di accesso, con due `TextEditingController` per email e password.

`_accedi()` : Valida i campi e chiama `UserRepository.login()`. Durante l'attesa disattiva il pulsante con `setState()`. Se l'accesso riesce va alla HomeShell con `pushReplacementNamed()`, altrimenti mostra l'errore in una SnackBar.

4.9.11 ACCOUNT: RegisterScreen

`class RegisterScreen extends StatefulWidget`

`_valida()` : Controlla campi vuoti, formato email, telefono di almeno 8 cifre, password di almeno 6 caratteri e conferma. Restituisce il messaggio d'errore o `null`.

`_registrati()` : Chiama `UserRepository.registra()` e va alla HomeShell svuotando la pila delle schermate.

4.9.12 ACCOUNT: ForgotPasswordScreen

`class ForgotPasswordScreen extends StatefulWidget`

`_invia()` : Valida l'email e chiama `UserRepository.inviaEmailRecupero()`. Mostra un messaggio generico, che non rivela se l'email è registrata, e dopo 2,5 secondi torna al login.

4.9.13 PROFILO: ProfileScreen

`class ProfileScreen extends StatefulWidget`

`_caricaProfilo() / _salvaModifiche()` : Leggono il profilo all'apertura e salvano le modifiche a nome, cognome e telefono.

`_confermaLogout()` : Chiede conferma con un `AlertDialog`, poi chiude la sessione e torna al login.

4.9.14 MAPPA: MapScreen

`class MapScreen extends StatefulWidget`

Mappa con i mezzi e gestione del noleggio. Il `Future` dei mezzi viene creato una sola volta e mostrato con un `FutureBuilder`, che visualizza lo spinner o il messaggio d'errore con "Riprova".

`_creaMarker(mezzi)` : Un marker per mezzo, verde per le bici classiche e arancione per le e-bike.

`_apriDettaglio(mezzo)` : Apre VehicleDetailSheet con `showModalBottomSheet()`. Se restituisce `true` passa al noleggio.

`_noleggia(mezzo)` : Mostra le durate di 15, 30 e 60 minuti con il prezzo in un `SimpleDialog`. Crea il Ride con la durata scelta e lo salva con `RideRepository.salvaCorsa()`.

4.9.15 MAPPA: VehicleDetailSheet

`class VehicleDetailSheet extends StatelessWidget`

Riceve il Vehicle nel costruttore e mostra tariffe e, per le e-bike, batteria e autonomia. Il pulsante "Noleggia" chiude il pannello restituendo `true`.

4.9.16 STORICO: HistoryScreen

`class HistoryScreen extends StatefulWidget`

`_carica()` : Sincronizza le corse da Firestore e poi le legge dal database locale. Il risultato è mostrato con un `FutureBuilder` come lista di RideCard; il tocco su una corsa apre RideDetailScreen.

4.9.17 STORICO: RideCard e RideDetailScreen

`class RideCard extends StatelessWidget`

Riga dello storico con costo, data, durata e tipo di mezzo.

`class RideDetailScreen extends StatelessWidget`

Legge il Ride dagli argomenti della rotta e mostra costo, mezzo, durata, data e composizione del costo.

4.9.18 COMMUNITY: CommunityScreen

`class CommunityScreen extends StatefulWidget`

Ottiene una sola volta lo Stream delle recensioni e lo mostra con uno `StreamBuilder`, sempre in ascolto: la lista si aggiorna da sola quando qualcuno pubblica. Il pulsante "Scrivi" apre NewReviewScreen.

4.9.19 COMMUNITY: NewReviewScreen

`class NewReviewScreen extends StatefulWidget`

Il tipo si sceglie con tre `ChoiceChip`, aggiornati con `setState()`.

`_pubblica()` : Controlla che il testo abbia almeno 10 caratteri, legge il nome dell'autore dal profilo e chiama `ReviewRepository.pubblicaRecensione()`.

4.9.20 COMMUNITY: ReviewCard

`class ReviewCard extends StatelessWidget`

Mostra autore, data, tipo e testo di una recensione, con un colore diverso per ogni tipo: rosso per guasto, verde per opinione, arancione per consiglio.